Bahir Dar University

Bahir Dar Institute of Technology Faculty of Computing

Requirement Analysis Document (RAD) and Requirement Analysis Document (RAD)

For

Industrial project on Innovation & Incubation Management System(IIMS)

Submitted to the faculty of computing in partial fulfillment of the requirements for the degree of Bachelor of Science in Software Engineering

Group members

Name	IDNumber
1. Arsema Degu	**1405454**
2. Tsion Bantegize	**1404976**
3. Rediet Berhanu	**1404976**
4. Tsedenya Emkedemealem	**1404499**

Advisor : MS. Yeneguse

2025

Bahir Dar University, Bahir Dar Institute of Technology

Declaration

The Project is our own and has not been presented for a degree in any other university and all the sources of material used for the project have been duly acknowledged.

Name	**Signature**
Arsema Degu	**Signature**
Tsion Bantegize	**Signature**
Rediet Berhanu	**Signature**
Tsedenya Emkedemealem	**Signature**

Faculty: Computing

Program: Software Engineering

Project Title: Innovation & Incubation Management System(IIMS)

This is to certify that I have read this project and that in my supervision and the students' performance, it is fully adequate, in scope and quality, as a project for the degree of Bachelor of Science.

MS. Yeneguse	
Name of Advisor	**Signature**

Examining committee members	**signature**	**Date**
1. Examiner1		
2. Examiner2		

It is approved that this project has been written in compliance with the formatting rules laid down by the faculty.

Roles and Responsibilities of the Group Members

List of Tasks	Arsema Degu	Tsion Bantegize	Rediet Berhanu	Tsedenya Emkedemealem
Requirement Gathering & Analysis	x	x	x	x
Use Case Modeling & Descriptions		x		x
Class Diagram Design	x		x	
Sequence & Activity Diagrams			x	
Interface Prototype (UI/UX)	x	x		
Pseudocode & Algorithm Design				x
System Validation & Consistency	x	x	x	x
System design (architecture, UI,		x	x	

database)				
Backend development (Spring Boot, MVC)	x		x	x
Frontend development (React UI, Dashboards)		x	x	x
Database design and implementation (PostgreSQL)	x	x		
Integration and deployment (Docker, REST APIs)			x	x
Testing and evaluation (Security, Functional)	x	x	x	x
Final Documentation Formatting	x	x	x	x

Acknowledgment

We would like to express our sincere gratitude to all individuals and organizations who contributed to the successful completion of this project entitled **Incubation and Innovation Management System (IIMS)**.

First and foremost, we extend our heartfelt appreciation to our academic advisors and instructors for their continuous guidance, valuable feedback, and encouragement throughout the project lifecycle. Their technical insights and academic support played a significant role in shaping the quality and direction of this work.

We are also grateful to the management and staff of **BiTec (BiT Business Incubation & Techno-Entrepreneurship Center)** at Bahir Dar Institute of Technology for allowing us to observe their incubation processes and for sharing practical insights that inspired the development of this system. Our appreciation further extends to **Orbit Innovation Hub, InnobizK, Ministry of Labor and Skills (MoLS – Ethiopia), and DxValley Incubation Hub** for their cooperation and willingness to provide information during requirement gathering.

Special thanks go to startup founders, mentors, and administrators who participated in interviews and discussions, providing real-world perspectives that helped us understand the challenges faced by incubation centers.

Finally, we acknowledge the efforts and collaboration of all group members whose dedication, teamwork, and commitment made the successful completion of this project possible.

List of acronyms

Acronym	Expanded Form
IIMS	Innovation & Incubation Management System
BiT	Bahir Dar Institute of Technology
BiTech	BiT Business Incubation & Techno-Enterpreneurship
OOAD	Object-Oriented Analysis and Design
UML	Unified Modeling Language
DBMS	Database Management System
JWT	JSON Web Token
API	Application Program Interface
ER	Entity Relation
CSV	Comma-Separated Value
MVC	Model-View-Control
RBAC	Role-Based Access Control
API	Application Programming Interface
PDF	Portable Document Format
UX	User Experience
UI	User Interface
MoLS	Ministry of Labor and Skills
DB	Database
OAuth	Open Authorization
JSON	JavaScript Object Notation

List of Figures

List of Tables

Table of Contents

Abstract

Incubation and innovation centers play a critical role in supporting early-stage startups by providing mentorship, training, resources, and structured incubation programs. However, many of these centers still rely on manual or semi-automated processes to manage applications, mentor assignments, startup progress tracking, and reporting. These practices lead to inefficiencies, data fragmentation, limited transparency, and increased administrative workload.

This project presents the design and development of a web-based Incubation and Innovation Management System (IIMS) that automates and centralizes the core operations of incubation and innovation centers. The system supports multi-tenant incubation centers, enabling each organization to manage its programs independently while maintaining data security and role-based access control. IIMS provides functionalities for startup application management, mentor assignment, progress tracking, assessments, communication, scheduling, and reporting through a unified digital platform.

A working prototype of IIMS was implemented, covering tenant management, subscription management, application forms and application processing, startup and mentor dashboards, meeting and calendar scheduling, chat and contact management, progress tracking, assessment management, and help center management. These modules were integrated and validated through functional and user-based testing to ensure correctness, usability, and system reliability.

The system was developed using Object-Oriented Analysis and Design (OOAD) methodology and implemented with modern web technologies including Java, Spring Boot, React, and PostgreSQL. Requirement gathering was conducted through interviews, observations, document analysis, and site visits to several incubation centers. The proposed system improves efficiency, transparency, coordination, and decision-making while reducing manual workload and operational delays.

Future extensions of the system include startup resources and news management, landing page builder, analytics and reporting, investor and alumni dashboards, graduation management, and advanced AI-powered support tools. These AI tools are envisioned to assist startups in business plan preparation, mentor matching, and other incubation-related tasks, providing intelligent recommendations, predictive insights, and automated guidance to enhance decision-making and operational efficiency.

Overall, IIMS contributes to the digital transformation of incubation centers by providing a scalable, secure, and user-friendly solution that enhances startup support and strengthens the innovation ecosystem.

Chapter One: Introduction

1.1. Background

Incubation and innovation centers are established to support early-stage startups and innovators by providing structured programs, mentorship, training, access to resources, and networking opportunities. These centers play a vital role in transforming innovative ideas into viable businesses, contributing to economic growth, job creation, and technological advancement. As startup ecosystems expand, incubation centers are required to manage increasing numbers of applications, startups, mentors, programs, and performance indicators, which makes efficient management systems essential.

The Incubation and Innovation Management System (IIMS) is designed to address the operational and managerial needs of incubation and innovation centers. The system targets organizations such as university-based incubation centers, government-supported innovation hubs, and private accelerators. These organizations typically consist of a management team (director or coordinator), program officers, mentors, administrative staff, and external stakeholders such as investors and partners. While the year of establishment, location, and number of employees vary across centers, most share a similar organizational structure and operational challenges related to managing startups and incubation programs effectively.

The mission of incubation centers is generally to nurture innovative ideas and support startups toward sustainability and growth. Their vision often focuses on building a strong innovation ecosystem and producing competitive startups that contribute to national and regional development. Core values commonly include innovation, collaboration, transparency, accountability, and impact-driven support. To fulfill these goals, incubation centers must efficiently manage applications, mentor relationships, startup progress, assessments, and reporting tasks that require reliable and well-structured information systems.

IIMS automates the core business processes of incubation centers by providing a centralized, digital platform. The system supports tenant-based incubation centers, allowing each center to manage its own programs independently. It enables startups to apply through dynamic application forms, allows administrators to review and assess applications, assigns mentors to startups, tracks startup progress through defined milestones, and generates reports for monitoring and decision-making. By integrating these processes into a single system, IIMS replaces fragmented manual workflows with an organized, transparent, and scalable digital solution.

1.2. Statement of the problem

Despite the critical role of incubation and innovation centers, many currently rely on manual or semi-automated systems to manage their operations. Application submissions are often handled through emails or paper-based forms, startup data is stored in spreadsheets, and progress tracking is conducted through informal reports or meetings. Administrators reported that reviewing applications manually could take **several weeks**, delaying startup selection and program initiation. To address this challenge, the system introduces **automated application management and centralized data storage**, reducing processing time and improving accessibility.

One major problem is the lack of centralized information management. Startup profiles, mentor data, application records, and assessment results are stored across different platforms or files, making it difficult to access accurate and up-to-date information. This fragmentation leads to data inconsistency, duplication, and increased risk of information loss. Administrators spend significant time searching for records instead of focusing on strategic support activities. To address this issue, the system introduces **multi-tenant centralized information management with role-based access**, ensuring accurate, consistent, and secure data availability for all stakeholders.

Another challenge is inefficient application and progress tracking processes. Without standardized digital workflows, monitoring startup progress and conducting assessments becomes slow and error-prone. Tracking application status, mentor assignments, progress milestones, and assessment results requires repeated meetings and manual follow-ups. Administrators reported that monitoring startup progress could take **several hours per week per startup**, limiting their ability to provide timely support. To address this challenge, the system introduces **structured progress tracking and assessment management modules**, enabling tenant admins to efficiently record, monitor, and evaluate startups' incubation progress.

Mentor assignment is also a manual process handled by tenant administrators, which can be time-consuming and prone to inconsistencies. While matching is not automated, the system provides **dashboards and tools to simplify mentor assignment, monitor mentor–startup engagement, and facilitate communication**, reducing administrative effort.

The effects of these problems include increased administrative workload, delayed decision-making, reduced accountability, poor communication among stakeholders, and limited ability to measure program impact. As

the number of startups and programs grows, these inefficiencies become more severe, making current systems unsustainable. Therefore, there is a clear need for a comprehensive, automated, and scalable system like IIMS to bridge the gap between incubation center objectives and their operational capabilities.

1.3. Objectives

The objective of this project is to design and develop a software system that addresses the operational challenges faced by incubation and innovation centers. The Incubation and Innovation Management System (IIMS) aims to digitize and integrate core incubation processes in order to improve efficiency, transparency, coordination, and decision-making. The objectives are categorized into a general objective and specific objectives that collectively contribute to the successful implementation of the system within the defined project timeframe.

1.3.1. General Objective

The general objective of this project is to **design and develop a web-based Incubation and Innovation Management System (IIMS) that automates and centralizes the management of incubation center operations**, including startup application handling, mentor assignment, progress tracking, assessments, communication, and reporting, within the given academic project duration.

1.3.2. Specific Objectives

The specific objectives of the project are to:

- **Analyze the existing incubation management processes** to identify gaps, inefficiencies, and user requirements related to application handling, startup monitoring, and reporting.
- **Design a centralized and scalable system architecture** that supports multiple incubation centers (tenants) while ensuring proper data isolation and role-based access control.
- **Develop a dynamic application management module** that enables incubation centers to create customized application forms and allows startups to submit applications electronically.
- **Implement role-based user management** for different stakeholders, including administrators,

startups, mentors, investors, and staff, with clearly defined permissions and responsibilities.

- **Design and implement a mentor management and assignment mechanism** that matches mentors to startups based on expertise, availability, and startup needs.
- **Develop a startup progress tracking module** that allows startups to submit milestone updates and enables administrators and mentors to monitor progress in real time.
- **Implement an assessment and evaluation system** that supports structured reviews, scoring, feedback, and decision tracking throughout the incubation lifecycle.
- **Provide communication and notification features** to improve coordination among startups, mentors, and administrators.
- **Design and implement role-based dashboards** that provide incubation center administrators, mentors, and startups with real-time insights, and ensuring relevant information is easily accessible for each roles.
- **Ensure system security, usability, and reliability** by applying appropriate authentication, authorization, and software engineering best practices

1.4. Methodology

1.4.1. Requirement gathering methods

Requirement gathering was carried out through direct engagement with incubation centers and a review of existing practices and documentation. The initial idea for IIMS emerged from our observation of incubation activities at BiTec (BiT Business Incubation & Techno-Entrepreneurship Center) in Bahir Dar Institute of Technology (BiT), where challenges related to application management, mentor coordination, and progress tracking were identified. To validate and generalize these observations, we also visited incubation and innovation centers in Addis Ababa, including Orbit Innovation Hub, InnobizK, Ministry of Labor and Skills (MoLS – Ethiopia), and DxValley Incubation Hub.

The following methods were used to collect system requirements:

- **Interviews:** Short and focused interviews were conducted with incubation center administrators, mentors, and startup founders to understand their needs and expectations.
- **Observation:** Existing workflows and daily operations were observed to identify manual processes and inefficiencies.
- **Document Analysis:** Application forms, evaluation sheets, and progress reports were reviewed to

understand data and reporting requirements.

- **Literature Review:** Relevant studies and similar systems were reviewed to identify common features and best practices.
- **Prototyping:** Simple interface mockups were used to clarify requirements and validate workflows

Requirement Modeling Approach

An **Object-Oriented Analysis and Design (OOAD)** approach was used to model the system requirements. This approach was selected because IIMS consists of multiple interacting entities such as users, startups, mentors, applications, and assessments. OOAD allows clear abstraction, modular design, and easier system maintenance and scalability.

Tools Used

- UML diagrams (Use Case, Class, Sequence, Activity)
- Draw.io / Lucidchart
- PostgreSQL (database design reference)
- Java and Spring Boot (backend reference)

1.4.2. Analysis and design Methodology

The system analysis and design phase followed the **Object-Oriented Analysis and Design (OOAD)** methodology.

Analysis and Design Method Used

- **Methodology:** Object-Oriented Analysis and Design (OOAD)
- **Justification:**
  OOAD enables the system to be modeled as interacting objects, which reflects real-world incubation processes and supports system extensibility and maintainability.

Tools Used for Analysis and Design

- UML modeling tools for:
  - Use Case Diagrams
  - Class Diagrams
  - Sequence Diagrams

    - Activity Diagrams
    - State Chart Diagrams
- ER diagrams for database modeling
- Draw.io / Lucidchart for diagram creation

1.4.3. Implementation Methodology

The implementation of IIMS follows a modular and layered development approach using modern web technologies. The system is developed as a web-based application to ensure accessibility and scalability.

Software Development Tools and Technologies

- **Programming Language:** Java
- **Backend Framework:** Spring Boot
- **Frontend Framework:** React
- **Database Management System (DBMS):** PostgreSQL
- **Application Server:** Embedded Apache Tomcat
- **Security:** JWT-based authentication and authorization
- **Version Control:** Git and GitHub
- **Development Tools:** IntelliJ IDEA, Visual Studio Code
- **API Testing and Documentation:** Postman, Swagger UI
- **Containerization:** Docker

This implementation methodology ensures that the system is reliable, maintainable, and aligned with the requirements identified during the requirement analysis phase.

1.5. Feasibility

1.5.1. Economic Feasibility

The development of IIMS is economically feasible because it relies primarily on open-source technologies and tools, which significantly reduce development and deployment costs. Technologies such as Java, Spring Boot, React, and PostgreSQL are freely available and widely supported.

By automating manual incubation processes, the system reduces administrative workload, minimizes errors,

and improves operational efficiency. This leads to cost savings for incubation centers by reducing paperwork, manual record-keeping, and repetitive tasks. In the long term, the system can also support sustainability through scalable deployment and potential subscription-based usage models for multiple incubation centers.

Overall, the benefits gained from improved efficiency, transparency, and scalability outweigh the relatively low development and operational costs, making the project economically viable.

1.5.2. Technical Feasibility

The project is technically feasible as the required technologies are mature, well-documented, and within the technical capability of the development team. The system is built using modern web technologies that support scalability, security, and maintainability.

The development team possesses foundational knowledge and practical experience in:

- Java and Spring Boot for backend development
- React for frontend user interface development
- PostgreSQL for relational database management
- RESTful API design and integration

Additionally, the system design follows Object-Oriented Analysis and Design (OOAD) principles, which simplifies complexity and supports future enhancements. Supporting tools such as Git, UML modeling tools, and API testing frameworks further enhance technical feasibility.

1.5.3. Time feasibility

The project is time-feasible within the academic schedule allocated for the industrial project. The system is developed in phases, starting with requirement analysis and design, followed by implementation, testing, and documentation.

The scope of IIMS has been carefully defined to focus on core incubation management features, allowing the team to prioritize essential functionalities within the available timeframe. Any advanced or non-critical features are identified for future enhancement, ensuring that the project can be completed successfully within the given time constraints.

1.6. Beneficiaries or significant of the project

The Incubation and Innovation Management System (IIMS) provides benefits to multiple stakeholders involved directly or indirectly in the incubation ecosystem. The system also contributes to the broader field of incubation management and digital transformation.

1.6.1 Direct Beneficiaries

1. **Incubation Center Administrators**
    - Simplified management of applications, startups, mentors, and programs
    - Reduced manual workload and improved operational efficiency
    - Access to real-time reports and performance data

2. **Startups**
    - Easy and transparent application submission process
    - Clear visibility into incubation progress and milestones
    - Improved access to mentors, resources, and feedback

3. **Mentors**
    - Structured mentor assignment and startup tracking
    - Clear overview of assigned startups and mentoring activities
    - Efficient communication with startups and administrators

4. **Investors and Partners**
    - Access to organized and up-to-date startup information
    - Better visibility into incubation outcomes and progress

1.6.2 Indirect Beneficiaries

- **Innovation and Startup Ecosystem:** Improved coordination and support for startups
- **Educational Institutions:** Better management of student-led innovation programs
- **Policy Makers and Funding Organizations:** Reliable data for evaluation and decision-making

1.6.3 Significance of the Project

The significance of IIMS lies in its contribution to the digital transformation of incubation and innovation centers. By replacing manual and fragmented processes with an integrated digital platform, the system enhances transparency, accountability, and efficiency.

The project also contributes academically by demonstrating the practical application of software engineering principles such as requirement analysis, object-oriented design, and system modeling. In a broader context, IIMS supports entrepreneurship development by enabling incubation centers to operate more effectively, thereby increasing the success rate of startups and strengthening the innovation ecosystem.

1.7. Limitations of the project

The IIMS is designed to automate and streamline the core processes of incubation and innovation management, including startup application handling, mentor assignment, progress tracking, and reporting. While the system significantly improves efficiency and transparency, it does not address every possible operational need of incubation centers due to scope and time limitations.

1.7.1 Project Limitations and Constraints

- **Time Constraint:** The project was developed within a limited academic timeframe. As a result, the focus was placed on implementing essential system functionalities, while some advanced features were postponed for future enhancement.
- **Resource Limitation:** The project was developed with limited financial and human resources. The system relies on open-source tools and a small development team, which limited the ability to implement highly complex or resource-intensive features.
- **Integration Limitations:** Integration with external systems such as government databases, payment gateways, and third-party investor platforms was not implemented due to access restrictions and time constraints.
- **Advanced Analytics and AI Features:** Features such as predictive analytics, automated startup scoring using artificial intelligence, and recommendation engines were not included in the current version of the system.
- **Mobile Application Support:** The system is implemented as a web-based application. Native mobile applications for Android and iOS are not developed in this phase.

1.7.2 Activities Left Undone

Due to the above constraints, the following activities were not completed in the current project phase:

- Full integration with external financial and governmental systems
- Advanced data analytics and intelligent reporting features
- Development of native mobile applications vc
- Automated investor–startup matchmaking mechanisms

These features are considered future enhancements and can be implemented in subsequent versions of the system.

1.8. Scope of the project

The **scope** of the Innovation and Incubation Management System (IIMS) defines the boundaries and services of the system. The system focuses on automating and integrating the core operations of incubation and innovation centers. The main **business processes and subsystems** to be implemented include:

1. **Startup Application Management**
    - Submission of applications through dynamic, customizable forms
    - Review, approval, or rejection of applications by administrators
    - Tracking application status and maintaining records
2. **User and Role Management**
    - Registration and management of multiple user types: administrators, startups, mentors, investors, alumni
    - Role-based access control to ensure security and appropriate permissions
3. **Mentorship Management**
    - Assign mentors to startups based on expertise, availability, and startup needs
    - Track mentorship sessions, feedback, and engagement
4. **Startup Progress Tracking**
    - Monitor startup milestones and task completion
    - Visual indicators like progress bars, badges, and completion charts
    - Notifications for upcoming deadlines or overdue tasks
5. **Communication and Notifications**
    - Messaging between startups, mentors, and administrators

- Real-time alerts for meetings, tasks, feedback, and announcements

6. **Reporting and Analytics**
    - Generate performance dashboards and reports for startups, mentors, and programs
    - Export reports in multiple formats (CSV, PDF, Excel, JSON)
7. **Program and Cohort Management**
    - Organize startups into cohorts and programs
    - Track progress toward graduation and issue certificates
8. **Investments and Funding Management**
    - Enable investor-startup connections and monitor engagement
    - Support basic analytics for funding activities

Excluded from this project scope (for future enhancements):

- Mobile application support for iOS and Android
- Advanced AI-based analytics and startup recommendation systems
- Full integration with external financial or government systems

Scope Summary:

The system is a web-based, multi-center, scalable platform that centralizes incubation processes, improves operational efficiency, enhances transparency, and facilitates data-driven decision-making.

1.9. Organization of the project

This project document is organized into two main chapters. **Chapter One** introduces the background of incubation centers, presents the problem statement, outlines the objectives, and explains the methodology, feasibility, beneficiaries, limitations, and scope of the project. **Chapter Two** focuses on the system features and requirement analysis, including a description of the existing and proposed system, functional and non-functional requirements, use case diagrams, business rules, system modeling with diagrams, user interface prototypes, and system requirements. The document concludes with references and appendices containing supporting materials used during the requirement analysis.

Chapter Two: System features

2.1. The Existing System

In most incubation and innovation centers, the current system for managing operations is largely **manual or semi-automated**, which creates inefficiencies and makes it difficult to handle growing workloads. The workflow typically begins with startups submitting their applications through **paper forms, email, or PDF documents**. Administrators manually collect these applications and store them in spreadsheets or physical files. There is no centralized system to track applications, making it time-consuming to locate information or compare applicants.

Once applications are received, **review and approval** are performed manually. Program officers or administrators evaluate each application individually, often using inconsistent criteria. Decisions are communicated to applicants through email or phone, which can result in delays and lack of transparency.

Mentor assignment is another challenge in the existing system. Administrators match mentors to startups manually, without considering mentor expertise, availability, or the specific needs of each startup. As a result, some startups may not receive appropriate guidance, and there is no structured record of mentorship activities.

Startup progress tracking is also handled informally. Startups report milestones via email or during in-person meetings, while administrators manually update spreadsheets to monitor progress. This approach makes it difficult to detect delays, assess startup performance accurately, or provide timely interventions.

Communication and reporting processes are fragmented. Important updates, reminders, and announcements are often missed because there is no centralized notification system. Reporting is labor-intensive, as administrators must gather data from multiple sources to evaluate program outcomes or mentor performance.

Overall drawbacks of the existing system include:

- Inefficient and time-consuming administrative processes
- Lack of standardized procedures for application review and mentor assignment
- Fragmented and inconsistent data storage
- Limited visibility into startup progress and program performance

- Poor scalability to handle multiple programs, startups, and mentors

2.2. Proposed System

The **Innovation and Incubation Management System (IIMS)** is designed to address the limitations of the existing system by providing a **centralized, web-based platform** that automates and integrates all core processes of incubation centers.

In the proposed system, startups submit their applications online using **dynamic, customizable forms**, which are automatically recorded in the system. Administrators can review applications efficiently, track their status (approved, rejected, or pending), and provide feedback directly through the platform. The system ensures **standardization and transparency** in the evaluation process.

Mentor assignment is structured and data-driven. Administrators can assign mentors based on their expertise, availability, and the specific requirements of each startup. Mentors can track their assigned startups, schedule sessions, provide feedback, and update engagement records, all through the system.

Startup progress tracking is automated and visual. Milestones and tasks are organized into phases, and progress is displayed using charts, badges, and dashboards. Administrators can monitor startup performance in real time and identify any issues early. Notifications and alerts ensure that both startups and mentors are reminded of deadlines and upcoming tasks.

The system also includes **reporting and analytics**, allowing administrators to generate performance dashboards, track mentor engagement, and evaluate program outcomes efficiently. Communication is centralized, reducing delays and improving coordination among startups, mentors, and administrators.

By integrating all core operations, IIMS enhances **efficiency, accuracy, transparency, and scalability**, allowing incubation centers to manage multiple programs, startups, and mentors effectively while reducing manual workload. It replaces fragmented, error-prone processes with a structured and organized digital solution.

2.3. Requirement Analysis

2.3.1. Incubation and Innovation Management System (IIMS) Functional requirement

1. Core System (Implemented Features)

Tenant & Organization Management

FREQ1: The system shall allow organizations to register as tenants and apply for incubation center services.

FREQ2: The system shall allow super admins to review and approve tenant applications for incubation center access.

FREQ3: The system shall manage tenant subscriptions with different plan tiers and feature access levels.

FREQ4: The system shall allow tenants to subscribe to specific features and services based on their needs.

User Registration & Authentication

FREQ5: The system shall allow users to register and authenticate with email and password credentials.

FREQ6: The system shall support multiple user roles including Super Admin, Tenant Admin, Startup, Mentor, Investor, and Alumni.

FREQ7: The system shall allow users to manage their profiles and account settings.

Startup Application & Onboarding

FREQ8: The system shall allow startups to submit applications through customizable application forms.

FREQ9: The system shall allow tenant admins to review, approve, or reject startup applications.

FREQ10: The system shall automatically create startup profiles upon application approval.

FREQ11: The system shall allow startups to complete their profiles with business information, team details, and documents.

Mentor Assignment & Management

FREQ12: The system shall allow tenant admins to assign mentors to startups individually or in bulk.

FREQ13: The system shall support mentor-startup matching based on expertise areas and startup needs.

FREQ14: The system shall allow startups to view their assigned mentors and schedule sessions.

FREQ15: The system shall allow mentors to view their assigned startups and track their progress.

FREQ16: The system shall allow startups to rate and provide feedback on mentor performance.

Progress Tracking & Milestones

FREQ17: The system shall allow tenant admins to create customizable progress tracking templates with phases and tasks.

FREQ18: The system shall allow tenant admins to assign progress templates to startups for milestone tracking.

FREQ19: The system shall allow startups to submit task completions and required documents.

FREQ20: The system shall allow mentors to review startup submissions, provide feedback, and approve or request revisions.

FREQ21: The system shall track and display startup progress across all assigned templates and phases.

Mentorship Programs

FREQ22: The system shall allow alumni to register as mentors and provide mentorship to current startups.

FREQ23: The system shall allow startups to seek mentorship from alumni members.

FREQ24: The system shall facilitate mentorship connections and relationship management between alumni and startups.

Assessments & Evaluations

FREQ25: The system shall allow tenant admins to create assessments with multiple-choice questions for startup evaluation.

FREQ26: The system shall allow startups and applicants to complete assessments and receive scores.

FREQ27: The system shall provide assessment analytics and response tracking for tenant admins.

Program Organization

FREQ48: The system shall allow tenant admins to organize startups into cohorts and industry categories.

FREQ49: The system shall support filtering and management of startups by cohort and industry.

Help & Support

FREQ50: The system shall provide a help center for tenants to request features and communicate with support.

Communication & Collaboration

FREQ36: The system shall provide real-time chat messaging between users (one-on-one and group chats).

FREQ37: The system shall allow users to schedule meetings with calendar integration.

FREQ38: The system shall integrate with Google Calendar for meeting scheduling and video conferencing links.

FREQ39: The system shall send automated notifications for important events and updates.

Content Management

FREQ40: The system shall allow tenant admins to create and publish news posts, announcements, and success stories.

FREQ41: The system shall allow tenant admins to upload and manage resource documents (templates, guides, legal documents).

FREQ42: The system shall allow startups to access relevant resources and recommendations based on their needs.

2. Extended / Future System (Planned Features)

Graduation & Alumni Management

FREQ28: The system shall allow tenant admins to mark startups as graduated from the incubation program.

FREQ29: The system shall allow tenant admins to convert graduated startups to alumni

members.

FREQ30: The system shall allow alumni to maintain their profiles and view their milestone history.

FREQ31: The system shall provide alumni networking and connection features.

Investor Network

FREQ32: The system shall allow investors to create profiles with investment focus, criteria, and portfolio information.

FREQ33: The system shall allow investors to connect with startups and alumni for investment opportunities.

FREQ34: The system shall allow startups to view investor profiles and send connection requests.

FREQ35: The system shall facilitate investor-startup connections and relationship management.

Public Landing Pages

FREQ43: The system shall allow tenants to create and customize public-facing landing pages.

FREQ44: The system shall allow tenants to showcase their services, news, and startup highlights on landing pages.

Analytics & Reporting

FREQ45: The system shall provide comprehensive analytics on startups, mentors, applications, and program progress.

FREQ46: The system shall generate reports with filtering by cohort, industry, date range, and other criteria.

FREQ47: The system shall export analytics data and reports to CSV format.

2.3.2. System Use case

2.3.2.1. Use case Diagram

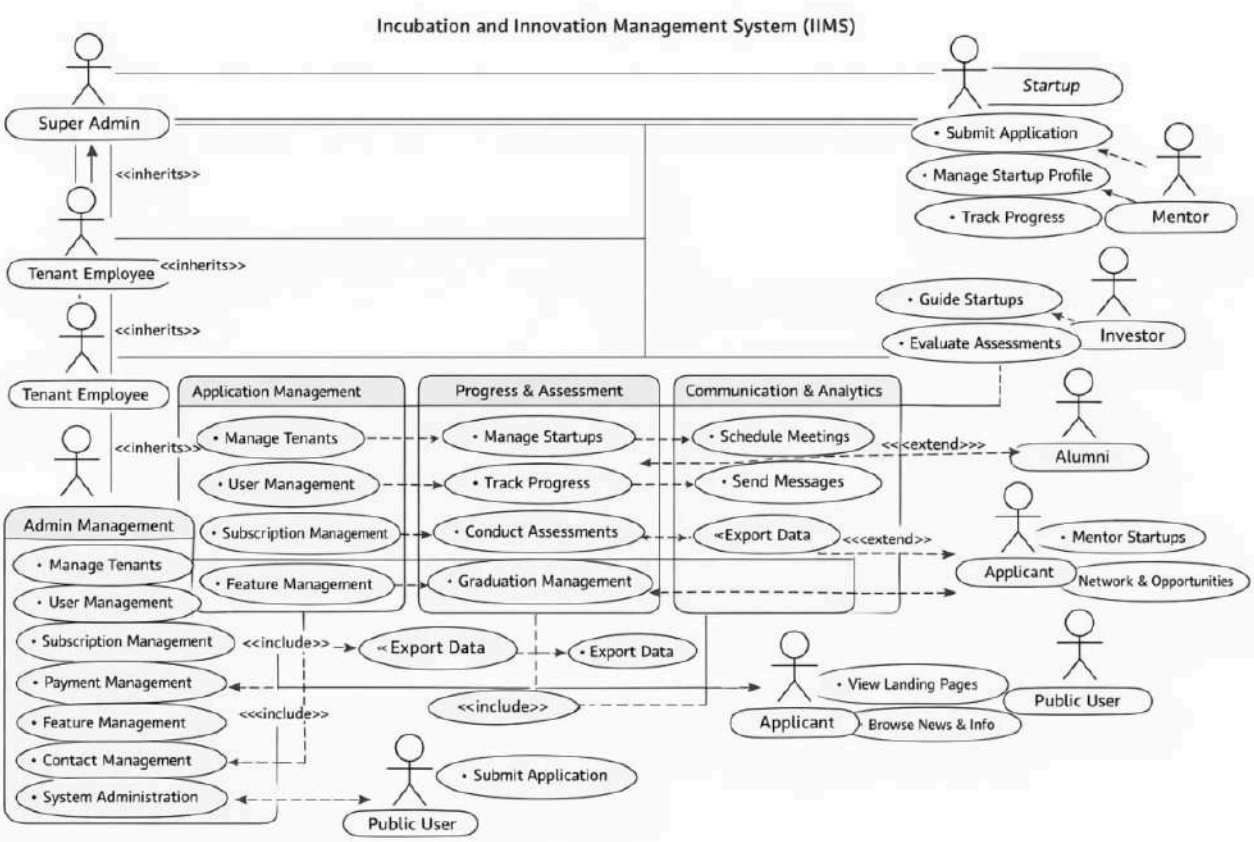


Figure 2.1 Use Case Diagram

2.3.2.2. Use Case documentation

Table 2.3.2.2-1: Actors Description

Super Admin	Manages system-wide operations including tenants, subscriptions, payments, features, and system administration.
Tenant Admin	Manages tenant-level users, applications, startups, mentors, assessments, and graduation.
Tenant Employee	Performs limited administrative tasks based on assigned permissions.
Startup	Applies for incubation, manages profile, tracks progress, and interacts with mentors.
Mentor	Guides startups, reviews progress, and evaluates assessments.
Investor	Explores startups and connects with startups and alumni.
Alumni	Graduated startup members who mentor startups and participate in networking.
Applicant	Submits applications to incubation programs.
Public User	Views public information and submits applications.

Table 2.3.2.2-2: Description of Login to System Use Case

Use Case ID	UC-01
UseCase Name	Login to System
Participating Actor	All Authenticated Actors (Super Admin, Tenant Admin, Startup, etc.)
Description	Allows users to securely log into the system to access their specific dashboard and tools.
Pre-Condition	A valid user account must exist in the system database.
Flow of Events	1. User enters username and password. 2. System validates credentials against the database. 3. System grants access and redirects the user to their specific dashboard.
Alternative Flow	Invalid Credentials: If the username or password does not match, the system displays an "Invalid login" error message and allows the user to try again.
Post-Condition	The user is authenticated and an active session is established.

Table 2.3.2.2-3: Description of Manage Tenants Use Case

Use Case ID	UC-02

Use Case Name	Manage Tenants
Participating Actor	Super Admin
Description	Allows the Super Admin to oversee the lifecycle of tenants by approving new applications, activating accounts, or suspending access.
Pre-Condition	Super Admin must be successfully logged into the system (UC-01).
Flow of Events	1. Admin navigates to the 'Tenant Management' section and views the list of all tenants. 2. Admin selects a specific tenant from the list. 3. Admin chooses an action: **Approve** (for new applicants), **Activate** (for inactive/new), or **Suspend** (to revoke access). 4. System prompts for confirmation. 5. System updates the tenant record.
Alternative Flow	**Tenant Application Rejected:** If the Admin finds the tenant details insufficient or invalid, they select 'Reject'. The system prompts for a reason and notifies the applicant.
Post-Condition	The tenant's status is updated in the database, and their access permissions are adjusted immediately.

Table 2.3.2.2-4: Description of User Management Use Case

Use Case ID	UC-03
Use Case Name	User Management
Participating Actor	Super Admin, Tenant Admin
Description	Allows administrators to create, update, and manage user accounts, including the assignment of roles and account deactivation.
Pre-Condition	Administrators must be logged into the system with appropriate privileges.
Flow of Events	1. Admin navigates to the 'User Management' dashboard to view the list of existing users. 2. Admin chooses to **Create a New User** or selects an existing user to **Edit**. 3. Admin enters/modifies details (Name, Email, etc.) and assigns a specific **Role** (e.g., Mentor, Startup). 4. Admin saves the changes or toggles the **Deactivate** switch to revoke access. 5. System validates the data and updates the user directory.
Alternative Flow	**Invalid Information:** If required fields are missing or the email format is incorrect, the system highlights the errors and prevents the save until corrected.

Post-Condition	User details and roles are updated in the system, and an email notification is optionally sent to the user.

Table 2.3.2.2-5: Description of Subscription Management Use Case

Use Case ID	UC-04
Use Case Name	Subscription Selection & Automated Management
Participating Actor	Public User (Applicant), Super Admin
Description	Applicants select their plan and features; the system automatically calculates the price and discounts, and the Super Admin provides the final activation.
Pre-Condition	Plans, Features, and Discount Rules (e.g., 20% off for Annual) are defined in the system.
Flow of Events	1. **User Selection:** During application, the User selects a **Base Plan** and specific **Features**. 2. **Automated Calculation:** The system instantly calculates the price 3. **Submission:** User submits the application with the calculated quote; status is PENDING_CONFIRMATION. 4. **Admin Verification:** Super Admin reviews the application and the system-generated price. 5. **Approval:** Super Admin clicks **Activate Subscription** (with the option to grant the

	first cycle as "Free").
Alternative Flow	**Discount Update:** If the user changes their billing cycle (e.g., from Monthly to Annual), the system automatically updates the discount percentage without Admin intervention.
Post-Condition	Subscription is **ACTIVE**; the system logs the final price and grants feature access.

Table 2.3.2.2-6: Description of Submit Application Use Case

Use Case ID	UC-05
Use Case Name	Submit Application
Participating Actor	Startup, Applicant
Description	Allows users to provide business details and necessary documentation to apply for an incubation program.
Pre-Condition	An active incubation program must be open, and the application form must be accessible to the user.

Flow of Events	1. The user navigates to the 'Apply' section and fills out the digital application form. 2. User uploads required supporting documents (e.g., Pitch Deck, Business Plan, ID). 3. The user reviews the summary and clicks **Submit Application**. 4. The system validates the input and issues a submission confirmation ID.
Alternative Flow	**Missing Required Fields:** If the user attempts to submit with empty mandatory fields or invalid file formats, the system flags the specific errors and prevents submission.
Post-Condition	The application is logged in the system with a "Pending Review" status, and the Tenant Admin is notified.

Table 2.3.2.2-7: Description of Application Management Use Case

Use Case ID	UC-06
Use Case Name	Application & Onboarding Management
Participating Actor	Tenant Admin, Tenant Employee
Description	Allows admins to review applications from both prospective Startups and Mentors. Approval grants system access.
Pre-Condition	A pending application (Startup or Mentor) exists in the system.

Flow of Events	1. Admin opens the pending application (Startup or Mentor). 2. Admin evaluates the profile and documentation. 3. Admin selects **Approve**. 4. The system creates the user account and assigns the relevant role. 5. The system sends a **Welcome Email** with **Login Credentials**.
Alternative Flow	**Rejection:** Admin selects **Reject** and provides a reason. System sends a **Rejection Email** explaining why they were not accepted.
Post-Condition	The applicant becomes an active user (Startup/Mentor) or is notified of rejection.

Table 2.3.2.2-8: Description of Manage Startups Use Case

Use Case ID	UC-07
Use Case Name	Manage Startups
Participating Actor	Tenant Admin
Description	Allows the Admin to manage the startup's progress by assigning a Mentor and updating their status to "Graduated" when the program is complete.

Pre-Condition	The startup must be approved (UC-06) and a Mentor must be available in the system.
Flow of Events	1. Admin views the list of approved startups and selects a **Startup Profile**. 2. Admin selects a **Mentor** from the available list and assigns them to the Startup. 3. Admin monitors progress and eventually updates the status to **Graduated**. 4. The system detects the "Graduated" status and automatically converts the user role to **Alumni**.
Alternative Flow	**Removal from Mentor:** Admin can unassign or change a Mentor if the pairing is no longer effective.
Post-Condition	Startup record is updated; if graduated, the user gains Alumni permissions and networking access.

Table 2.3.2.2-9: Description of Track Progress Use Case

Use Case ID	UC-08
Use Case Name	Progress Tracking & Mentorship
Participating Actor	Startup, Mentor
Description	Startups execute tasks from an assigned template, while Mentors provide feedback, approvals, and learning resources.

Pre-Condition	A Progress Template has been assigned to the Startup by the Admin.
Flow of Events	1. **Startup** selects their **Assigned Template** and views the active **Phase** and **Tasks**. 2. **Mentor** reviews the current tasks and optionally **Recommends Resources** (links/files) to help the Startup. 3. **The startup** completes the work and **submits** it for review. 4. **Mentor** reviews the submission and provides **Feedback**. 5. **Mentor** selects **Approve** to move the Startup to the next task/phase.
Alternative Flow	**Request Revision:** If the work is insufficient, the Mentor requests a revision. The Startup is notified to review the feedback and resources before resubmitting.
Post-Condition	The task is completed, and progress is updated toward the Graduation milestone.

Table 2.3.2.2-10: Description of Conduct Assessments Use Case

Use Case ID	UC-09
Use Case Name	Phase-Based Assessments & Analytics

Participating Actor	Startup, Tenant Admin, Mentor
Description	Admins create assessments linked to phases. Startups complete them before and after each phase to measure progress and generate analytical reports.
Pre-Condition	Tenant Admin has created the assessment questions and linked them to the Progress Template phases.
Flow of Events	1. **The startup** attempts the **Pre-Assessment** before starting a Phase. 2. The system automatically scores the assessment and unlocks the Phase tasks. 3. **Startup** completes all tasks in the Phase (UC-08). 4. **Startup** attempts the **Post-Assessment** upon Phase completion. 5. The system calculates the score and compares it with the Pre-Assessment. 6. **Admin/Mentor** views the **Analysis Report** (Growth/Learning Gain).
Alternative Flow	**Incomplete Assessment:** If a startup exits before submitting, the system saves the progress but does not unlock the next phase or generate an analysis.
Post-Condition	Assessment scores are stored, and the comparative analysis is added to the Startup's performance profile.

Table 2.3.2.2-11: Description of Graduation Management Use Case

Use Case ID	UC-10
Use Case Name	Graduation Management
Participating Actor	Tenant Admin
Description	Allows the Admin to review a startup's completion of the program and manually permit their conversion to the Alumni role.
Pre-Condition	The startup is marked as "Program Completed" based on tasks and assessments.
Flow of Events	1. Admin accesses the **Graduation Review** list. 2. Admin reviews the Startup's final progress, mentor feedback, and assessment analytics. 3. Admin selects the option to **Approve Alumni Conversion**. 4. The system prompts the Admin to confirm the role change. 5. The system updates the user's role and notifies them of their new status.
Alternative Flow	**Graduation Postponed:** If the Admin decides the startup isn't ready for alumni status, they postpone the conversion. The startup remains in the system as "Completed" but does not receive Alumni privileges.
Post-Condition	The Startup is officially converted to an **Alumni** user only after the Admin's manual permission is granted.

Table 2.3.2.2-12: Description of Communication Management Use Case

Use Case ID	UC-11
Use Case Name	Communication & Calendar Orchestration
Participating Actor	All Authenticated Actors
Description	Manages real-time communication and scheduling, supporting internal users and external email-based guests.
Pre-Condition	The user is logged in. Google Calendar API is integrated for meeting synchronization.
Flow of Events	**1. Chat Initialization:** User creates a chat by selecting an existing system user to start a private or group conversation. **2. Meeting Creation:** User opens the calendar and creates a new meeting event. **3. Attendee Management:** User adds internal participants (selected from system list) or types in external guest emails. **4. Sync & Dispatch:** System creates the event in the in-app calendar and pushes it to all attendees' Google Calendars via the integration.
Alternative Flow	**Invalid Email:** If an external guest email is formatted incorrectly, the system flags the error and prevents the invite from being sent until corrected.
Post-Condition	Chat channels are established; Meeting invites and Google Calendar events are distributed to all attendees.

Table 2.3.2.2-13: Description of View Landing Pages Use Case

Use Case ID	UC-12
Use Case Name	View Landing Pages
Participating Actor	Public User
Description	Allows any user to view an incubator's public information and available programs, regardless of whether applications are open.
Pre-Condition	The Tenant Admin has published the landing page.
Flow of Events	1. **Public User** navigates to the Tenant's public URL. 2. User views program descriptions, mentor profiles, and success stories. 3. System checks for **Active Application Calls**. 4. If a call is active, the "Apply Now" button redirects to the application form; if not, "Application is not available" message appears. 5. The user browses content to learn about the incubator.
Alternative Flow	**Application Call Entry:** If the user clicks an active call, they are redirected to the registration/login page to begin **UC-05**.

Post-Condition	Information is displayed to the user.

Table 2.3.2.2-14: Description of Export Data Use Case

Use Case ID	UC-13
Use Case Name	Global Data Export & Reporting
Participating Actor	Super Admin, Tenant Admin
Description	Allows admins to generate comprehensive reports across all system modules (Users, Startups, Mentors, Applications, Investers) using structural filters.
Pre-Condition	Admin is logged in; the system contains active records.
Flow of Events	1. Admin selects the **Data Category** (e.g., Startups, Mentors, Application Logs, or Assessments). 2. Admin applies **Global Filters**: Date Range (Time) and Tenant. 3. Admin applies **Structural Filters**: **Industry** and **Cohort**. 4. Admin selects the export format (**CSV, XLSX, or PDF**). 5. The system aggregates the data and triggers the download.

Alternative Flow	**No Data Found:** If the combination of Industry and Cohort filters returns no results, the system notifies the user and prevents the empty export.
Post-Condition	A structured file is generated and downloaded to the Admin's device.

2.3.3. Business Rule Documentation

BR1: Valid Login Credentials

- **Identifier**: BR1
- **Name**: Valid Login Credentials
- **Description**: Users must provide valid email and password credentials to access the system. Failed login attempts may be tracked for security purposes. The system authenticates users and issues JWT tokens upon successful login.
- **Reference**: Authentication & Authorization Module, AuthController, SecurityConfig
- **Revision History**: Version 1.0 - Initial implementation

BR2: Supported File Types

- **Identifier**: BR2
- **Name**:Supported File Types
- **Description**: The system shall accept file uploads in standard formats including PDF, DOC, DOCX, PPT, PPTX, JPG, JPEG, PNG, and other common document and image formats for applications, resources, and profile documents. File size limits may apply as configured by the system administrator.
- **Reference**:FileController, ApplicationDocumentController, File Upload Services
- **Revision History**:Version 1.0 - Initial implementation

BR3: Role-Based Access Control

- **Identifier**: BR3
- **Name**:Role-Based Access Control

- **Description**: Users can only access features and data appropriate to their assigned role (SUPER_ADMIN, TENANT_ADMIN, TENANT_EMPLOYEE, STARTUP, MENTOR, INVESTOR, ALUMNI). Access to tenant data is restricted to users belonging to that tenant. Users cannot access data or features outside their role permissions.
- **Reference**: SecurityConfig, User Entity, Role Enum, FeatureAccessInterceptor
- **Revision History**:Version 1.0 - Initial implementation

BR4: Tenant Subscription Status

- **Identifier**: BR4
- **Name**: Tenant Subscription Status
- **Description**: Tenants with SUSPENDED subscription status are blocked from accessing premium features, but retain access to core functionality and help center services. Tenants with ACTIVE status have full access to their subscribed features. PENDING status indicates the subscription is awaiting approval.
- **Reference**: SubscriptionService, FeatureAccessInterceptor, TenantSubscription Entity
- **Revision History**:Version 1.0 - Initial implementation

BR5: Feature Access Based on Subscription

- **Identifier**:BR5
- **Name**: Feature Access Based on Subscription
- **Description**:Tenants can only use features included in their active subscription plan. Access to premium features requires appropriate subscription tier or feature purchase. The system enforces feature-based access control on API endpoints and prevents unauthorized feature usage.
- **Reference**:FeatureService,SubscriptionService,FeatureAccessInterceptor,FeatureMappingService
- **Revision History**:Version 1.0 - Initial implementation

BR6: Application Approval Workflow

- **Identifier**:BR6
- **Name**:Application Approval Workflow

- **Description**: Startup applications must be reviewed and approved by Tenant Admin before startup accounts are created and profiles are activated. Applications remain in PENDING status until admin action. Only approved applications result in user account creation. Rejected applications are marked as REJECTED with optional feedback.
- **Reference**:ApplicationService, ApplicationController, ApplicationStatus Enum
- **Revision History**:Version 1.0 - Initial implementation

BR7: Graduation Prerequisites

- **Identifier**:BR7
- **Name**:Graduation Prerequisites
- **Description**: Startups can only be marked as GRADUATED by Tenant Admins or Super Admins. Startups must have GRADUATED status before they can be converted to ALUMNI members. The system tracks graduation timestamps and allows graduated startups to maintain their profile data.
- **Reference**: GraduationController, StartupProfile Entity, Graduation Service
- **Revision History**:Version 1.0 - Initial implementation

BR8: Mentor Assignment Rules

- **Identifier**: BR8
- **Name**: Mentor Assignment Rules
- **Description**:Mentors can only be assigned to startups within the same tenant. Tenant Admins have authority to assign or remove mentor-startup relationships. Mentors can view their assigned startups and track progress. Startups can view their assigned mentors and schedule sessions.
- **Reference**:StartupMentorAssignmentController,MentorAssignmentService, MentorAssignment Entity
- **Revision History**:Version 1.0 - Initial implementation

BR9: Progress Template Assignment

- **Identifier**: BR9
- **Name**:Progress Template Assignment
- **Description**: Progress tracking templates can only be assigned to users (startups) within the

same tenant. Templates must be created by Tenant Admins before assignment. Templates consist of phases and tasks that startups must complete. Templates can be assigned individually or in bulk to multiple startups.

- **Reference**:ProgressTemplateAssignmentController,ProgressTemplateService, ProgressTemplate Entity
- **Revision History**:Version 1.0 - Initial implementation

BR10: Task Submission Review

- **Identifier**:BR10
- **Name**: Task Submission Review
- **Description**: Startup task submissions require mentor or admin review before being marked as APPROVED. Submissions can be approved or returned for revision with feedback. The system tracks submission status (PENDING, APPROVED, NEEDS_REVISION) and notifies relevant parties of status changes.
- **Reference**: ProgressSubmissionController, ProgressSubmissionService, Task Submission Workflow
- **Revision History**:Version 1.0 - Initial implementation

BR11: Cohort and Industry Assignment

- **Identifier**: BR11
- **Name**:Cohort and Industry Assignment
- **Description**: Startups can be assigned to only one cohort and one industry category. Cohorts and industries are tenant-specific and created by Tenant Admins. These assignments enable filtering, grouping, and targeted management of startups. Assignments can be used for application forms and progress templates.
- **Reference**: CohortController, IndustryController, StartupProfile Entity, Cohort and Industry Services
- **Revision History**:Version 1.0 - Initial implementation

BR12: Meeting Scheduling Rules

- **Identifier**: BR12

- **Name**: Meeting Scheduling Rules
- **Description**: Users can schedule meetings only with other users within the same tenant. Meeting attendees must be selected from available users in the tenant. Meetings require specified date, time, duration, and attendee list. Google Calendar integration allows automatic event creation and video conferencing links.
- **Reference**:MeetingController, GoogleOAuthController, Meeting Entity, Calendar Integration Services
- **Revision History**: Version 1.0 - Initial implementation

BR13: Chat Room Creation

- **Identifier**: BR13
- **Name**: Chat Room Creation
- **Description**:Users can create chat rooms only with other users within the same tenant. Group chats require multiple participants from the same tenant. Individual (one-on-one) chats can be initiated between any two users in the same tenant. Support chat rooms are available for tenant help requests.
- **Reference**: ChatRoomController, ChatService, ChatRoom Entity, WebSocket Configuration
- **Revision History**:Version 1.0 - Initial implementation

BR14: Admin Request Approval

- **Identifier**: BR14
- **Name**: Admin Request Approval
- **Description**: Users requesting admin roles must be approved by Super Admin before gaining admin privileges. Requests remain in PENDING status until Super Admin action. Only Super Admins can approve or reject admin role requests. Approved requests result in role change and permission updates.
- **Reference**:UserController, AdminRequestService, AdminRequest Entity, User Role Management
- **Revision History**:Version 1.0 - Initial implementation

BR15: Tenant Registration Approval

- **Identifier**: BR15

- **Name**:Tenant Registration Approval
- **Description**: Organizations applying for tenant registration must be reviewed and approved by Super Admin before gaining access to the system. Tenants remain in PENDING status until approval. Super Admins can approve or reject tenant applications. Approved tenants receive subscription setup and access credentials.
- **Reference**: TenantController, TenantService, Tenant Entity, Tenant Application Workflow
- **Revision History**:Version 1.0 - Initial implementation

2.3.4. User Interface prototype

User interface requirements should be gathered with a prototype approach here and aligned with system use case documentation that shows the user interaction with the system. It should be labeled and referenced in the use case documentation.

2.3.5. State chart diagram

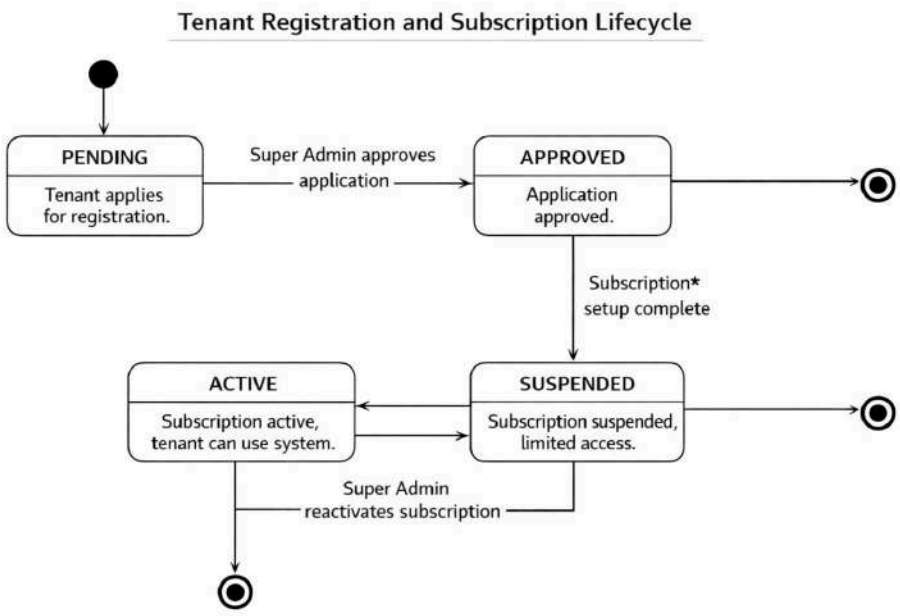


Figure 2.2 Tenant Registration and Subscription Lifecycle State Diagram

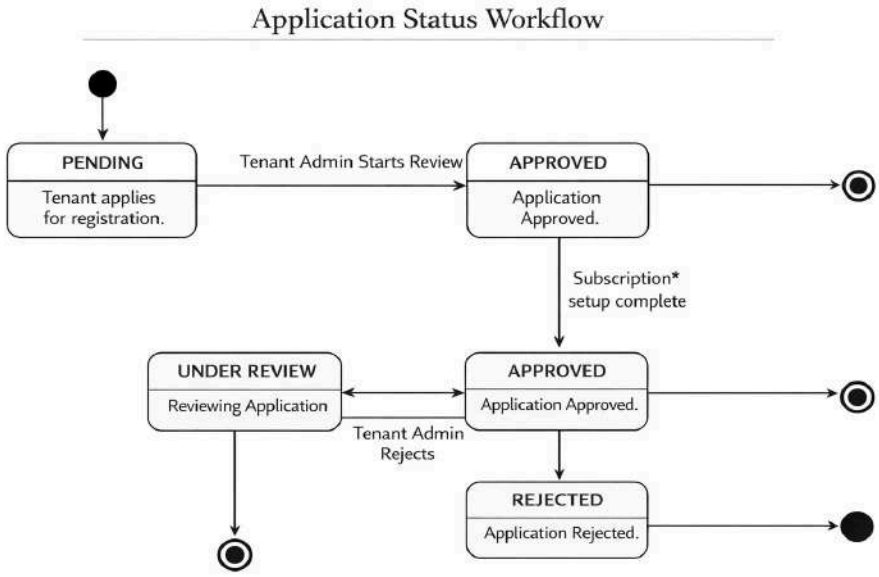


Figure 2.3 Application Status Workflow State Diagram

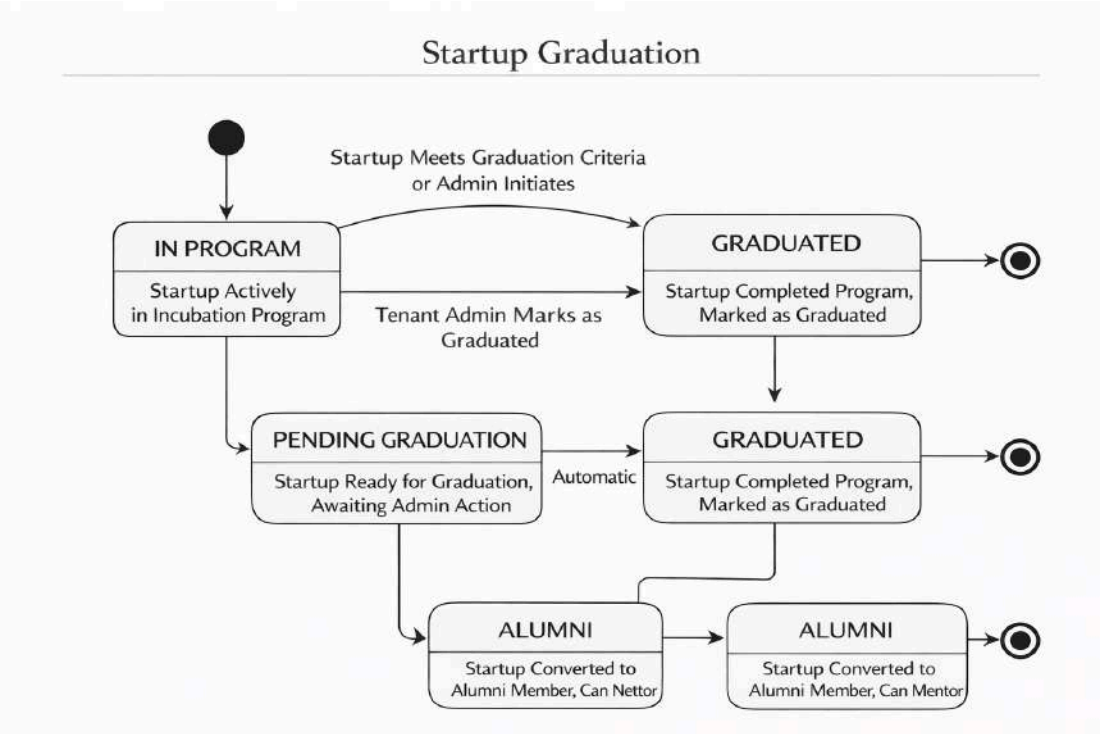


Figure 2.4 Startup Graduation State Diagram

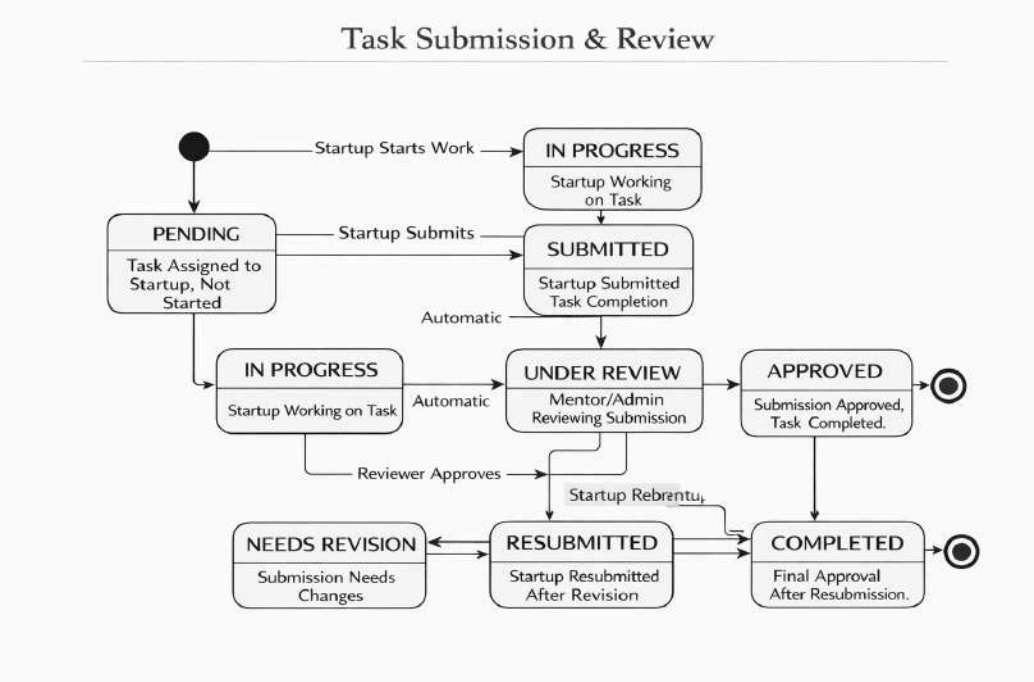


Figure 2.5 Task Submission State Diagram

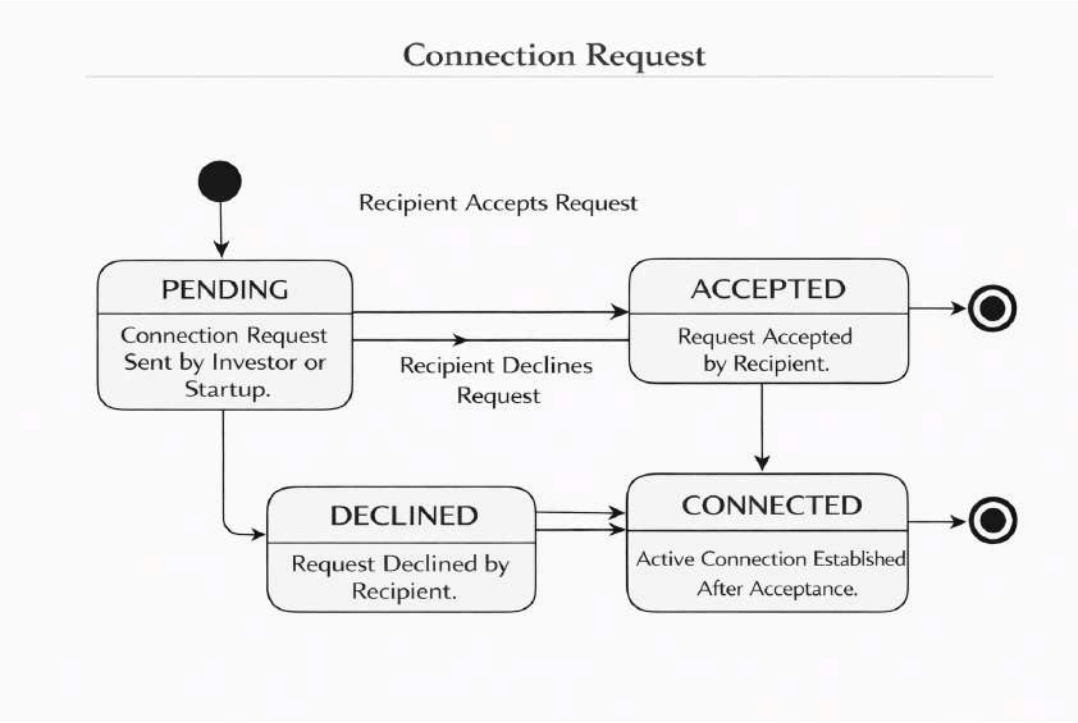


Figure 2.6 Connection Request State Diagram

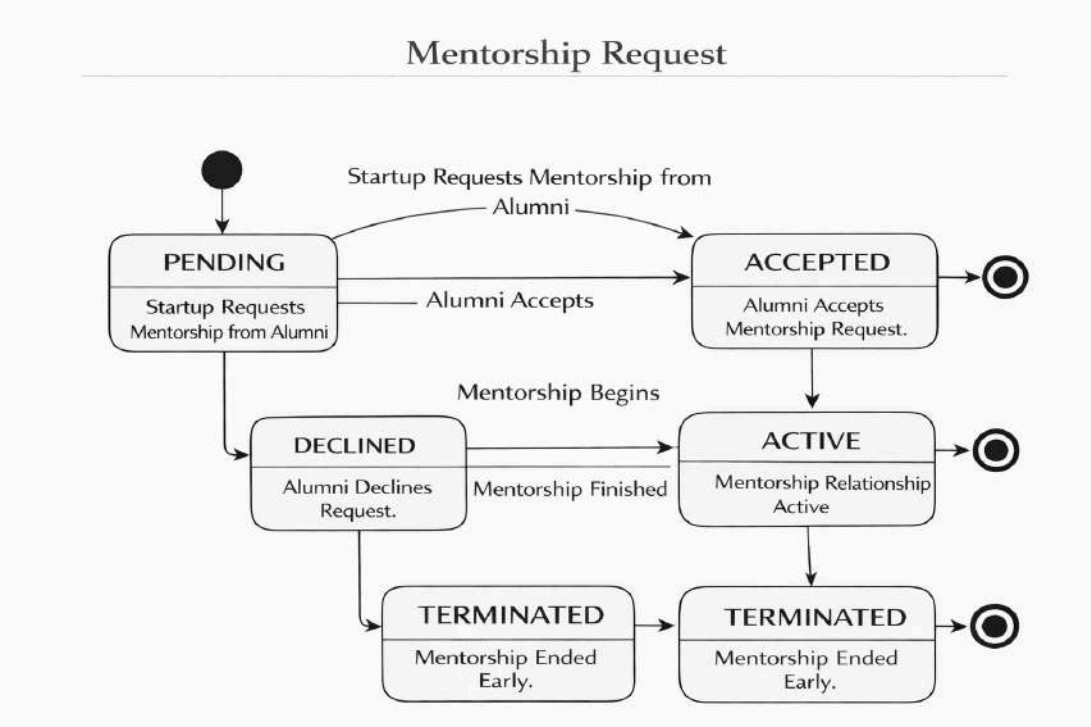


Figure 2.7 Mentorship Requests State Diagram

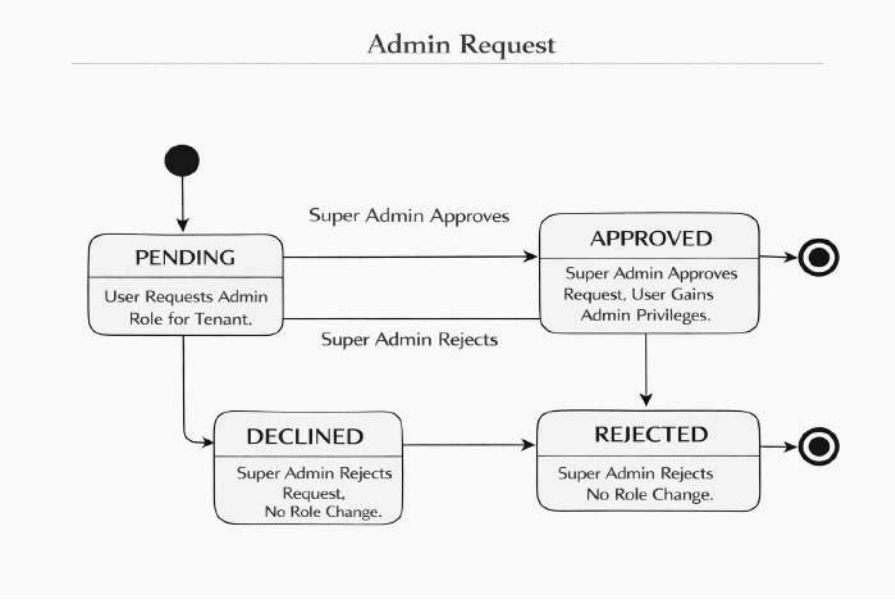


Figure 2.8 Admin Request State Diagram

2.3.6. Activity Diagram

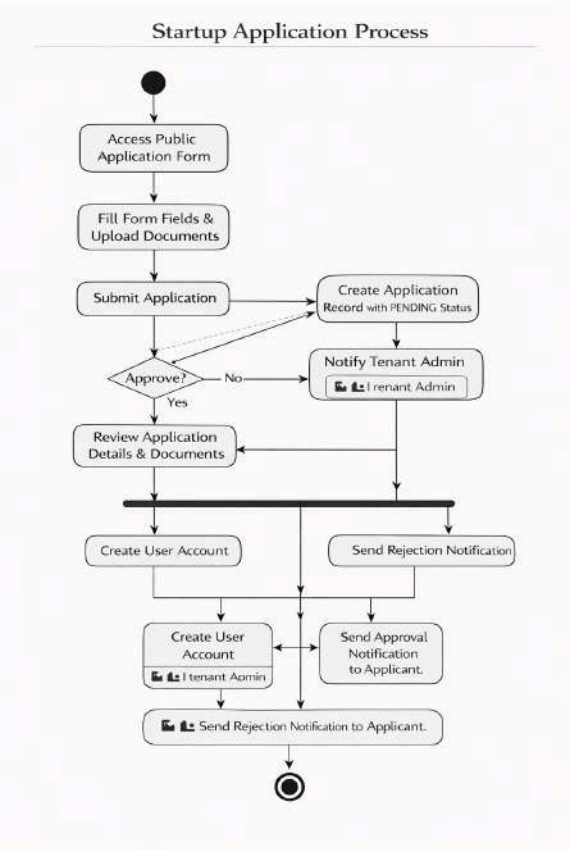


Figure 2.9 Startup Application Process Activity Diagram

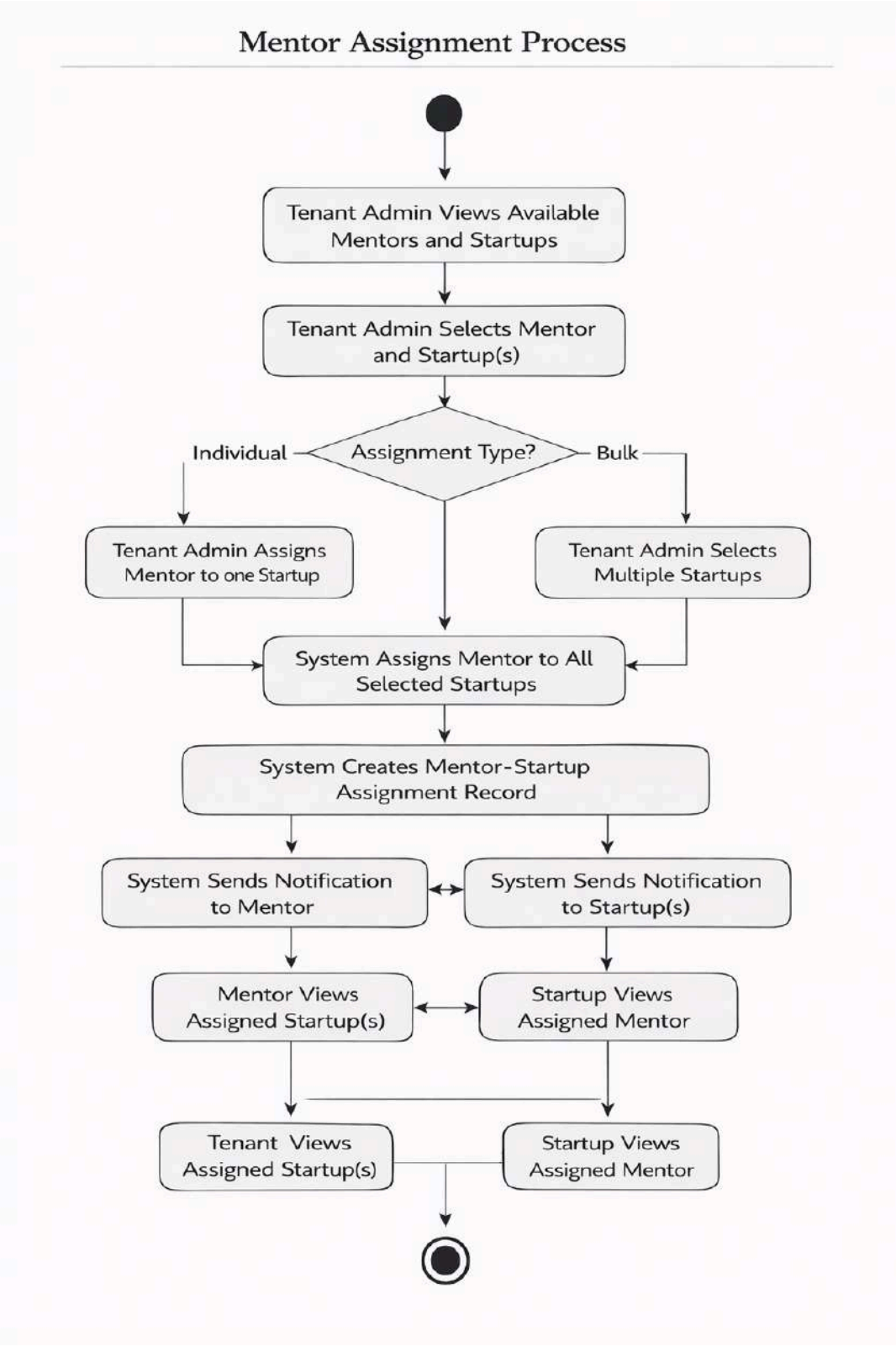


Figure 2.10 Mentor Assignment Activity Diagram

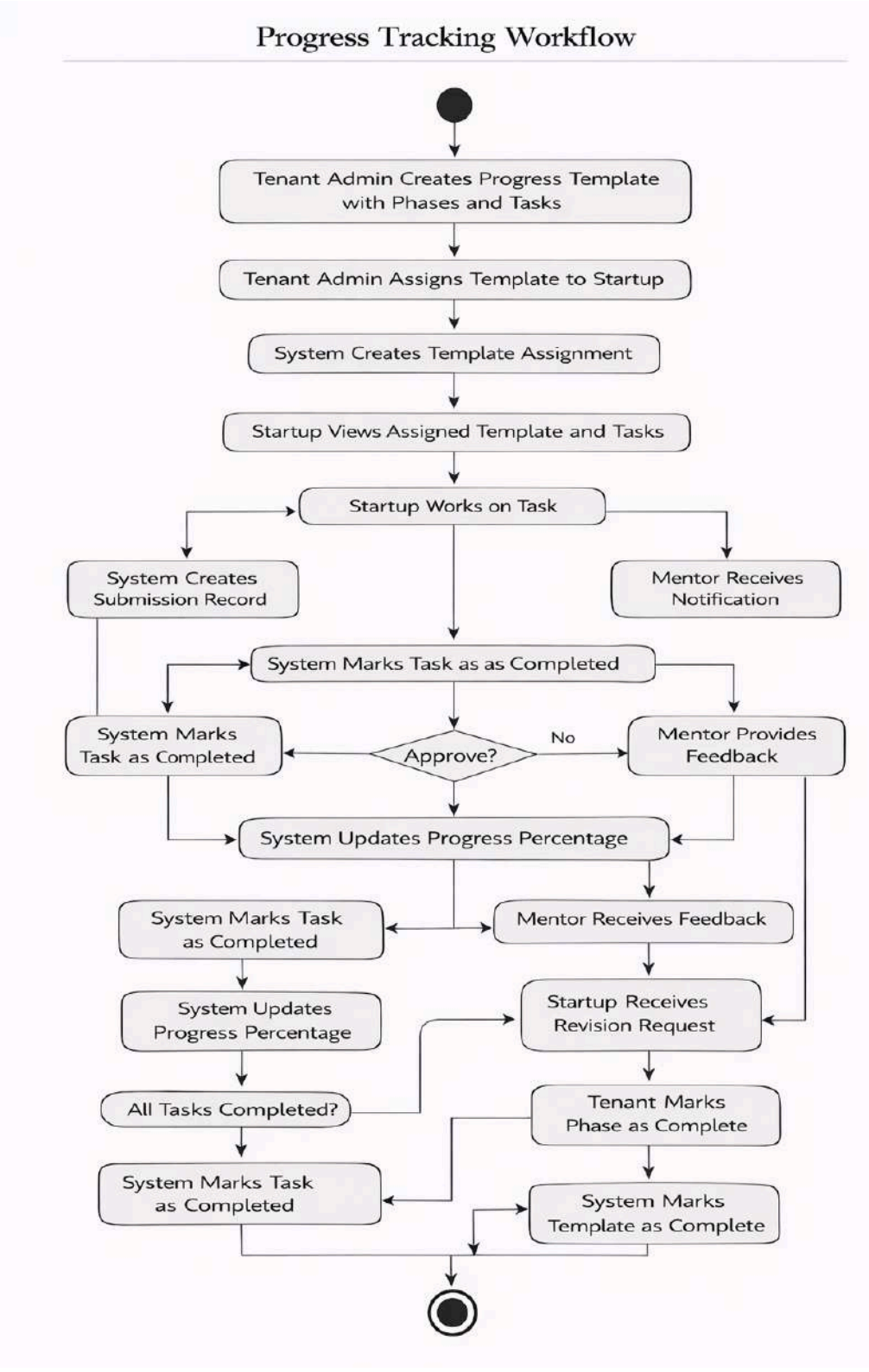


Figure 2.11 Progress tracking Activity Diagram

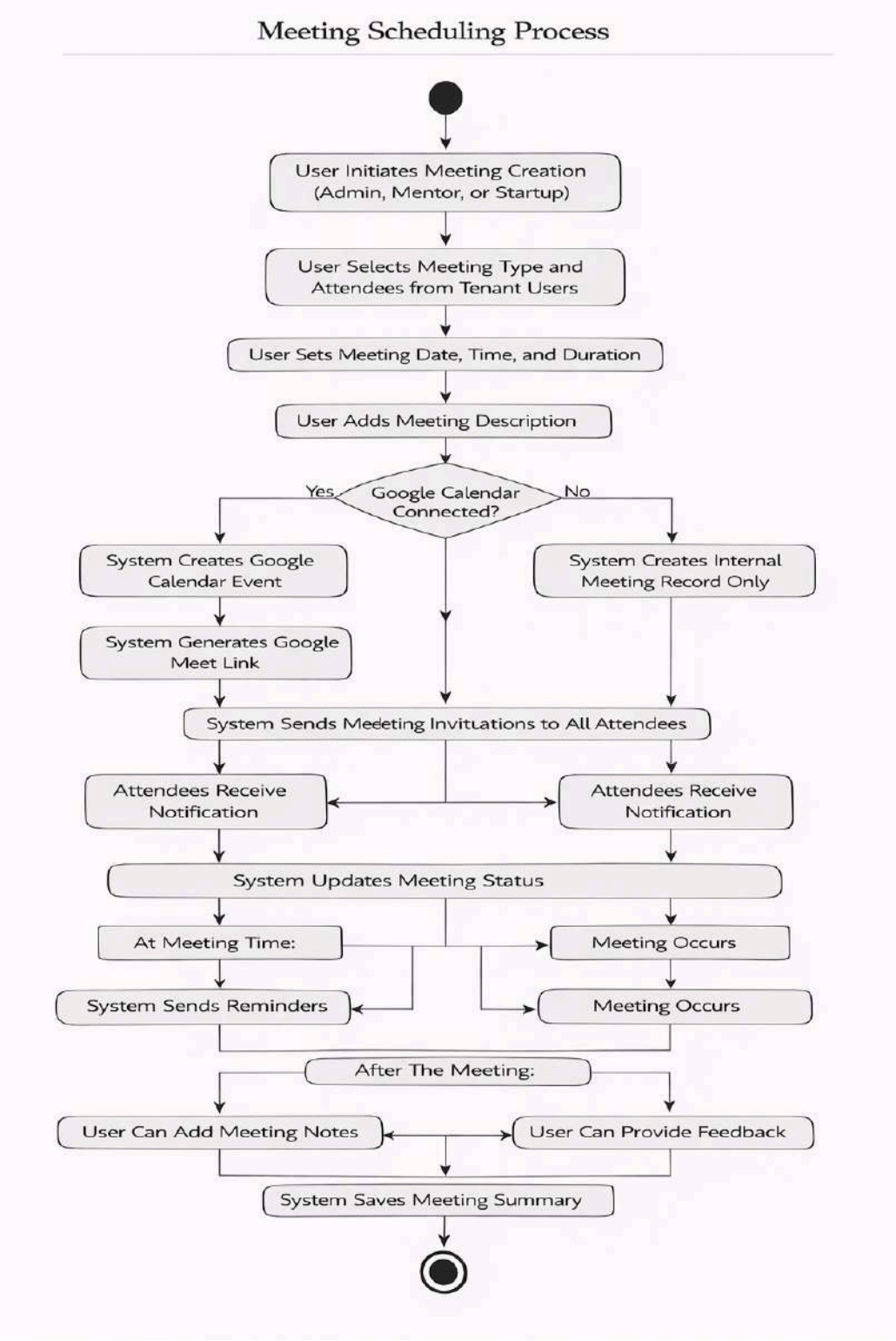
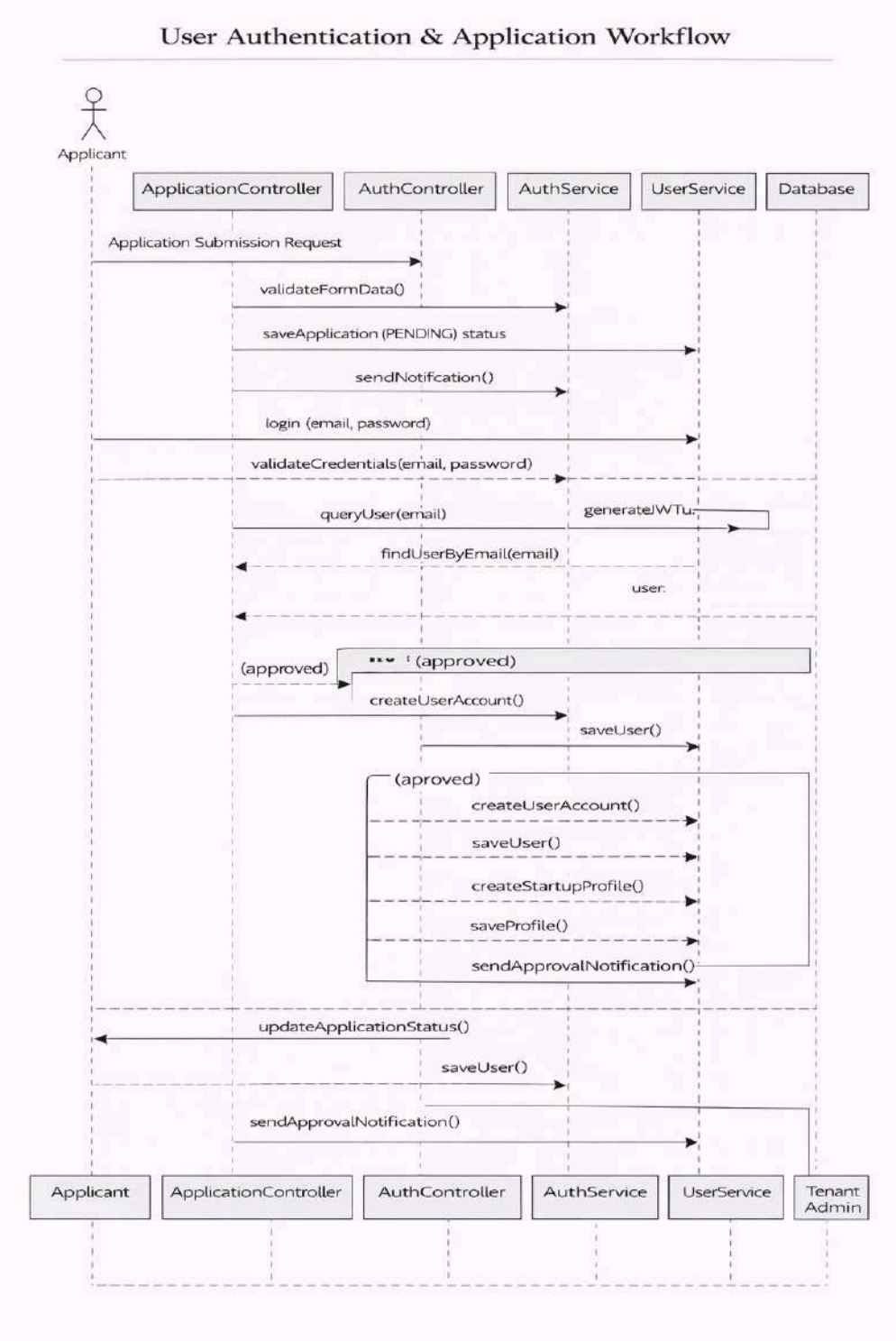
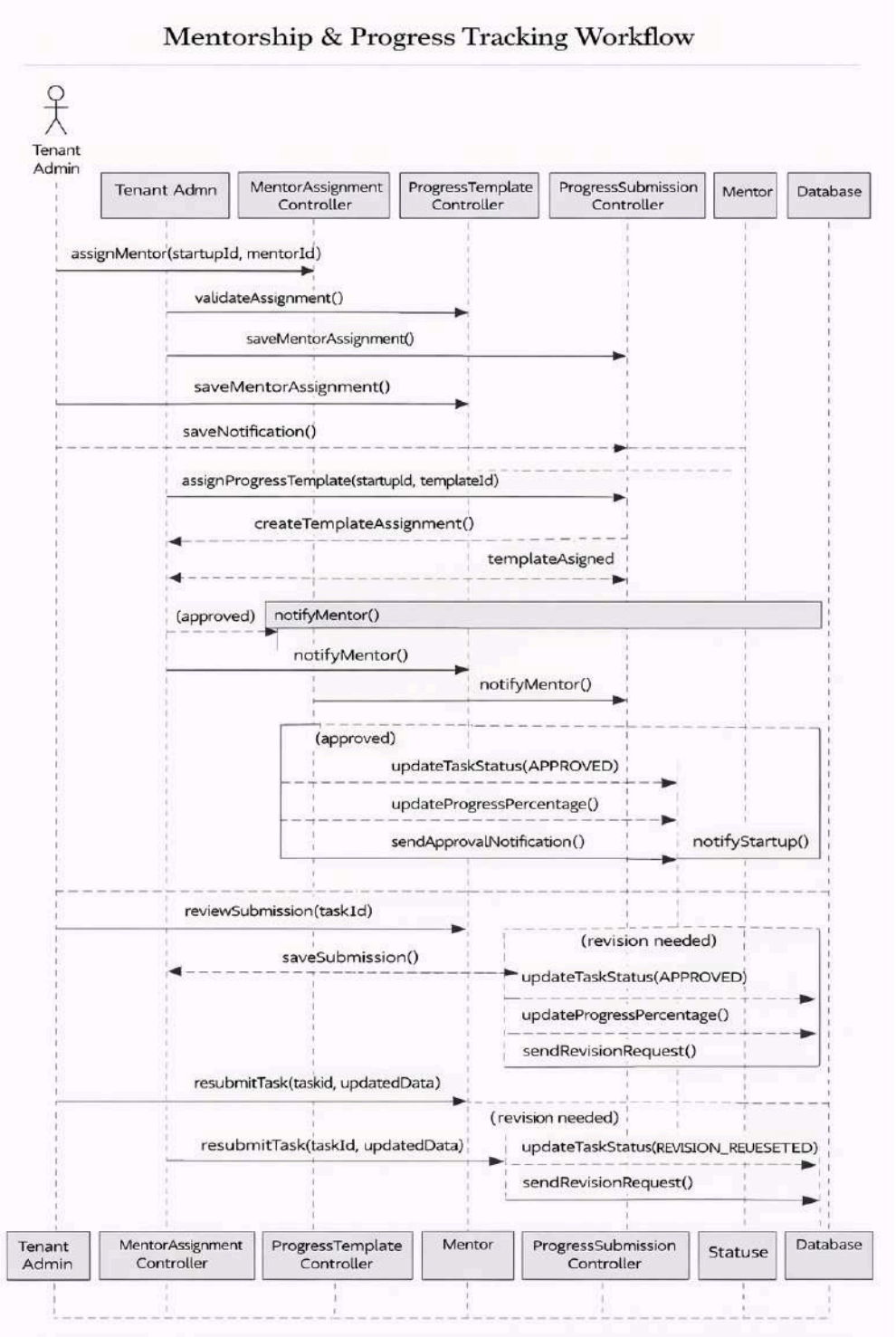


Figure 2.12 Meeting Schedule Activity Diagram

2.3.7. Sequence diagram

Figure 2.13 User Authentication and Application Sequence Diagram




Mentorship & Progress Tracking Workflow
Tenant Admin
Tenant Admn
MentorAssignment Controller
ProgressTemplate Controller
ProgressSubmission Controller
Mentor
Database
assignMentor(startupId, mentorId)
validateAssignment()
saveMentorAssignment()
saveMentorAssignment()
saveNotification()
assignProgressTemplate(startupId, templateId)
createTemplateAssignment()
templateAsigned
(approved)
notifyMentor()
notifyMentor()
notifyMentor()
(approved)
updateTaskStatus(APPROVED)
updateProgressPercentage()
sendApprovalNotification()
notifyStartup()
reviewSubmission(taskId)
(revision needed)
saveSubmission()
updateTaskStatus(APPROVED)
updateProgressPercentage()
sendRevisionRequest()
resubmitTask(taskid, updatedData)
(revision needed)
resubmitTask(taskId, updatedData)
updateTaskStatus(REVISION_REUESETED)
sendRevisionRequest()
Tenant Admin
MentorAssignment Controller
ProgressTemplate Controller
Mentor
ProgressSubmission Controller
Statuse
Database

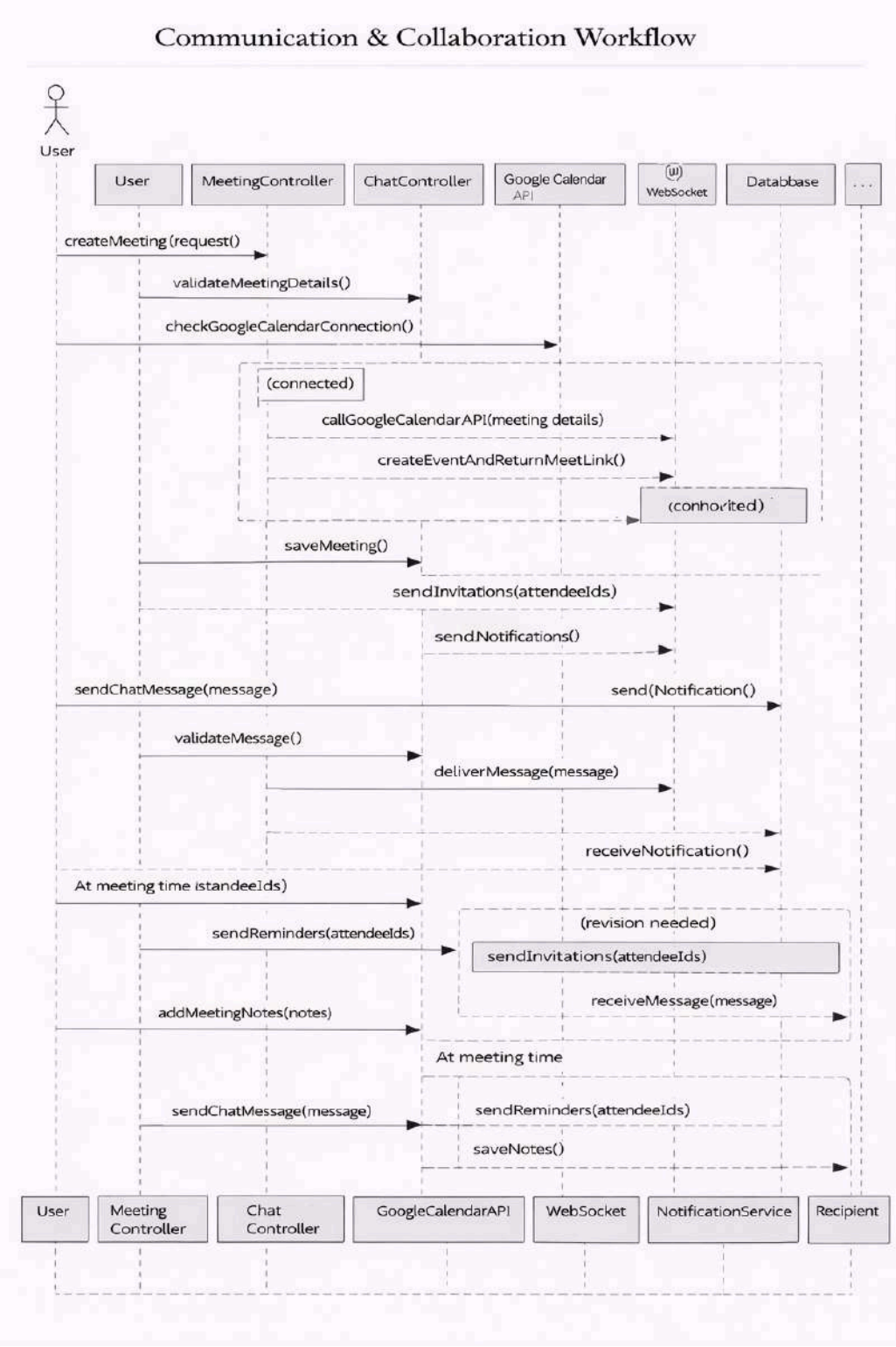


Figure 2.15 Communication and Collaboration Sequence Diagram

2.3.8. Analysis Class Model

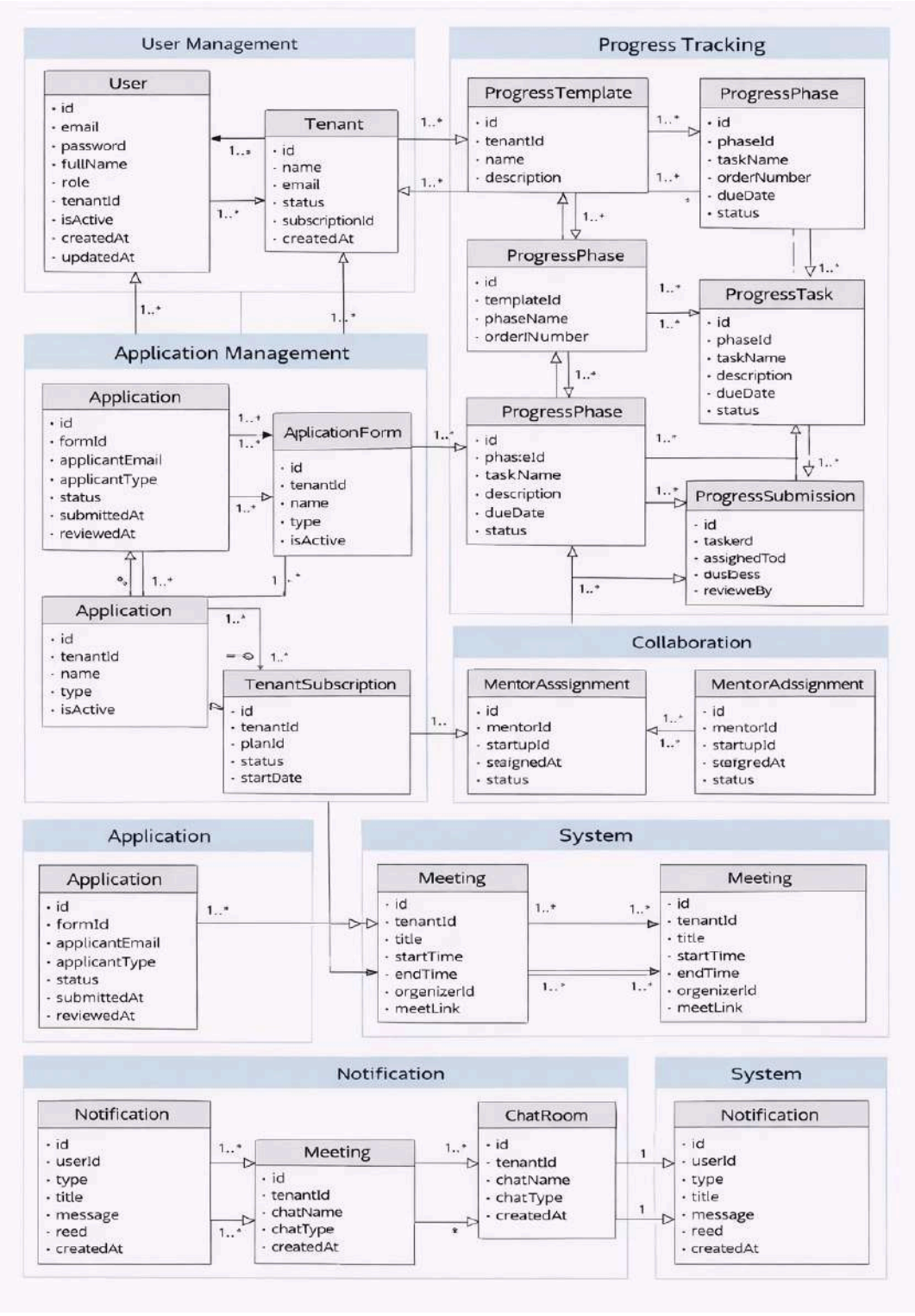


Figure 2.16 Analysis Class Model Diagram

2.3.9. Logic model

Algorithm 1: User Authentication & Login Process

```
BEGIN UserAuthentication(email, password)

  // Step 1: Validate input parameters
  IF email IS NULL OR email IS EMPTY THEN
    RETURN Error: "Email is required"
  END IF

  IF password IS NULL OR password IS EMPTY THEN
    RETURN Error: "Password is required"
  END IF

  // Step 2: Validate email format
  IF email format IS NOT VALID THEN
    RETURN Error: "Invalid email format"
  END IF

  // Step 3: Find user by email in database
  user = FIND User WHERE email = email
  IF user IS NOT FOUND THEN
    RETURN Error: "Invalid credentials"
  END IF

  // Step 4: Check if user account is active
  IF user.isActive = FALSE THEN
    RETURN Error: "Account is deactivated. Please contact administrator"
  END IF

  // Step 5: Verify password
  IF passwordHash(user.password) != passwordHash(password) THEN
    LOG_FAILED_LOGIN_ATTEMPT(user.id)
    RETURN Error: "Invalid credentials"
  END IF

  // Step 6: Generate JWT tokens
  accessToken = GENERATE_JWT_TOKEN(user.id, user.email, user.role, user.tenantId, 15 minutes)
  refreshToken = GENERATE_REFRESH_TOKEN(user.id, 7 days)

  // Step 7: Update user last login timestamp
  user.lastLogin = CURRENT_TIMESTAMP
  UPDATE user IN DATABASE

  // Step 8: Check tenant subscription status
  IF user.tenantId IS NOT NULL THEN
    tenantSubscription = FIND TenantSubscription WHERE tenantId = user.tenantId
    IF tenantSubscription.status = "SUSPENDED" THEN
```

```
        subscriptionStatus = "SUSPENDED"
      ELSE
        subscriptionStatus = tenantSubscription.status
      END IF
    ELSE
      subscriptionStatus = NULL
    END IF

    // Step 9: Prepare response
    authResponse = {
      accessToken: accessToken,
      refreshToken: refreshToken,
      user: {id, email, fullName, role, tenantId, subscriptionStatus},
      tokenType: "Bearer",
      expiresIn: 900
    }

    RETURN Success: authResponse

END UserAuthentication
```

Algorithm 2: Application Approval Workflow

```
BEGIN ApplicationApprovalWorkflow(applicationId, tenantAdminId, decision)

    // Step 1: Validate input parameters
    IF applicationId IS NULL THEN
      RETURN Error: "Application ID is required"
    END IF

    IF decision IS NOT "APPROVE" AND decision IS NOT "REJECT" THEN
      RETURN Error: "Invalid decision. Must be APPROVE or REJECT"
    END IF

    // Step 2: Retrieve application from database
    application = FIND Application WHERE id = applicationId
    IF application IS NOT FOUND THEN
      RETURN Error: "Application not found"
    END IF

    // Step 3: Check if application is in reviewable state
    IF application.status != "PENDING" THEN
      RETURN Error: "Application already processed"
    END IF

    // Step 4: Verify tenant admin has permission
    tenantAdmin = FIND User WHERE id = tenantAdminId
    IF tenantAdmin.role != "TENANT_ADMIN" OR tenantAdmin.tenantId != application.tenantId THEN
```

```
    RETURN Error: "Unauthorized access"
END IF

// Step 5: Update application status
application.status = decision
application.reviewedBy = tenantAdminId
application.reviewedAt = CURRENT_TIMESTAMP
UPDATE application IN DATABASE

// Step 6: If approved, create user account and profile
IF decision = "APPROVE" THEN
    // Step 6.1: Create user account
    newUser = CREATE User {
        email: application.applicantEmail,
        password: GENERATE_TEMPORARY_PASSWORD(),
        fullName: application.firstName + " " + application.lastName,
        role: DETERMINE_ROLE_FROM_APPLICANT_TYPE(application.applicantType),
        tenantId: application.tenantId,
        isActive: TRUE
    }
    SAVE newUser TO DATABASE

    // Step 6.2: Create appropriate profile based on applicant type
    IF application.applicantType = "STARTUP" THEN
        startupProfile = CREATE StartupProfile {
            userId: newUser.id,
            startupName: EXTRACT_FIELD_VALUE(application, "startupName"),
            description: EXTRACT_FIELD_VALUE(application, "description"),
            website: EXTRACT_FIELD_VALUE(application, "website"),
            phone: EXTRACT_FIELD_VALUE(application, "phone"),
            address: EXTRACT_FIELD_VALUE(application, "address"),
            industry: EXTRACT_FIELD_VALUE(application, "industry"),
            graduationStatus: "IN_PROGRAM"
        }
        SAVE startupProfile TO DATABASE
    END IF

    IF application.applicantType = "MENTOR" THEN
        mentorProfile = CREATE MentorProfile {
            userId: newUser.id,
            expertise: EXTRACT_FIELD_VALUE(application, "expertise"),
            bio: EXTRACT_FIELD_VALUE(application, "bio"),
            availability: EXTRACT_FIELD_VALUE(application, "availability")
        }
        SAVE mentorProfile TO DATABASE
    END IF

    IF application.applicantType = "INVESTOR" THEN
        investorProfile = CREATE InvestorProfile {
            userId: newUser.id,
            investmentFocus: EXTRACT_FIELD_VALUE(application, "investmentFocus"),
```

```
            investmentCriteria: EXTRACT_FIELD_VALUE(application, "investmentCriteria"),
            ticketSize: EXTRACT_FIELD_VALUE(application, "ticketSize")
        }
        SAVE investorProfile TO DATABASE
    END IF

    // Step 6.3: Send approval notification with credentials
    SEND_EMAIL(
        to: application.applicantEmail,
        subject: "Application Approved",
            message: "Your application has been approved. Login credentials: " + newUser.email + " / " +
TEMPORARY_PASSWORD
    )
  ELSE
    // Step 7: If rejected, send rejection notification
    SEND_EMAIL(
        to: application.applicantEmail,
        subject: "Application Status Update",
        message: "Your application has been reviewed and unfortunately not approved at this time."
    )
  END IF

  // Step 8: Return success response
  RETURN Success: {applicationId, status: decision, message: "Application processed successfully"}

END ApplicationApprovalWorkflow
```

Algorithm 3: Task Submission & Review Process

```
BEGIN TaskSubmissionReviewProcess(taskId, startupUserId, submissionData, reviewerId, reviewDecision)

  // Step 1: Validate input parameters
  IF taskId IS NULL OR startupUserId IS NULL THEN
    RETURN Error: "Task ID and Startup User ID are required"
  END IF

  // Step 2: Retrieve task and verify assignment
  task = FIND ProgressTask WHERE id = taskId
  IF task IS NOT FOUND THEN
    RETURN Error: "Task not found"
  END IF

  // Step 3: Check if task is assigned to startup
  assignment = FIND ProgressTemplateAssignment WHERE
    templateId = task.phase.templateId AND
    assignedToId = startupUserId AND
    assignedToType = "STARTUP"
  IF assignment IS NOT FOUND THEN
```

```
    RETURN Error: "Task not assigned to this startup"
  END IF

  // Step 4: Handle submission or review based on action
  IF submissionData IS NOT NULL THEN
    // SUBMISSION PROCESS
    // Step 4.1: Validate submission data
    IF submissionData.documents IS EMPTY AND task.requiresDocuments = TRUE THEN
      RETURN Error: "Task requires document submission"
    END IF

    // Step 4.2: Create or update submission
    existingSubmission = FIND ProgressSubmission WHERE taskId = taskId AND assignedToId = startupUserId
    IF existingSubmission EXISTS AND existingSubmission.status = "APPROVED" THEN
      RETURN Error: "Task already approved. Cannot resubmit."
    END IF

    IF existingSubmission EXISTS THEN
      submission = existingSubmission
      submission.status = "SUBMITTED"
      submission.submittedAt = CURRENT_TIMESTAMP
    ELSE
      submission = CREATE ProgressSubmission {
        taskId: taskId,
        assignedToId: startupUserId,
        status: "SUBMITTED",
        submittedAt: CURRENT_TIMESTAMP
      }
    END IF
    SAVE submission TO DATABASE

    // Step 4.3: Save submission files
    FOR EACH document IN submissionData.documents DO
      submissionFile = CREATE ProgressSubmissionFile {
        submissionId: submission.id,
        fileName: document.fileName,
        fileUrl: document.fileUrl,
        fileType: document.fileType
      }
      SAVE submissionFile TO DATABASE
    END FOR

    // Step 4.4: Update task status
    task.status = "SUBMITTED"
    UPDATE task IN DATABASE

    // Step 4.5: Notify assigned mentor
    IF task.mentorId IS NOT NULL THEN
      mentor = FIND User WHERE id = task.mentorId
      CREATE Notification {
        userId: mentor.id,
```

```
            type: "task_submitted",
            title: "New Task Submission",
            message: "Startup has submitted task: " + task.taskName,
            actionUrl: "/progress/submissions/" + submission.id
        }
    END IF

    RETURN Success: {submissionId: submission.id, status: "SUBMITTED"}

ELSE IF reviewDecision IS NOT NULL THEN
    // REVIEW PROCESS
    // Step 5.1: Verify reviewer permissions
    reviewer = FIND User WHERE id = reviewerId
    IF reviewer.role != "MENTOR" AND reviewer.role != "TENANT_ADMIN" THEN
        RETURN Error: "Unauthorized to review tasks"
    END IF

    // Step 5.2: Find submission
    submission = FIND ProgressSubmission WHERE taskId = taskId AND assignedToId = startupUserId
    IF submission IS NOT FOUND THEN
        RETURN Error: "Submission not found"
    END IF

    IF submission.status != "SUBMITTED" AND submission.status != "RESUBMITTED" THEN
        RETURN Error: "Submission not in reviewable state"
    END IF

    // Step 5.3: Process review decision
    IF reviewDecision = "APPROVE" THEN
        submission.status = "APPROVED"
        submission.approvedAt = CURRENT_TIMESTAMP
        submission.approvedBy = reviewerId
        task.status = "COMPLETED"
        UPDATE task IN DATABASE

        // Update progress percentage
        CALCULATE_AND_UPDATE_PROGRESS(startupUserId, task.phase.templateId)

        // Notify startup
        CREATE Notification {
            userId: startupUserId,
            type: "task_approved",
            title: "Task Approved",
            message: "Your submission for task: " + task.taskName + " has been approved",
            actionUrl: "/progress/tasks/" + taskId
        }

    ELSE IF reviewDecision = "REVISION" THEN
        submission.status = "NEEDS_REVISION"
        submission.feedback = reviewDecision.feedback
        submission.reviewedBy = reviewerId
```

```
        submission.reviewedAt = CURRENT_TIMESTAMP
        task.status = "NEEDS_REVISION"
        UPDATE task IN DATABASE

        // Notify startup for revision
        CREATE Notification {
          userId: startupUserId,
          type: "task_revision",
          title: "Task Needs Revision",
                message: "Your submission for task: " + task.taskName + " needs revision. Feedback: " +
reviewDecision.feedback,
          actionUrl: "/progress/tasks/" + taskId
        }
      END IF

      UPDATE submission IN DATABASE
      RETURN Success: {submissionId: submission.id, status: submission.status}
    END IF

END TaskSubmissionReviewProcess

FUNCTION CALCULATE_AND_UPDATE_PROGRESS(startupUserId, templateId)
    // Calculate overall progress percentage
    allTasks = FIND ALL ProgressTask WHERE phase.templateId = templateId
    completedTasks = COUNT ProgressSubmission WHERE
      task.phase.templateId = templateId AND
      assignedToId = startupUserId AND
      status = "APPROVED"
    progressPercentage = (completedTasks / allTasks.count) * 100

    // Update assignment progress
    assignment = FIND ProgressTemplateAssignment WHERE
      templateId = templateId AND
      assignedToId = startupUserId
    assignment.progressPercentage = progressPercentage
    UPDATE assignment IN DATABASE
END FUNCTION
```

Algorithm 4: Progress Tracking & Milestone Calculation

```
BEGIN ProgressTrackingAndMilestoneCalculation(startupUserId)

    // Step 1: Validate input
    IF startupUserId IS NULL THEN
      RETURN Error: "Startup User ID is required"
    END IF

    // Step 2: Retrieve all assigned templates for startup
    assignments = FIND ALL ProgressTemplateAssignment WHERE
```

```
    assignedToId = startupUserId AND
    assignedToType = "STARTUP"

IF assignments IS EMPTY THEN
    RETURN {milestones: [], overallProgress: 0}
END IF

// Step 3: Calculate progress for each template
milestoneData = []
totalOverallProgress = 0

FOR EACH assignment IN assignments DO
    template = FIND ProgressTemplate WHERE id = assignment.templateId
    phases = FIND ALL ProgressPhase WHERE templateId = template.id ORDER BY orderNumber

    // Step 3.1: Calculate phase progress
    phaseProgress = []
    FOR EACH phase IN phases DO
        tasks = FIND ALL ProgressTask WHERE phaseId = phase.id
        completedTasks = COUNT ProgressSubmission WHERE
            task.phaseId = phase.id AND
            assignedToId = startupUserId AND
            status = "APPROVED"

        phasePercentage = 0
        IF tasks.count > 0 THEN
            phasePercentage = (completedTasks / tasks.count) * 100
        END IF

        phaseProgress.ADD({
            phaseId: phase.id,
            phaseName: phase.phaseName,
            completedTasks: completedTasks,
            totalTasks: tasks.count,
            percentage: phasePercentage,
            status: IF phasePercentage = 100 THEN "COMPLETED" ELSE "IN_PROGRESS" END IF
        })
    END FOR

    // Step 3.2: Calculate template overall progress
    templateCompletedPhases = COUNT phaseProgress WHERE percentage = 100
    templateProgressPercentage = 0
    IF phases.count > 0 THEN
        templateProgressPercentage = (templateCompletedPhases / phases.count) * 100
    END IF

    // Step 3.3: Determine template status
    templateStatus = "NOT_STARTED"
    IF templateProgressPercentage > 0 AND templateProgressPercentage < 100 THEN
        templateStatus = "IN_PROGRESS"
    ELSE IF templateProgressPercentage = 100 THEN
```

```
        templateStatus = "COMPLETED"
    END IF

    // Step 3.4: Update assignment progress
    assignment.progressPercentage = templateProgressPercentage
    assignment.lastUpdated = CURRENT_TIMESTAMP
    UPDATE assignment IN DATABASE

    // Step 3.5: Add to milestone data
    milestoneData.ADD({
        templateId: template.id,
        templateName: template.name,
        progressPercentage: templateProgressPercentage,
        status: templateStatus,
        phases: phaseProgress,
        assignedAt: assignment.assignedAt
    })

    totalOverallProgress = totalOverallProgress + templateProgressPercentage
END FOR

// Step 4: Calculate overall progress across all templates
overallProgress = 0
IF assignments.count > 0 THEN
    overallProgress = totalOverallProgress / assignments.count
END IF

// Step 5: Determine if startup is ready for graduation
allTemplatesCompleted = TRUE
FOR EACH milestone IN milestoneData DO
    IF milestone.status != "COMPLETED" THEN
        allTemplatesCompleted = FALSE
        BREAK
    END IF
END FOR

graduationEligible = FALSE
IF allTemplatesCompleted AND overallProgress = 100 THEN
    graduationEligible = TRUE
END IF

// Step 6: Return comprehensive milestone data
RETURN {
    milestones: milestoneData,
    overallProgress: overallProgress,
    totalTemplates: assignments.count,
    completedTemplates: COUNT milestoneData WHERE status = "COMPLETED",
    graduationEligible: graduationEligible,
    lastUpdated: CURRENT_TIMESTAMP
}
```

```
END ProgressTrackingAndMilestoneCalculation
```

Algorithm 5: Subscription Feature Access Check

```
BEGIN SubscriptionFeatureAccessCheck(tenantId, featureCode, endpointPath)

  // Step 1: Validate input parameters
  IF tenantId IS NULL THEN
    RETURN Error: "Tenant ID is required"
  END IF

  IF featureCode IS NULL AND endpointPath IS NULL THEN
    RETURN Error: "Either feature code or endpoint path is required"
  END IF

  // Step 2: Get tenant subscription
  subscription = FIND TenantSubscription WHERE tenantId = tenantId
  IF subscription IS NOT FOUND THEN
    RETURN {hasAccess: FALSE, reason: "No subscription found"}
  END IF

  // Step 3: Check subscription status
  IF subscription.status = "SUSPENDED" THEN
    RETURN {hasAccess: FALSE, reason: "Subscription is suspended"}
  END IF

  IF subscription.status = "PENDING" THEN
    RETURN {hasAccess: FALSE, reason: "Subscription is pending approval"}
  END IF

  // Step 4: Determine feature code if endpoint path provided
  IF featureCode IS NULL THEN
    featureMapping = FIND FeatureMapping WHERE endpointPath = endpointPath
    IF featureMapping IS NOT FOUND THEN
      // Core feature, always available
      RETURN {hasAccess: TRUE, reason: "Core feature"}
    END IF
    featureCode = featureMapping.featureCode
  END IF

  // Step 5: Check if feature exists and is active
  feature = FIND Feature WHERE code = featureCode
  IF feature IS NOT FOUND THEN
    RETURN {hasAccess: FALSE, reason: "Feature not found"}
  END IF

  IF feature.isActive = FALSE THEN
    RETURN {hasAccess: FALSE, reason: "Feature is inactive"}
  END IF
```

```
    // Step 6: Check if tenant has this feature in subscription
    subscriptionFeature = FIND TenantSubscriptionFeature WHERE
        subscriptionId = subscription.id AND
        featureId = feature.id

    IF subscriptionFeature IS NOT FOUND THEN
        RETURN {
            hasAccess: FALSE,
            reason: "Feature not included in subscription",
            featureName: feature.name,
            featurePrice: feature.price
        }
    END IF

    // Step 7: Check if feature access is expired (if applicable)
    IF subscriptionFeature.expiresAt IS NOT NULL THEN
        IF subscriptionFeature.expiresAt < CURRENT_TIMESTAMP THEN
            RETURN {hasAccess: FALSE, reason: "Feature access has expired"}
        END IF
    END IF

    // Step 8: Return success
    RETURN {
        hasAccess: TRUE,
        featureCode: feature.code,
        featureName: feature.name,
        subscriptionStatus: subscription.status
    }

END SubscriptionFeatureAccessCheck

// Additional function for endpoint access check
FUNCTION CheckEndpointAccess(tenantId, endpointPath, httpMethod)
    // Map endpoint to feature code
    featureCode = MAP_ENDPOINT_TO_FEATURE(endpointPath, httpMethod)

    // Check feature access
    accessResult = SubscriptionFeatureAccessCheck(tenantId, featureCode, endpointPath)

    RETURN accessResult.hasAccess
END FUNCTION
```

Algorithm 6: Graduation & Alumni Conversion Process

```
BEGIN GraduationAndAlumniConversionProcess(startupUserId, tenantAdminId, convertToAlumni)

    // Step 1: Validate input parameters
```

```
IF startupUserId IS NULL OR tenantAdminId IS NULL THEN
    RETURN Error: "Startup User ID and Tenant Admin ID are required"
END IF

// Step 2: Verify tenant admin permissions
tenantAdmin = FIND User WHERE id = tenantAdminId
IF tenantAdmin.role != "TENANT_ADMIN" AND tenantAdmin.role != "SUPER_ADMIN" THEN
    RETURN Error: "Unauthorized. Only admins can graduate startups"
END IF

// Step 3: Retrieve startup user and profile
startupUser = FIND User WHERE id = startupUserId
IF startupUser IS NOT FOUND THEN
    RETURN Error: "Startup user not found"
END IF

IF startupUser.role != "STARTUP" THEN
    RETURN Error: "User is not a startup"
END IF

IF tenantAdmin.tenantId != startupUser.tenantId THEN
    RETURN Error: "Unauthorized. Startup belongs to different tenant"
END IF

startupProfile = FIND StartupProfile WHERE userId = startupUserId
IF startupProfile IS NOT FOUND THEN
    RETURN Error: "Startup profile not found"
END IF

// Step 4: Check current graduation status
IF startupProfile.graduationStatus = "GRADUATED" THEN
    RETURN Error: "Startup is already graduated"
END IF

// Step 5: Verify graduation eligibility (optional check)
progressData = ProgressTrackingAndMilestoneCalculation(startupUserId)
IF progressData.graduationEligible = FALSE THEN
    // Allow admin to override if needed, but log warning
    LOG_WARNING("Graduating startup that may not have completed all requirements")
END IF

// Step 6: Update startup profile graduation status
startupProfile.graduationStatus = "GRADUATED"
startupProfile.graduatedAt = CURRENT_TIMESTAMP
UPDATE startupProfile IN DATABASE

// Step 7: Handle alumni conversion if requested
IF convertToAlumni = TRUE THEN
    // Step 7.1: Check if already alumni
    IF startupUser.role = "ALUMNI" THEN
        RETURN Error: "Startup is already an alumni"
```

```
    END IF

    // Step 7.2: Convert user role to ALUMNI
    startupUser.role = "ALUMNI"
    UPDATE startupUser IN DATABASE

    // Step 7.3: Create alumni profile if it doesn't exist
    alumniProfile = FIND AlumniProfile WHERE userId = startupUserId
    IF alumniProfile IS NOT FOUND THEN
       alumniProfile = CREATE AlumniProfile {
          userId: startupUserId,
          milestoneHistory: EXTRACT_MILESTONE_HISTORY(startupUserId),
          joinedAsAlumni: CURRENT_TIMESTAMP
       }
       SAVE alumniProfile TO DATABASE
    END IF

    // Step 7.4: Generate new credentials for alumni (optional, for security)
    newPassword = GENERATE_RANDOM_PASSWORD()
    startupUser.password = HASH_PASSWORD(newPassword)
    UPDATE startupUser IN DATABASE

    // Step 7.5: Send alumni credentials email
    SEND_EMAIL(
       to: startupUser.email,
       subject: "Congratulations! You are now an Alumni",
       message: "Congratulations on graduating! Your account has been converted to Alumni status. " +
             "New login credentials: " + startupUser.email + " / " + newPassword +
             " Please change your password after first login."
    )

    // Step 7.6: Create notification
    CREATE Notification {
       userId: startupUserId,
       type: "alumni_converted",
       title: "Welcome to Alumni Network",
       message: "Congratulations! You have been converted to Alumni status.",
       actionUrl: "/alumni/dashboard"
    }

    RETURN Success: {
       message: "Startup graduated and converted to Alumni",
       startupName: startupProfile.startupName,
       graduatedAt: startupProfile.graduatedAt,
       alumniProfileId: alumniProfile.id
    }
 ELSE
    // Step 8: If not converting to alumni, just send graduation notification
    CREATE Notification {
       userId: startupUserId,
       type: "startup_graduated",
```

```
        title: "Congratulations on Graduation",
        message: "Congratulations! You have successfully graduated from the incubation program.",
        actionUrl: "/startup/dashboard"
      }

      SEND_EMAIL(
        to: startupUser.email,
        subject: "Congratulations on Graduation",
        message: "Congratulations! You have successfully completed the incubation program."
      )

      RETURN Success: {
        message: "Startup graduated successfully",
        startupName: startupProfile.startupName,
        graduatedAt: startupProfile.graduatedAt,
        convertToAlumni: FALSE
      }
    END IF

END GraduationAndAlumniConversionProcess

FUNCTION EXTRACT_MILESTONE_HISTORY(startupUserId)
    // Extract all completed milestones and progress data
    progressData = ProgressTrackingAndMilestoneCalculation(startupUserId)
    milestoneHistory = {
      completedTemplates: progressData.completedTemplates,
      totalTemplates: progressData.totalTemplates,
      overallProgress: progressData.overallProgress,
      milestones: progressData.milestones,
      graduationDate: CURRENT_TIMESTAMP
    }
    RETURN milestoneHistory
END FUNCTION
```

2.4. Non functional requirement

1. Performance Requirements

The system must support at least 400 concurrent users per tenant with response times under 2 seconds for standard operations. Database queries must complete within 500ms for typical operations, and complex analytics reports must be generated within 10 seconds. The system must handle at least 10,000 API requests per minute per tenant, with file uploads up to 50MB completing within 30 seconds. Page load times must be under 3 seconds on standard broadband connections, and real-time chat messages must be delivered within 1 second.

2. Reliability Requirements

The system must achieve 99.9% uptime, with planned maintenance windows communicated 48 hours in advance. All critical operations must include error handling and graceful degradation. Data loss must not

exceed 1 hour of transactions in case of system failure, and automatic recovery mechanisms must restore service within 15 minutes of failure detection. The system must log all errors and provide monitoring alerts for critical failures.

3. Security Requirements

The system must implement end-to-end encryption for sensitive data, use HTTPS for all communications, and store passwords using BCrypt hashing. JWT tokens must expire after 15 minutes for access tokens and 7 days for refresh tokens, with secure token storage. Role-based access control must enforce tenant data isolation, preventing cross-tenant data access. All user inputs must be validated and sanitized to prevent SQL injection, XSS, and CSRF attacks. The system must maintain audit logs for all administrative actions and sensitive operations.

4. Scalability Requirements

The system must support horizontal scaling to accommodate up to 50 tenants with 400 users each. Database architecture must support read replicas and connection pooling for high-traffic scenarios. The system must be designed to handle 10x growth in users and data without major architectural changes. File storage must support cloud-based solutions with automatic scaling, and the application must support load balancing across multiple server instances.

5. Usability Requirements

The system must provide an intuitive user interface that requires minimal training, with a consistent design across all modules. All forms must include clear validation messages and error handling. The interface must be responsive and accessible on desktop, tablet, and mobile devices. Navigation must be logical with breadcrumbs and clear menu structures. Help documentation and tooltips must be available for complex features.

6. Maintainability Requirements

The system must follow modular architecture with clear separation of concerns, making code updates and bug fixes straightforward. All code must include comprehensive documentation and follow consistent coding standards. The system must support version control and allow for incremental deployments without downtime. Database schema changes must be backward compatible or include migration scripts. Logging and monitoring must provide sufficient information for troubleshooting.

7. Compatibility Requirements

The system must support modern web browsers including Chrome, Firefox, Safari, and Edge (latest 2 versions). The backend must run on Java 21+ with Spring Boot framework, and the frontend must support React 18+. Database must be compatible with PostgreSQL 14+ or higher. The system must integrate with Google Calendar API and support standard REST API protocols.

8. Data Integrity Requirements

The system must enforce referential integrity constraints at the database level and implement transaction management to ensure data consistency. All critical operations must be atomic, and the system must prevent orphaned records. Data validation must occur at both client and server levels, and foreign key relationships must be properly maintained. The system must prevent concurrent modification conflicts using optimistic locking mechanisms.

9. Backup and Recovery Requirements

The system must perform automated daily backups of all databases with retention of 30 days of daily backups and 12 months of weekly backups. Backup restoration must be tested quarterly, and the system must support point-in-time recovery within the last 7 days. All backups must be encrypted and stored in geographically separate locations. Recovery time objective (RTO) must be less than 4 hours, and recovery point objective (RPO) must be less than 1 hour.

10. Response Time Requirements

Standard CRUD operations must respond within 500ms, search and filter operations within 1 second, and report generation within 10 seconds. Real-time features like chat messaging must deliver messages within 1 second, and notification delivery must occur within 5 seconds of event trigger. File uploads must show progress indicators and complete within 30 seconds for files up to 50MB. Page navigation must be instantaneous with client-side routing.

11. Throughput Requirements

The system must handle at least 10,000 API requests per minute per tenant, with peak capacity of 50,000 requests per minute across all tenants. Database must support at least 1,000 concurrent connections with connection pooling. File upload throughput must support at least 100 simultaneous uploads, and email notifications must be sent at a rate of at least 1,000 emails per minute. WebSocket connections must support at least 5,000 concurrent connections per server instance.

12. Storage Requirements

The system must support scalable storage for user-uploaded files with automatic archival of files older than 2 years. Database storage must be optimized with indexing strategies to maintain query performance as data grows. Log files must be rotated daily and archived after 90 days. The system must support at least 1TB of file storage per tenant with automatic expansion capabilities. Image files must be automatically compressed and optimized for web delivery.

13. Network Requirements

The system must operate over standard HTTP/HTTPS protocols with minimum bandwidth of 10 Mbps per tenant for optimal performance. The system must support CDN integration for static content delivery to reduce latency globally. API endpoints must support CORS configuration for authorized domains, and WebSocket connections must support secure WSS protocol. Network timeouts must be configured

appropriately with retry mechanisms for transient failures.

2.5. System Requirement

2.5.1. Hardware requirements

For server deployment, the system requires a minimum of 8 CPU cores (Intel Xeon or AMD EPYC, 2.5 GHz or higher), 32GB RAM (64GB recommended for production with high user load), and 500GB SSD storage (1TB+ recommended for production with database and file storage). The server should support RAID configuration for data redundancy, network interface cards with at least 1Gbps bandwidth, and power supply redundancy. For development environments, minimum specifications include 4 CPU cores, 16GB RAM, and 250GB storage. The system requires sufficient disk I/O performance (minimum 10,000 IOPS) for database operations and file uploads, and memory should be expandable to support future growth. Client-side hardware requirements are minimal, requiring standard desktop or laptop computers with 4GB RAM, modern processors, and internet connectivity with minimum 5Mbps bandwidth for optimal performance.

2.5.2. Software requirements

The system requires server-side software including Java Development Kit (JDK) version 21 or higher, Spring Boot framework version 3.2+, PostgreSQL database version 14 or higher, and an application server (embedded Tomcat with Spring Boot or standalone Tomcat 10+). The operating system must be Linux (Ubuntu 20.04 LTS or higher, CentOS 8+ or RHEL 8+) or Windows Server 2019/2022, with Node.js version 18+ and npm for frontend build processes. Client-side requirements include modern web browsers (Google Chrome 100+, Mozilla Firefox 100+, Microsoft Edge 100+, Safari 15+) with JavaScript enabled, and stable internet connectivity. For development, developers need JDK 21+, Maven 3.8+, Node.js 18+, npm or yarn, Git version control, and an IDE such as IntelliJ IDEA, Eclipse, or VS Code. Additional software includes Docker and Docker Compose for containerized deployment, and Nginx or Apache as reverse proxy and load balancer for production environments.

2.6. Key abstraction with CRC analysis

2.6.1. Conceptual modeling: Class diagram

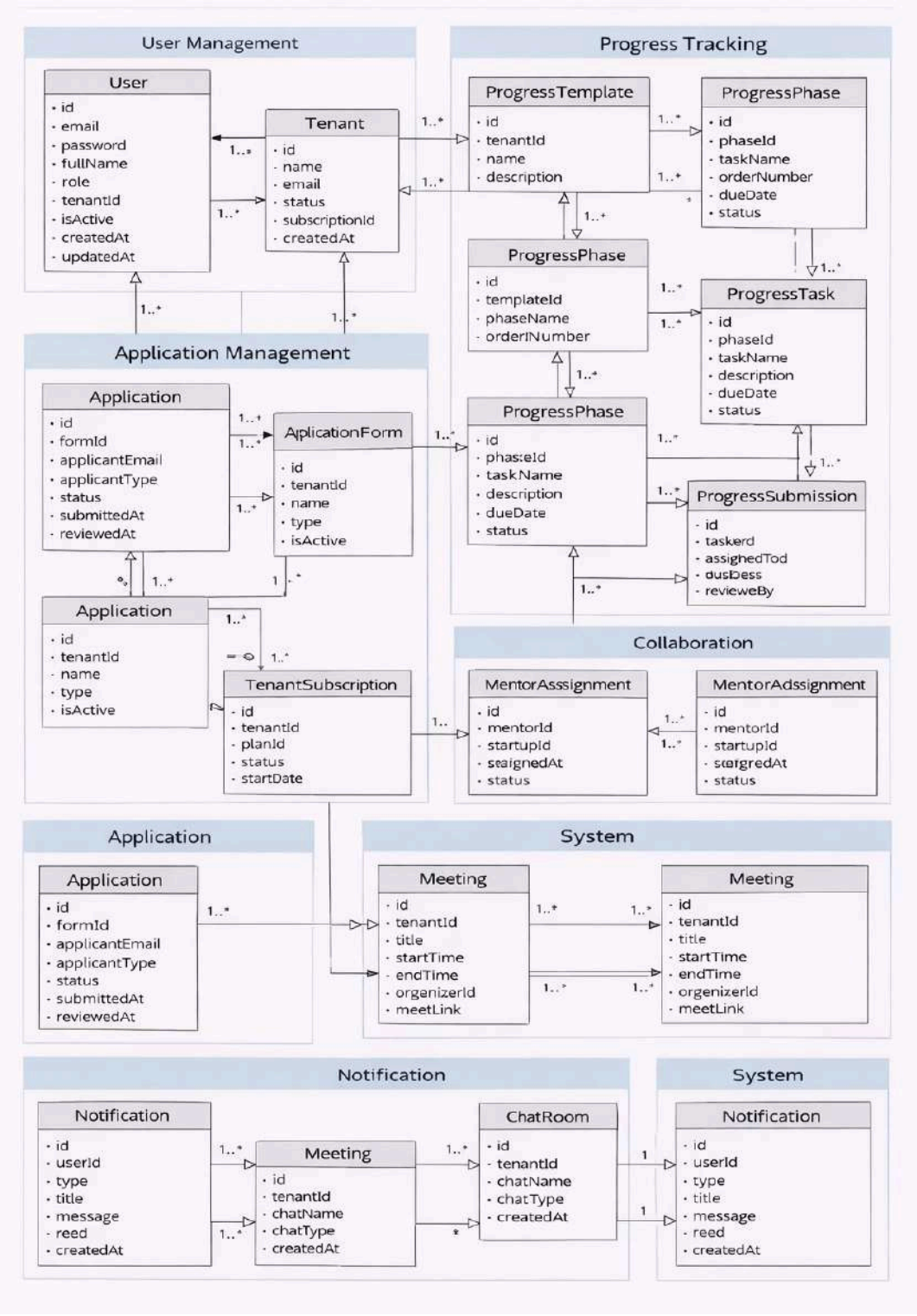


Figure 2.17 Conceptual modeling: Class diagram

2.6.2. Identifying change case

Table 2.6.1: Change Case

Change Case ID	Change Description	Likelihood	Impact	Affected Components
CC!	Add mobile application support (iOS and Android native apps) to extend system accessibility beyond web browser interface	High	Medium	Frontend Architecture, Authentication Module, API Gateway, Notification System, File Upload Service
CC2	Integrate additional video conferencing platforms (Zoom, Microsoft Teams) alongside Google Meet for meeting scheduling and collaboration	Medium	Medium	Meeting Management Module, Calendar Integration Service, OAuth Authentication Module
CC3	Add multi-language support (internationalizatio n) to support multiple languages for global tenant expansion	Medium	Medium	Frontend UI Components, Backend Localization Service, Database Content Management, Notification Templates
CC4	Integrate with external funding platforms (AngelList, Crunchbase, Kickstarter) to enable direct investment connections and fundraising features	Medium	High	Investor Network Module, External API Integration Service, Payment Processing Module, Investor Profile Module

CC5	Add blockchain-based certificate verification system for startup graduation certificates and milestone achievements to ensure authenticity	Low	High	Graduation Management Module, Certificate Generation Service, Blockchain Integration Service, Verification API

Chapter 3: System Design

The **Incubation and Innovation Management System (IIMS)** is a centralized platform designed to manage the full lifecycle of innovation projects—from application and selection to graduation and tracking. The system supports a diverse user base, including administrators, startups, mentors, and investors, through a modular and scalable architecture.

The architecture is built on a **layered approach** (Presentation, Application, Domain, and Data Access), ensuring a clear separation of concerns. This modularity allows for the seamless addition of new features, such as funding schemes or specialized reporting, with minimal impact on the core system.

Design Goals and Criteria

- **Performance:** IIMS is optimized for high concurrency, utilizing database indexing, caching, and batch processing to ensure standard operations respond within **2–3 seconds**. The architecture supports both horizontal and vertical scaling.
- **Dependability:** The system ensures reliability through centralized error handling, automated logging, and strict data validation rules. Regular backups and idempotent operations are implemented to prevent data loss and support system recovery.
- **Usability:** The UI features a role-tailored design, ensuring that users (startups, mentors, etc.) only see data relevant to their specific tasks. The interface is fully responsive for desktop and mobile access.
- **Security:** Security is maintained via **Role-Based Access Control (RBAC)**, HTTPS/TLS encryption for data in transit, and salted/hashed password storage. The system is protected against common

vulnerabilities like SQL injection and XSS.

- **Scalability & Extensibility:** The use of clear APIs and modular boundaries allows IIMS to integrate with external systems (e.g., University HR or Financial systems) and scale toward a microservices architecture as organizational needs grow.
- **Maintainability:** By adhering to consistent coding standards and data-driven configurations, administrators can update program criteria or notification templates without requiring direct code modifications.
- **Data Integrity:** As the "source of truth" for startup data, the system enforces strong relational integrity and supports longitudinal reporting on performance indicators and program outcomes.

3.1 Architectural Design

This section describes the **actual, current software architecture** of the Incubation and Innovation Management System (IIMS). At this stage, the system consists of:

- A **web frontend** (browser-based UI), and
- A **backend** implemented using the **MVC (Model–View–Controller)** architectural pattern, connected to a relational database.

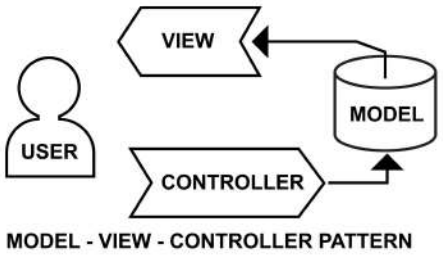


Figure 3.1.1 MVC

Current Software Architecture

Backend:

- Implemented using **MVC Architecture**.
- **Controllers** handle HTTP requests, call service/business logic, and return views or JSON responses.
- **Models** represent domain entities (e.g., users, programs, startups, subscriptions, etc.) mapped to database tables.

- **Views** (where used) render HTML pages or templates for the web frontend; in some cases, controllers return JSON for AJAX calls from the frontend.

Web Frontend:

- Implemented as a web application that consumes the backend through MVC views and/or REST-style endpoints.
- Uses standard web technologies (**React**) to present pages for different roles (admins, staff, startups, mentors, etc.).
- The frontend is tightly integrated with the backend routes (URLs map to controllers/actions), and uses forms, links, and AJAX calls to send data to the backend.

Database Layer:

- A **relational database** stores persistent data such as users, roles, programs, cohorts, applications, evaluations, subscriptions/features, etc.
- The backend's Models and repository/data-access components communicate with the database using ORM or SQL (depending on your implementation).

Backend – MVC Architecture

The backend follows the classic MVC layers:

1. Controller Layer

- Receives HTTP requests from the web frontend (e.g., login, create program, submit application, manage subscriptions).
- Validates input (basic validation), invokes the appropriate service or business logic, and decides which view or response to return.
- Acts as the integration point between UI and business logic.

2. Service / Business Logic Layer

- Encapsulates the core use cases of IIMS:
    - User authentication and authorization.
    - Managing incubation programs and cohorts.

    - Handling applications and evaluations.
    - Managing subscriptions/features and access control.
    - Generating reports and summaries.
- Ensures business rules are correctly applied before updating the database (for example: only specific roles can approve applications, only valid subscriptions can access premium features, etc.).

3. Model & Data Access Layer

- Models map to database tables and represent entities such as **User, Role, Program, Startup, Application, Evaluation, Subscription, Feature, etc.**
- Data access components (repositories/DAOs) perform CRUD operations and queries.
- This layer hides raw database operations from controllers and services, improving cohesion and maintainability.

Web Frontend

UI Structure The web UI is organized into different pages/screens based on roles and features:

- Login and registration pages.
- Dashboards for admins/incubator staff.
- Startup-facing pages (applications, status tracking).
- Mentor/evaluator pages (assigned startups, evaluations).
- Subscription/feature access pages.
- Each page corresponds to backend routes (controller actions) and uses forms and links to trigger backend logic.

Interaction with Backend

- **Forms** submit data to specific controller endpoints (e.g., /login, /program/create, /application/submit).
- Some pages may use **AJAX/Fetch/XHR** to call JSON endpoints for dynamic interactions (e.g., loading lists, filtering tables without full page reload).
- The frontend is therefore tightly coupled to the backend's URL and controller structure, but logically decoupled from the underlying database and internal business logic.

Subsystem Diagram (Conceptual – Current State)

Logically, the system is organized into subsystems within the single MVC backend. They communicate with each other through service calls and shared models, while the web frontend communicates with them via controllers.

1. User and Role Management Subsystem

- **Services it provides:** User registration, login, logout, and password management; Role and permission handling; Authorization checks for protected actions and routes.
- **Services it accepts:** Authentication/authorization requests from other subsystems; Login/profile-update requests from the web frontend.

2. Program and Cohort Management Subsystem

- **Services it provides:** Create and update incubation/innovation programs and cohorts; Manage program lifecycle (open/close applications, move to incubation, etc.).
- **Services it accepts:** Requests from admins/staff via the web frontend; Queries from the Application subsystem to check program status and rules.

3. Application and Evaluation Management Subsystem

- **Services it provides:** Allow centers to create, edit, and submit applications; Manage evaluation processes (compute results); Handle decisions (acceptance, rejection).
- **Services it accepts:** Application submissions and updates from startup users; Evaluation inputs from evaluators/mentors via the web frontend; Program details and eligibility rules from the Program subsystem.

4. Subscription and Feature Access Subsystem

- **Services it provides:** Manages subscription plans, enabled features, and limits; Enforces feature-level access control; Intercepts or checks requests to ensure authorized access.
- **Services it accepts:** Requests from controllers in other subsystems to verify feature availability and plan limits; Configuration inputs from admins.

5. Reporting and Dashboard Subsystem

- **Services it provides:** Aggregated views and statistics; Dashboards for admins and staff to monitor performance and system usage.
- **Services it accepts:** Data from User, Program, Application, and Subscription subsystems; Filter parameters and report requests from the web frontend.

Coupling and Cohesion

High cohesion within subsystems: Each subsystem focuses on a specific business area (users, programs, applications, subscriptions, reporting), and its controllers, services, and models mainly serve that purpose.

Controlled coupling between subsystems: Subsystems interact via service calls within the backend (for example, Application Management calls Program Management to verify program status, or Subscription/Feature Access to verify eligibility). The web frontend is coupled to the backend's controllers, but the backend layers (services, models, data access) remain internally modular and cohesive.

3.1.1 Component modeling

The current Incubation and Innovation Management System (IIMS) follows a **modular, component-based architecture** built on top of a **Java Spring MVC backend** and a **React-based web frontend**.

Each component groups related controllers, services, entities/models, and React UI modules around a specific business domain (users, tenants, applications, subscriptions, progress tracking, etc.). Components interact primarily through **REST endpoints** exposed by Spring MVC controllers, which are consumed by React service modules and pages using **fetch/axios**.

1. User & Tenant Management Component

- **Current Architecture**
    - **Backend (MVC):** AuthController, UserController, TenantController, permission/role controllers and services.
    - **Frontend (React):** Authentication and profile pages, tenant admin dashboards, and related API service modules.
- **Key Classes & Responsibilities**
    - **User, Role, Permission:** Represent user accounts, roles (SuperAdmin, TenatAdmin, startup,

mentor, etc.), and access rights.
    - **Tenant:** Represents an incubator/innovation hub instance and its configuration.
    - **Auth and user services:** Handle login, registration, password reset, profile updates, and role/tenant assignment.
- **Current Functions**
    - User registration, login, logout and password reset.
    - Role-based access control (RBAC) and tenant scoping (users belong to specific tenants).
    - Basic tenant setup and administration.
- **Communication & Dependencies**
    - **Frontend → Backend:** React pages call /api/auth/..., /api/users/..., /api/tenants/... using fetch/axios.
    - **Other Backend Components** depend on this component to authenticate requests and resolve current user and tenant, and check permissions before allowing operations.

2. Application & Program Management Component

- **Current Architecture**
    - **Backend (MVC):** ApplicationFormController, ApplicationController, PublicApplicationFormController, CohortController, related services and entities.
    - **Frontend (React):** Startup application pages, program/cohort management screens, and form-related service modules.
- **Key Classes & Responsibilities**
    - **ApplicationForm, Application, Cohort, Program:** Represent configurable forms, submitted applications, cohorts, and program structures.
    - **Services for application lifecycle:** Submission, update, status changes, and cohort assignment.
- **Current Functions**
    - Define and manage application forms per program/tenant.
    - Allow startups to submit, edit, and track applications.
    - Support admin and staff workflows: reviewing, shortlisting, and assigning accepted startups to cohorts.
- **Communication & Dependencies**
    - **Frontend → Backend:** Startup users fill application forms sent via REST calls (e.g.,

/api/applications/...) without full page reloads. Admin/staff UI uses APIs to manage forms and cohorts.
    - **Depends on:** User & Tenant Management (for applicant identity and permissions).

3. Progress Tracking & Analytics Component

- **Current Architecture**
    - **Backend (MVC):** ProgressTemplateController, ProgressPhaseController, ProgressTemplateAssignmentController, ProgressSubmissionFileController, AnalyticsController.
    - **Frontend (React):** Progress tracking pages and service modules using axios.
- **Key Classes & Responsibilities**
    - **ProgressTemplate, ProgressPhase, Task, ProgressSubmission, ProgressSubmissionFile:** Model milestones, tasks, submissions, and attachments.
    - **Analytics services:** Aggregate data for dashboards (e.g., completion rates, active tasks).
- **Current Functions**
    - Define templates and phases for tracking startup progress.
    - Assign templates/tasks to startups or cohorts.
    - Allow startups/mentors/staff to submit updates and upload files.
    - Generate analytics data for dashboards.
- **Communication & Dependencies**
    - **Frontend → Backend:** Uses axios calls like /api/progresstracking/templates to load and update data asynchronously.
    - **Depends on:** User & Tenant Management and Application & Program Management.

4. Mentorship, Meetings & Communication Component

- **Current Architecture**
    - **Backend (MVC):** StartupMentorAssignmentController, MeetingController, MeetingSummaryController, ChatRoomController, NotificationController, GoogleOAuthController.
    - **Frontend (React):** Mentor dashboards, assignment screens, meeting scheduling UI, and chat/notification UIs.

- **Key Classes & Responsibilities**
  - **Mentor, StartupMentorAssignment:** Map mentors to startups and cohorts.
  - **Meeting, MeetingSummary:** Capture scheduled/held meetings and notes.
  - **Notification, ChatRoom, Message:** Handle in-app notifications and communication.
- **Current Functions**
  - Assign mentors to startups and manage relationships.
  - Schedule and manage meetings (integrated with Google Calendar via OAuth).
  - Provide chat rooms and notifications for real-time communication.
- **Communication & Dependencies**
  - **Frontend → Backend:** React mentor/admin UIs call APIs like /api/mentor-assignments/... and /api/meetings/....
  - **Depends on:** User & Tenant Management and Application & Program Management.

5. Subscription & Feature Access Component

- **Current Architecture**
  - **Backend (MVC):** SubscriptionPlanController, SubscriptionManagementController, PaymentController, FeatureController, FeatureAccessController, and FeatureAccessInterceptor.
  - **Frontend (React):** Tenant admin subscription management pages and feature toggles in various UIs.
- **Key Classes & Responsibilities**
  - **SubscriptionPlan, Subscription, Feature, FeatureAccessRule:** Represent plans, active subscriptions, and rules/limits.
  - **FeatureAccessInterceptor:** Intercepts backend requests to enforce plan/feature access rules.
- **Current Functions**
  - Define and manage subscription plans and features per tenant.
  - Enforce module availability based on the tenant's plan.
  - Support payment handling and subscription lifecycle.
- **Communication & Dependencies**
  - **Frontend → Backend:** Tenant admins manage plans via REST endpoints (e.g., /api/subscriptions/...).
  - **Interactions:** Other backend components rely on the FeatureAccessInterceptor to check

feature availability before controller methods execute.

6. Resources, News & Help Center Component

- **Current Architecture**
  - **Backend (MVC):** HelpCenterController, ContactController.
  - **Frontend (React):** Public landing page, news listings, and help center pages.
- **Key Classes & Responsibilities**
  - **NewsPost, NewsFile:** Represent tenant-specific news and related media.
  - **Recommendation and Help Center services:** Provide curated resources and support information.
- **Current Functions**
  - Manage tenant-specific news, announcements, and learning resources.
  - Display curated content on public and internal landing pages.
  - Provide FAQs/help content and contact forms.
- **Communication & Dependencies**
  - **Frontend → Backend:** PublicLandingPage calls /news/tenant/{tenantId} via fetch to load news dynamically.
  - **Depends on:** Tenant Management for scoping content per tenant.

Relationships, Dependencies, and Communication Summary

- All React UI components communicate with the Spring MVC backend via **REST endpoints**, implemented using **fetch** and **axios** (AJAX style).
- **Backend components are loosely coupled:** They interact through service calls and shared domain entities, with clearly defined controller endpoints per component.
- **Cross-cutting concerns** like authentication, authorization, and feature access are handled by dedicated components (Auth, Subscription/FeatureAccessInterceptor).

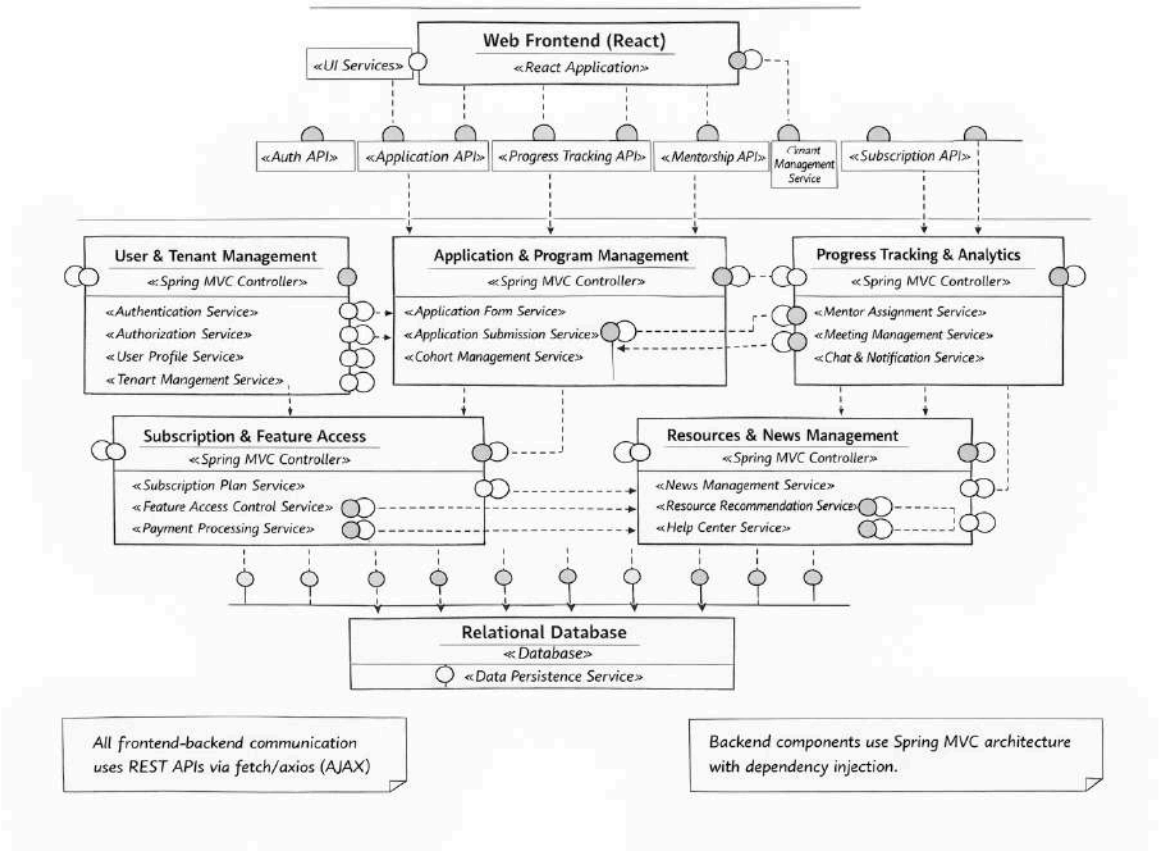


Figure 3.1.2 Component modeling

3.1.2 Deployment Modeling

The IIMS system uses a containerized deployment with **Docker Compose**. The backend (Spring Boot MVC) and database (PostgreSQL) run in containers, while the React frontend is served via a web server or development server. Components communicate over HTTP/REST APIs.

Deployment Diagram

1. Client Devices (Web Browsers)

- **Location:** End-user devices (desktops, laptops, tablets, smartphones)
- **Components Deployed:** Web browsers (Chrome, Firefox, Safari, Edge)
- **Function:** Users access the IIMS web application through their browsers
- **Communication:** HTTP/HTTPS requests to the web server hosting the React frontend

2. Web Server / Frontend Hosting

- **Location:** Web server (Vercel and Render server)
- **Components Deployed:** Static React application files (HTML, CSS, JavaScript bundles)
    - React build artifacts served as static files
- **Port:** Typically port 80 (HTTP) or 443 (HTTPS) for production
- **Function:** Serves the React frontend application to client browsers
- **Communication:** Receives requests from client browsers; Makes AJAX calls (via fetch/axios) to the backend API server

3. Backend Application Server

- **Location:** Docker container (or physical/virtual server)
- **Components Deployed:** Spring Boot MVC application (packaged as JAR: iims-0.0.1-SNAPSHOT.jar)
- **All backend components:** User & Tenant Management, Application & Program Management, Progress Tracking & Analytics, Mentorship & Communication, Subscription & Feature Access, Resources & News Management
- **Port:** 8081 (configurable via PORT environment variable)
- **Runtime:** Java 21 (Eclipse Temurin JRE)
- **Function:** Handles all business logic, REST API endpoints, authentication, authorization, and data processing
- **Communication:** Receives HTTP requests from the frontend; Connects to PostgreSQL via JDBC; Connects to external services (SMTP, Google OAuth)

4. Database Server

- **Location:** Docker container (PostgreSQL 15)
- **Components Deployed:** PostgreSQL database server; Database instance: iims
- **Port:** 5432 (internally), mapped to 5332 on host machine
- **Function:** Stores all persistent data: User accounts, Tenant data, Application forms/submissions, Progress tracking, Mentor records, Subscription plans, and Chat/News content.
- **Storage:** Persistent volume (pgdata) for data durability
- **Communication:** Receives SQL queries from the backend via JDBC; Returns query results to the

backend

5. File Storage

- **Location:** Local filesystem within the backend container or external storage (cloud/NAS)
- **Components Deployed:** Upload directory (/app/uploads)
- **Stored Assets:** Application documents (PDFs), Progress submission files, News images, Landing page images, and Profile pictures.
- **Function:** Persistent storage for user-uploaded files
- **Communication:** Backend reads/writes files directly; Frontend accesses files via backend API endpoints (e.g., /api/news/image/{id})

Deployment Architecture Details

Current Deployment Model:

- **Monolithic Backend:** All backend components are packaged together in a single Spring Boot JAR.
- **Containerized Infrastructure:** Backend and database run in Docker containers orchestrated via Docker Compose.
- **Single Database:** All backend components share one PostgreSQL database instance.
- **RESTful Communication:** Frontend and backend communicate via REST APIs over HTTP/HTTPS.

Deployment Configuration:

- **Backend Container:** Built from a multi-stage Dockerfile (Maven build + JRE runtime). Depends on the database container for startup.
- **Database Container:** Uses PostgreSQL 15 official image with a persistent volume and specific environment credentials.
- **Frontend:** Built as static files (via npm run build) and configured to make API calls to the backend server URL.

Physical Communication Links:

- **Client Browser ↔ Web Server:** HTTP/HTTPS (port 80/443) - Accessing the React app.
- **Web Server ↔ Backend Server:** HTTP/HTTPS (port 8081) - Frontend AJAX requests.

- **Backend Server ↔ Database Server:** JDBC/PostgreSQL protocol (port 5432) - SQL execution.
- **Backend Server ↔ File Storage:** Filesystem I/O - Reading/writing uploaded files.

The current monolithic deployment is suitable for small to medium-scale operations and simplifies development, deployment, and maintenance.

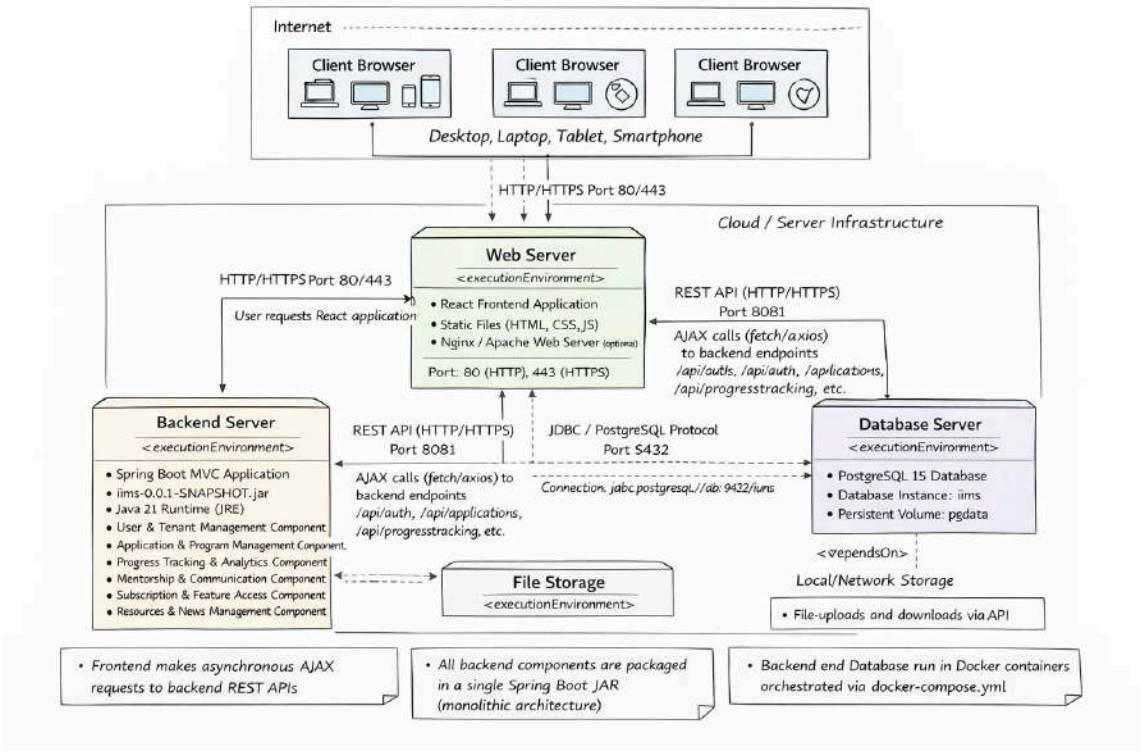


Figure 3.1.3 Deployment Model

3.2 Detail Design

3.2.1. Design class model

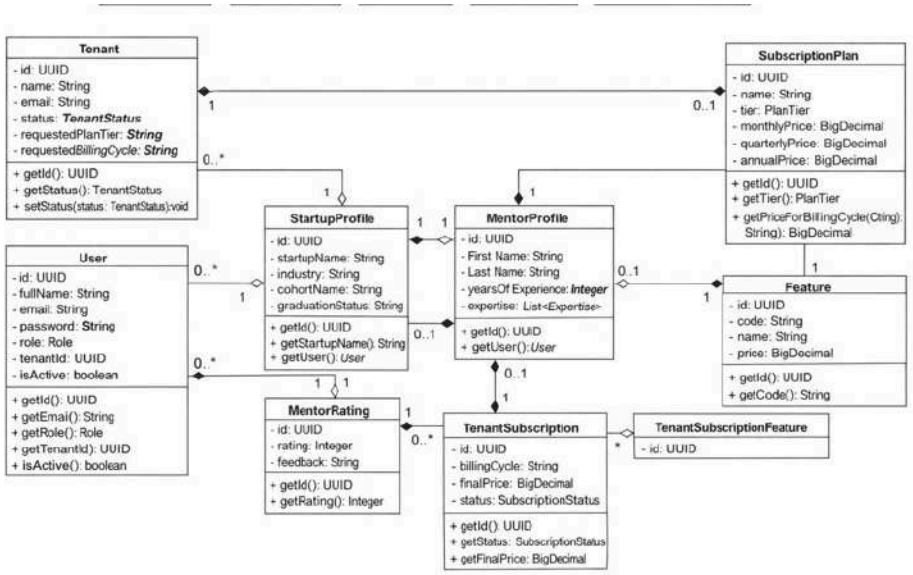


Figure 3.2.1 Design class model

3.2.1. Persistent model

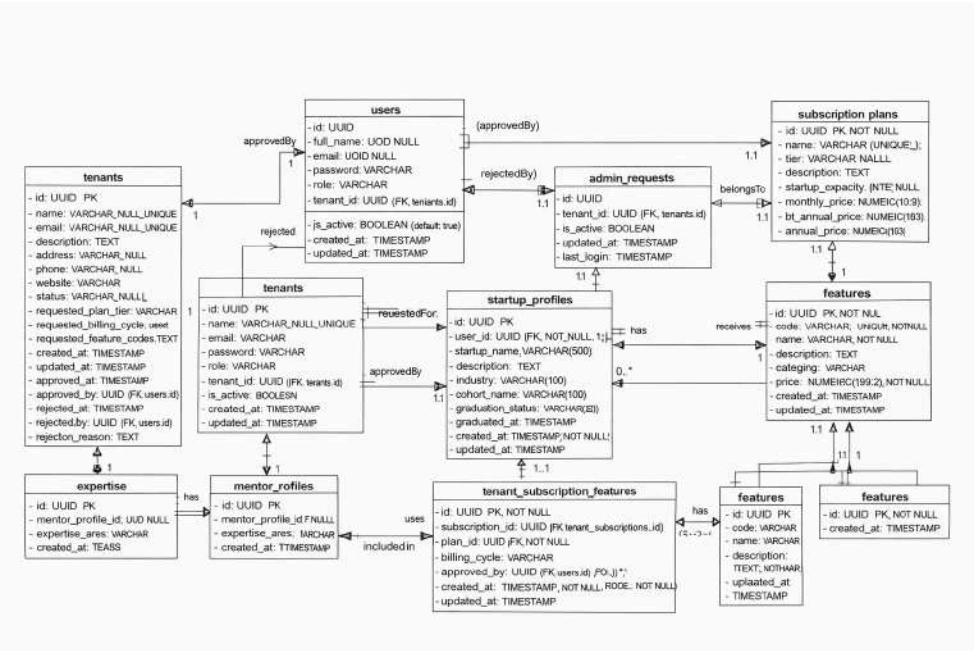


Figure 3.2.2 Persistent model

3.3 User Interface Design

Super Admin Dashboard

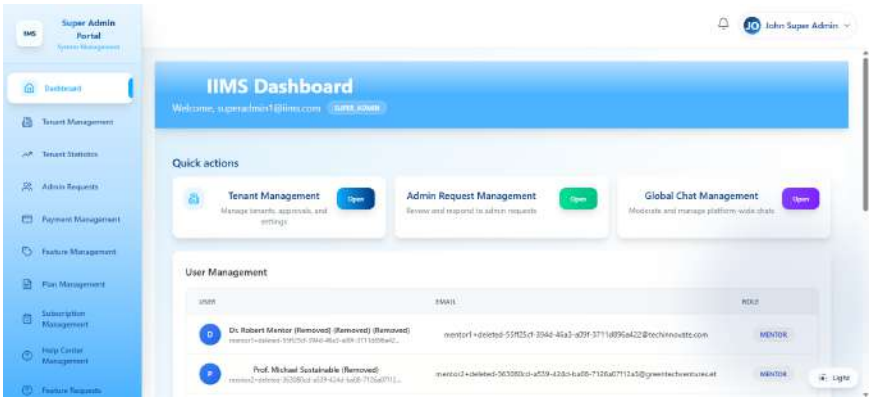


Figure 3.3.1 Super Admin Dashboard

Tenant Statistics page

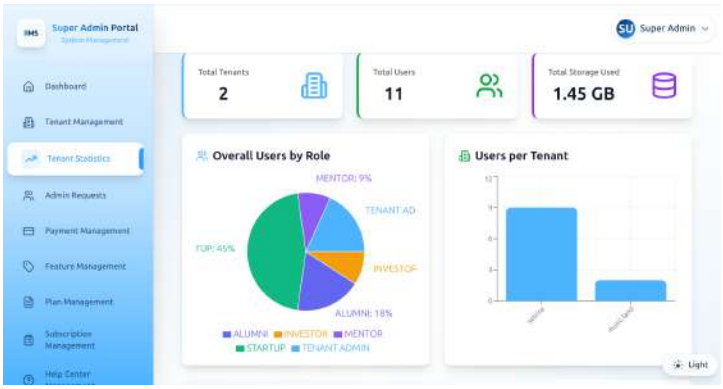


Figure 3.3.2 Tenant Statistics page

Application form

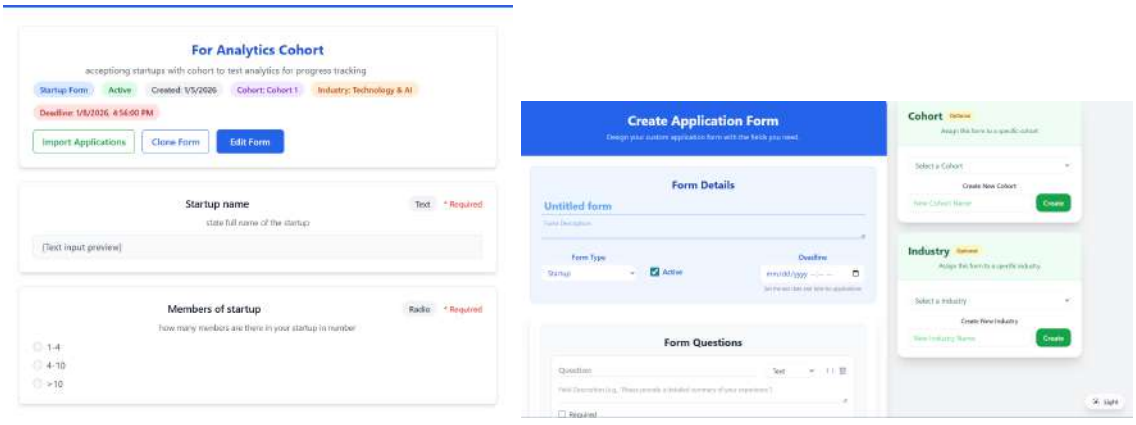


Figure 3.3.3 Application form

Applications

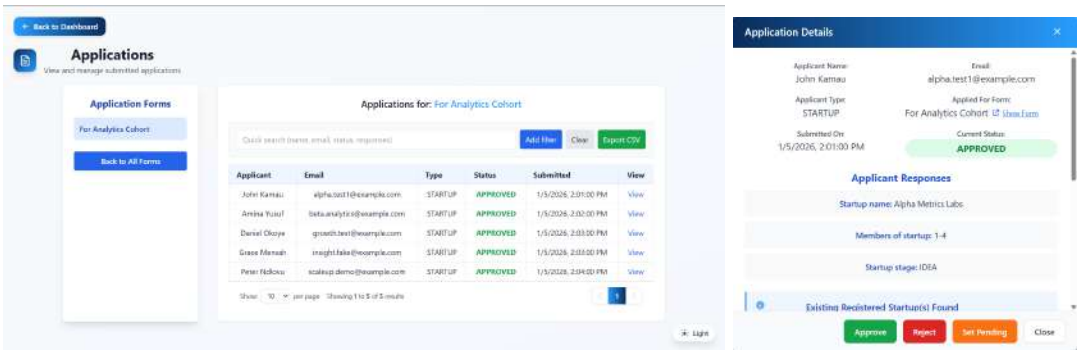


Figure 3.3.4 Applications

Subscription & Feature Management

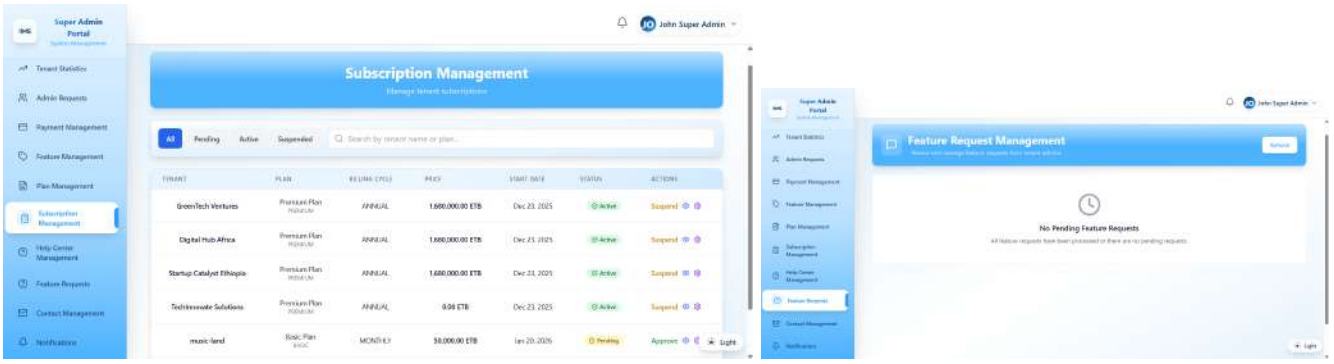


Figure 3.3.5 Subscription & Feature Management

Admin request management

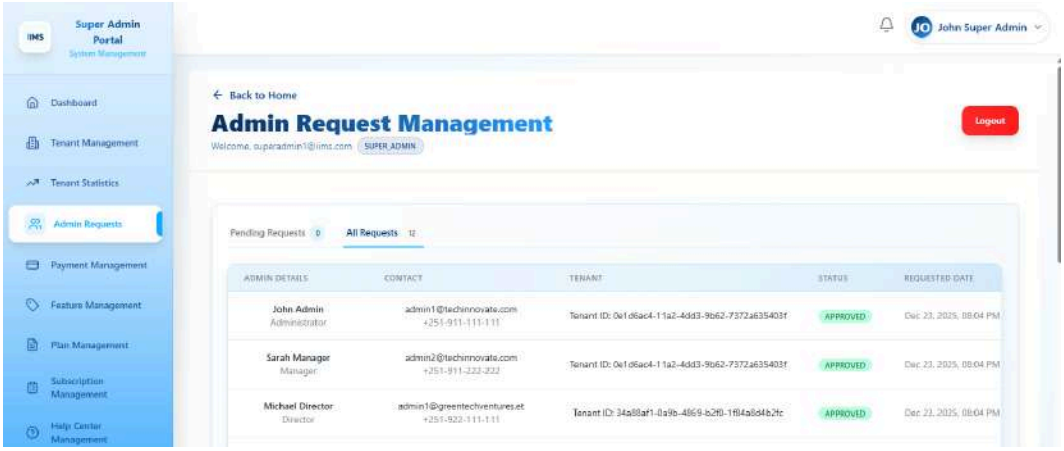


Figure 3.3.6 Admin request management

Tenant Management Page

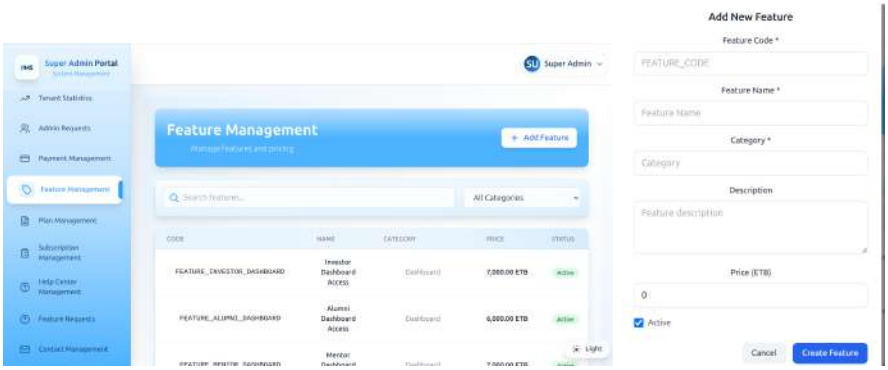


Figure 3.3.7 Tenant Management Page

Tenant Admin Dashboard

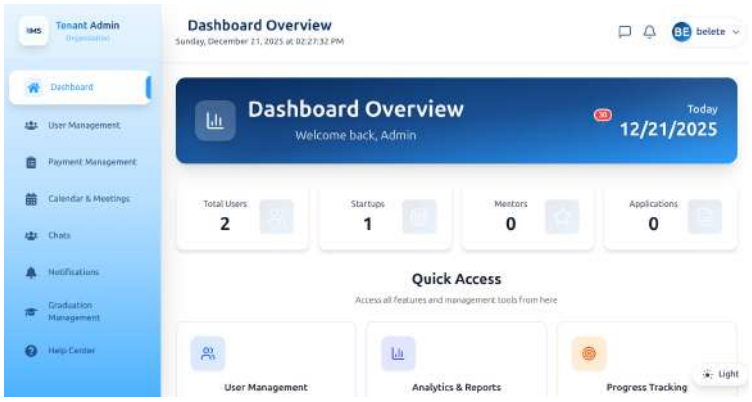


Figure 3.3.8 Tenant Admin Dashboard

Payment Management Page

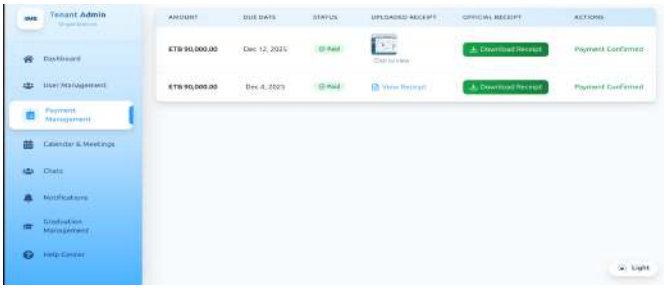


Figure 3.3.9 Payment Management Page

User Management Page

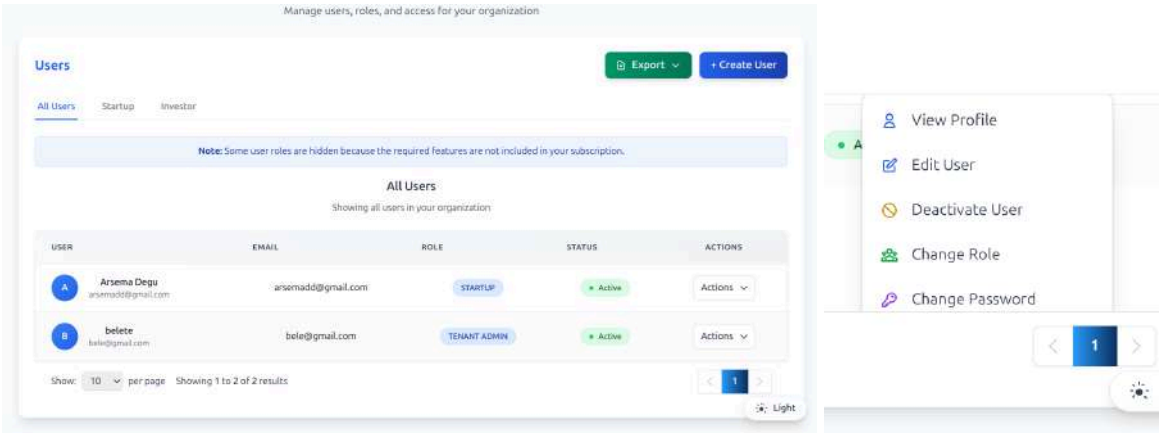


Figure 3.3.10 User Management Page

Feature Request Management Page

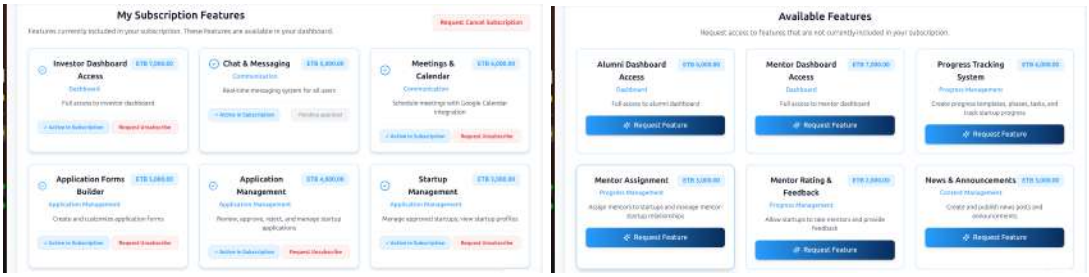


Figure 3.3.11 Feature Request Management Page

Contact management

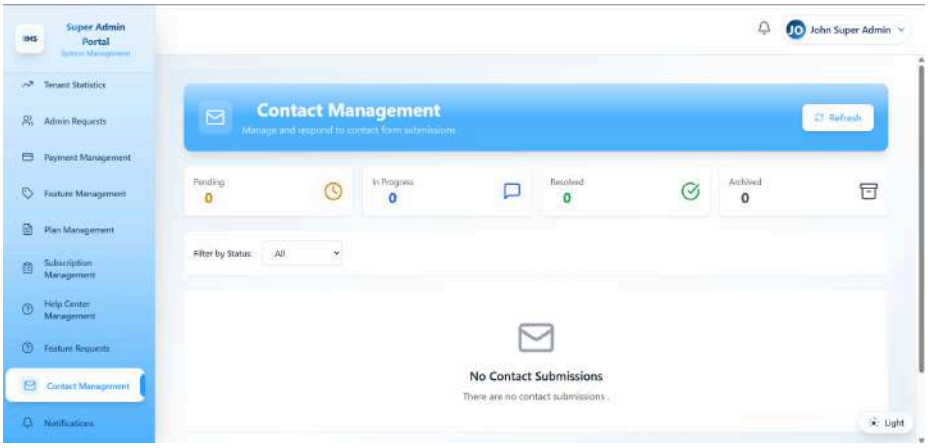


Figure 3.3.12 Contact management

Notification management

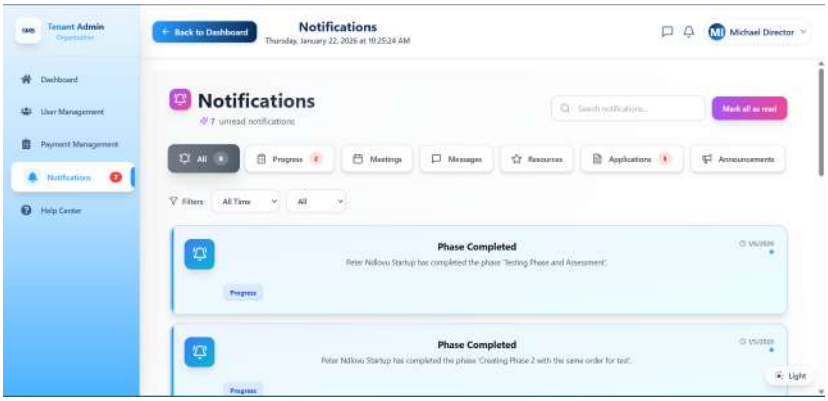


Figure 3.3.13 Notification management

Progress Tracking Management Page

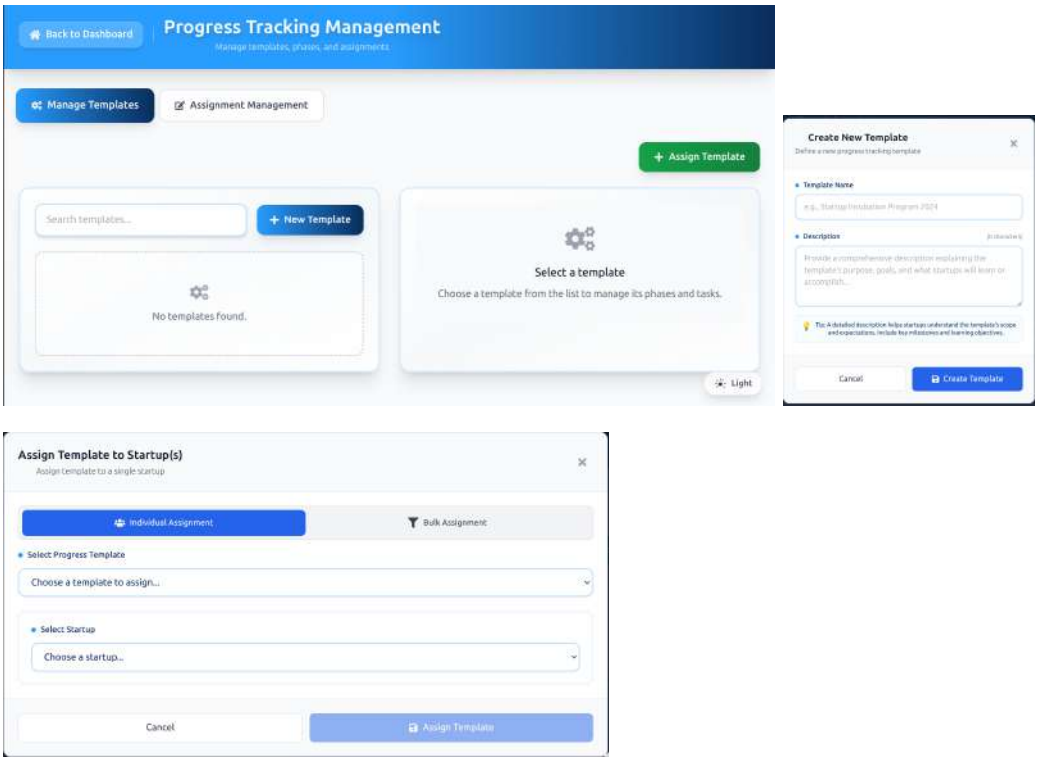


Figure 3.3.14 Progress Tracking Management Page

Mentor assignment management

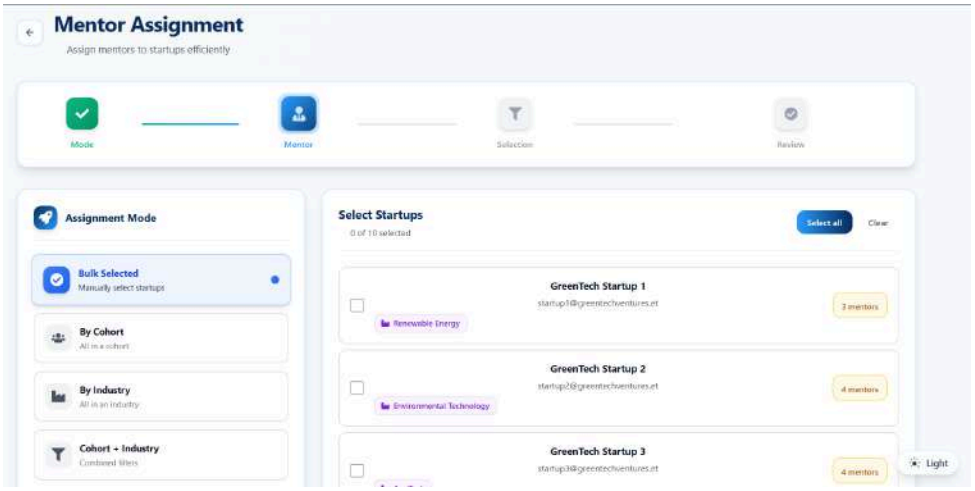


Figure 3.3.15 Mentor assignment management

Startup Dashboard

Figure 3.3.16 Mentor assignment management

Mentor Dashboard

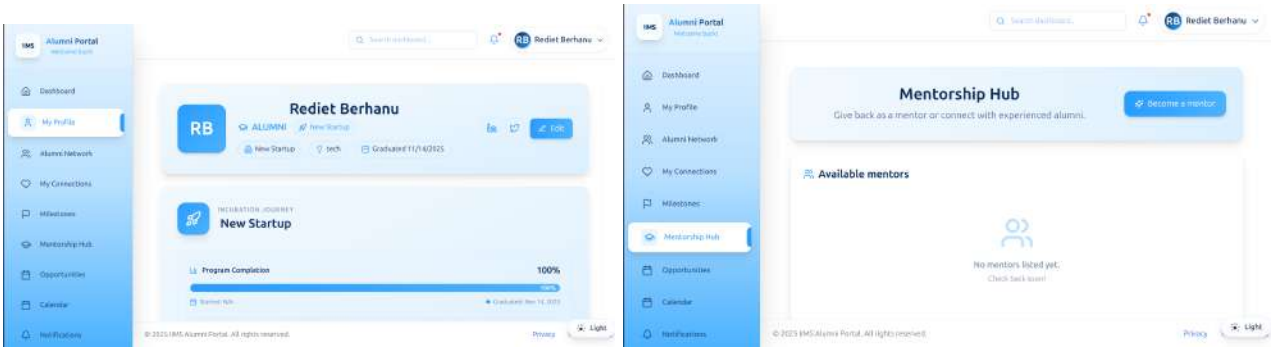


Figure 3.3.17 Mentor Dashboard

Chat

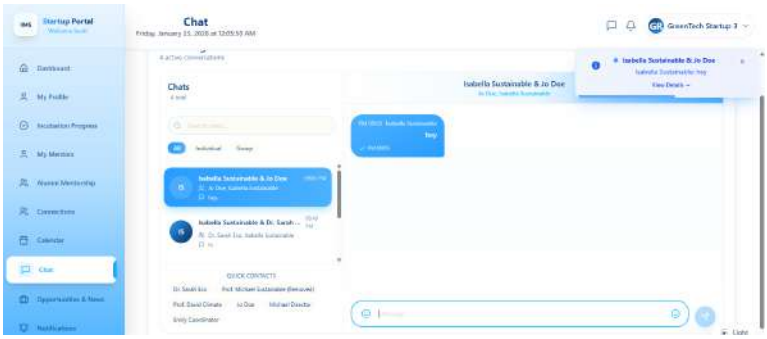


Figure 3.3.17 Chat

Calendar & Meeting

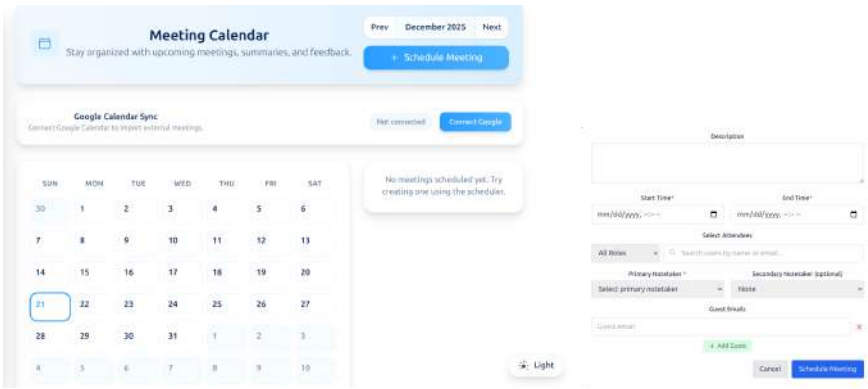


Figure 3.3.17 Calendar & meeting

3.4 Access Control and Security

In the Innovation and Incubation Management System (IIMS), multiple users interact with the platform, including **administrators, startups, and mentors**. Each user type has different responsibilities and access rights. Ensuring secure access to data and system functions is a critical part of the system design. IIMS implements strong access control and security mechanisms to protect sensitive information, prevent unauthorized access, and maintain data integrity.

3.4.1. Authentication

Authentication ensures that only valid users can access the system.

- **User Login:**

  All users (administrators, startups, and mentors) must log in using a registered email and password.
- **JWT (JSON Web Token) Authentication:**

  After successful login, the system generates a JWT token that contains the user's identity and role. This token is used to authenticate all future requests to the system.
- **Login Flow:**
  - The user enters their login credentials.
  - The system verifies the credentials.
  - If valid, a JWT token is generated and returned.
  - The token is included in the Authorization header for all protected API requests.
- **Token Expiration:**

  JWT tokens have a limited validity period. Once expired, users must log in again to continue using the system. This reduces the risk of unauthorized access if a token is compromised.

● **Token Validation:**

Every protected request is checked by the backend to ensure the token is valid before granting access.

3.4.2. Authorization

Authorization controls **what actions a user is allowed to perform** after authentication.

- **User Roles:**

  IIMS defines the following roles:
  - **SuperAdmin:** Full system access, including managing Tenant, reviewing Tenant applications, and monitoring feature and subscription management.
  - **TenantAdmin:** Includes managing users, reviewing applications, assigning mentors, and monitoring startup progress.
  - **Startup:** Can submit applications, update profile information, track progress, submit milestones, and communicate with assigned mentors.
  - **Mentor:** Can view assigned startups, provide feedback, schedule mentoring sessions, and update mentorship records.

- **Role-Based Access Control (RBAC):**

  The system uses RBAC to restrict access to features based on user roles.
  - SuperAdmins can access all administrative and management features.

- TenantAdmins can access all features included in the subscription for specific Incubation Center.
- Startups cannot access administrative tools or other startups' data.
- Mentors can only access information related to their assigned startups.

- **Role Assignment:**

  Roles are assigned during user registration or by administrators. Role information is stored in the database and embedded in the JWT token. The system checks this role before allowing access to specific routes or actions.

3.4.3. Data Validation

Data validation protects the system from incorrect or malicious input.

- **Input Validation:**

  All user inputs, such as application forms, mentor feedback, milestone updates, and profile information, are validated before processing.
  - Required fields must be filled.
  - Email formats must be valid.
  - Passwords must meet minimum strength requirements.
- **Server-Side Validation:**

  Validation is enforced on the backend using trusted validation libraries to ensure data integrity and prevent bypassing client-side checks.
- **Protection Against Injection Attacks:**

  The system uses parameterized queries and ORM tools to prevent SQL injection and other database-related attacks.

3.4.4. Encryption

Encryption ensures that sensitive data remains confidential.

- **Password Hashing:**

  User passwords are never stored in plain text. The system uses **bcrypt** to hash passwords before storing them in the database.
  - During login, the entered password is hashed and compared with the stored hash.
  - Even if the database is compromised, passwords cannot be easily recovered.
- **Data Encryption:**

Sensitive data, such as confidential startup information and reports, is encrypted before storage where necessary.

- **Secure Data Transmission:**

  The system uses **TLS (Transport Layer Security)** to encrypt all data transmitted between the frontend and backend, protecting user information from interception.

3.4.5. Database Access Control

- **Controlled Database Access:**

  Database access is restricted based on system roles and services. Only authorized backend services can read or write sensitive data.

- **ORM Usage:**

  IIMS uses an Object-Relational Mapping (ORM) tool to manage database interactions safely and consistently, reducing direct exposure to raw SQL queries.

- **Sensitive Data Handling:**

  Tables such as **Users, Applications, and Mentorship Records** contain sensitive information and are accessible only to authorized users.

Chapter Four: Implementation

4.1. User Entity

The User entity class demonstrates object-oriented programming through interface implementation. This class implements Spring Security's UserDetails interface, showcasing inheritance and polymorphism by overriding interface methods. It uses JPA annotations for database mapping and includes role-based access control for multi-tenant architecture.

```java
@Entity
@Table(name = "users")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class User implements UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    private String fullName;
    private String email;
    private String password;

    @Enumerated(EnumType.STRING)
    private Role role;

    private UUID tenantId;

    @Builder.Default
    private boolean isActive = true;

    @CreationTimestamp
    @Column(updatable = false)
    private LocalDateTime createdAt;

    @UpdateTimestamp
    private LocalDateTime updatedAt;

    private LocalDateTime lastLogin;

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return List.of(new SimpleGrantedAuthority("ROLE_" + role.name()));
    }

    @Override
    public String getUsername() {
        return email;
    }

    @Override
    public String getPassword() {
        return password;
    }

    @Override
    public boolean isAccountNonExpired() {
        return true;
    }

    @Override
    public boolean isAccountNonLocked() {
        return true;
    }

    @Override
    public boolean isCredentialsNonExpired() {
        return true;
    }

    @Override
    public boolean isEnabled() {
        return isActive;
    }
}
```

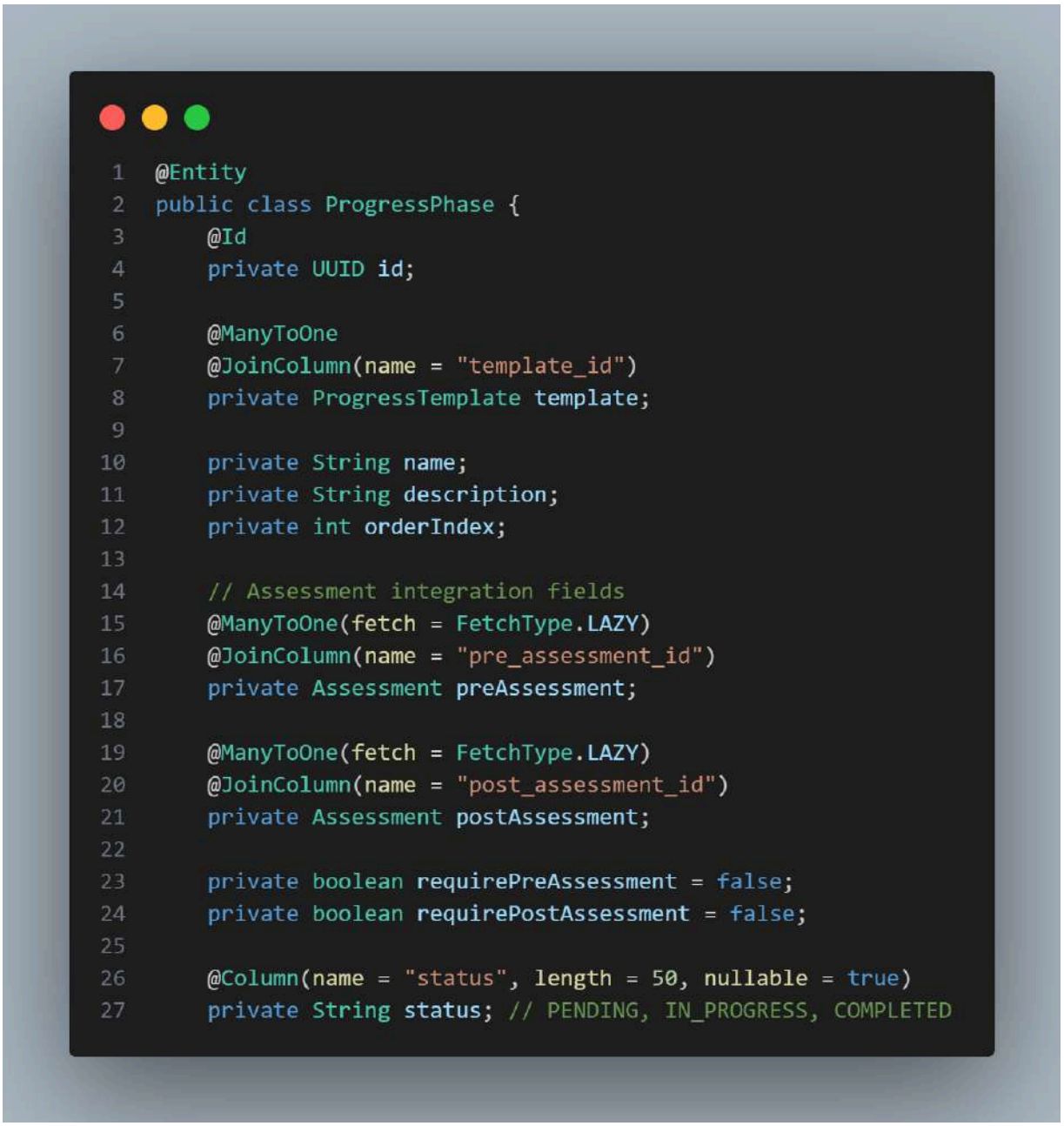
4.2. Application Entity

The Application entity class demonstrates complex object relationships using JPA annotations. This class shows ManyToOne relationships with ApplicationForm and Tenant, and OneToMany relationships with ApplicationResponse and ApplicationDocument, illustrating composition in object-oriented design. The use of cascade operations and lazy loading demonstrates efficient data access patterns.

```java
@Entity
@Table(name = "applications")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Application {

    @Id
    @GeneratedValue(generator = "UUID")
    @GenericGenerator(name = "UUID", strategy = "org.hibernate.id.UUIDGenerator")
    @Column (name = "id", updatable = false, nullable = false)
    private UUID id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "form_id", nullable = false)
    private ApplicationForm form;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "tenant_id", nullable = false)
    private Tenant tenant;

    @Column(name = "email", nullable = false)
    private String email;

    @Column(name = "firat_name")
    private String first_name;

    @Column(name = "last_name")
    private String last_name;

    @Enumerated(EnumType.STRING)
    @Column(name = "applicant_type", nullable = false)
    private ApplicantType applicantType;

    @Enumerated(EnumType.STRING)
    @Column(name = "status", nullable = false)
    private ApplicationStatus status = ApplicationStatus.PENDING;

    @Column(name = "submitted_at", updatable = false, nullable = false)
    private LocalDateTime submittedAt = LocalDateTime.now();


    @Column(name = "reviewed_at")
    private LocalDateTime reviewedAt;

    @Column(name = "reviewed_by")
    private UUID reviewedBy;

    @Column(name = "rejection_reason")
    private String rejectionReason;

    @Column(name = "rejection_category")
    private String rejectionCategory;

    @Column(name = "cohort_id")
    private UUID cohortId;

    @OneToMany(mappedBy = "application", cascade = CascadeType.ALL, orphanRemoval = true, fetch = FetchType.LAZY)
    private List<ApplicationResponse> responses;

    // New relationship for documents
    @OneToMany(mappedBy = "application", cascade = CascadeType.ALL, orphanRemoval = true, fetch = FetchType.LAZY)
    private List<ApplicationDocument> documents;
```

4.3. ProgressPhase Entity

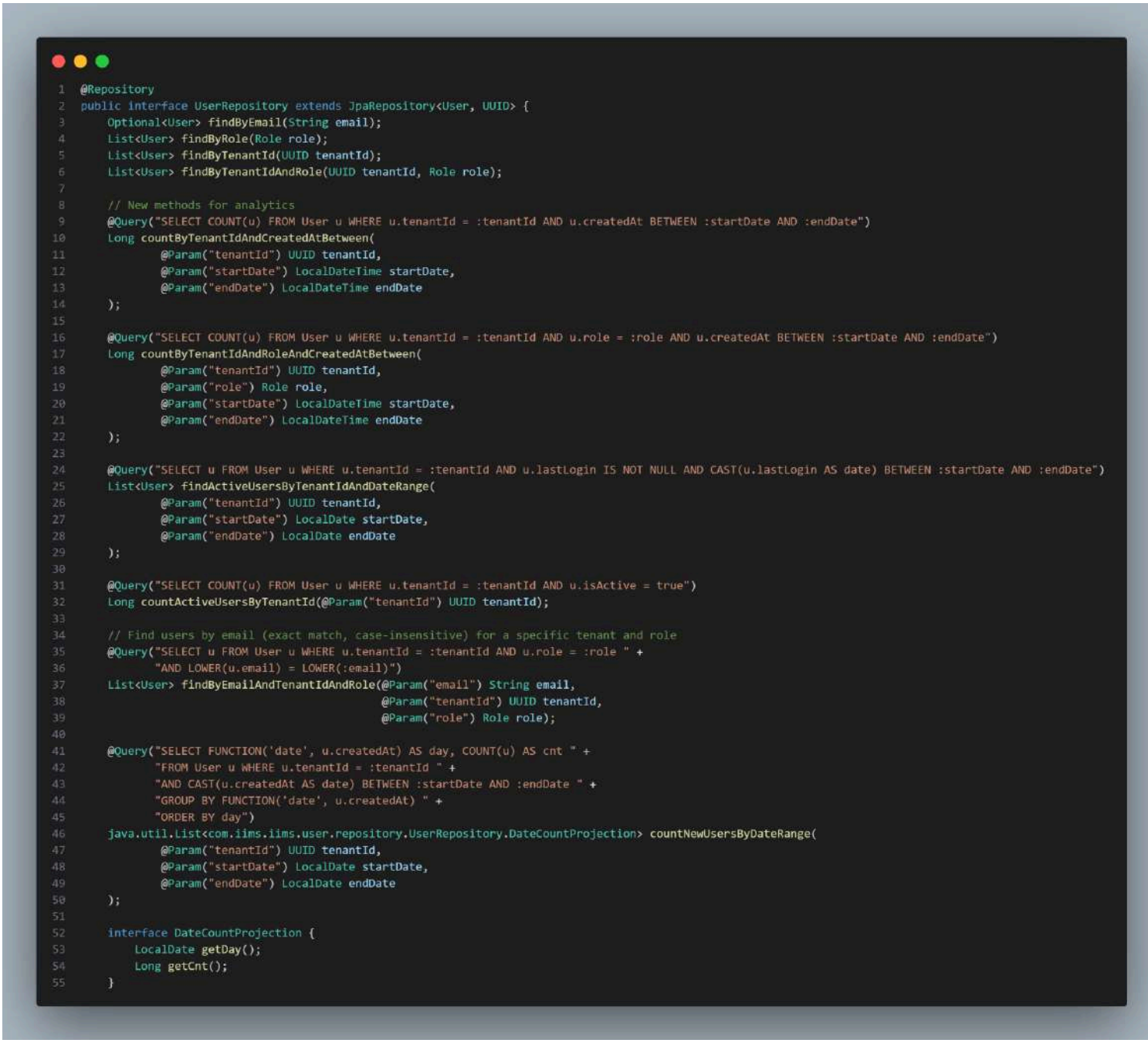
The ProgressPhase entity demonstrates object-oriented design with clear separation of concerns. This entity shows ManyToOne relationships with ProgressTemplate and Assessment entities, illustrating object collaboration. The class includes business logic fields like orderIndex for sequencing, demonstrating encapsulation of related data and behavior within a single class.



```java
@Entity
public class ProgressPhase {
    @Id
    private UUID id;

    @ManyToOne
    @JoinColumn(name = "template_id")
    private ProgressTemplate template;

    private String name;
    private String description;
    private int orderIndex;

    // Assessment integration fields
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "pre_assessment_id")
    private Assessment preAssessment;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "post_assessment_id")
    private Assessment postAssessment;

    private boolean requirePreAssessment = false;
    private boolean requirePostAssessment = false;

    @Column(name = "status", length = 50, nullable = true)
    private String status; // PENDING, IN_PROGRESS, COMPLETED
```

4.4. UserRepository Interface

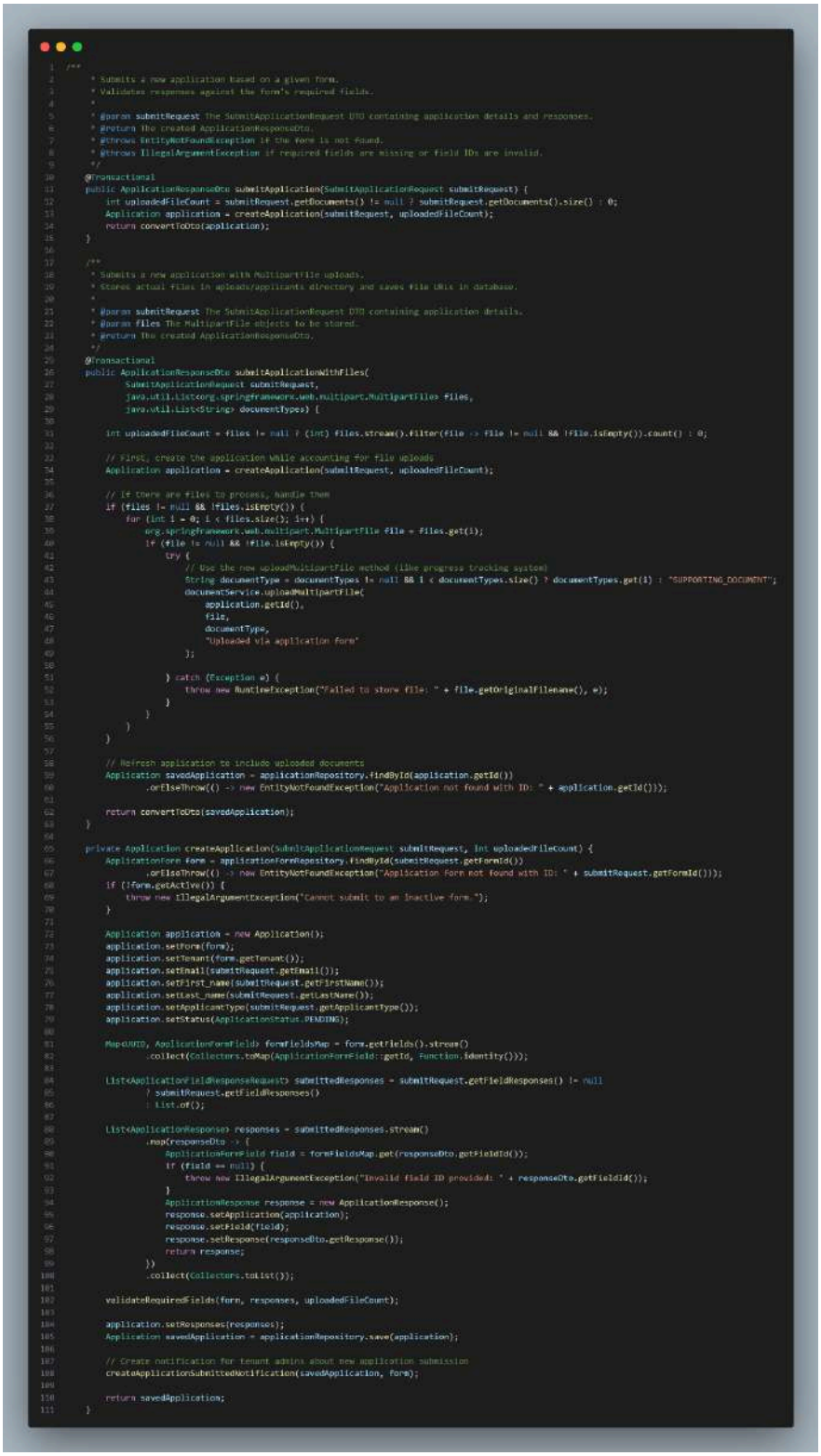
The UserRepository interface demonstrates the Repository design pattern for data access abstraction. This interface extends Spring Data JPA's JpaRepository, showcasing inheritance and interface-based programming. The custom query methods with @Query annotations demonstrate polymorphism, and the nested projection interface illustrates inner classes in object-oriented design.



```java
@Repository
public interface UserRepository extends JpaRepository<User, UUID> {
    Optional<User> findByEmail(String email);
    List<User> findByRole(Role role);
    List<User> findByTenantId(UUID tenantId);
    List<User> findByTenantIdAndRole(UUID tenantId, Role role);

    // New methods for analytics
    @Query("SELECT COUNT(u) FROM User u WHERE u.tenantId = :tenantId AND u.createdAt BETWEEN :startDate AND :endDate")
    Long countByTenantIdAndCreatedAtBetween(
            @Param("tenantId") UUID tenantId,
            @Param("startDate") LocalDateTime startDate,
            @Param("endDate") LocalDateTime endDate
    );

    @Query("SELECT COUNT(u) FROM User u WHERE u.tenantId = :tenantId AND u.role = :role AND u.createdAt BETWEEN :startDate AND :endDate")
    Long countByTenantIdAndRoleAndCreatedAtBetween(
            @Param("tenantId") UUID tenantId,
            @Param("role") Role role,
            @Param("startDate") LocalDateTime startDate,
            @Param("endDate") LocalDateTime endDate
    );

    @Query("SELECT u FROM User u WHERE u.tenantId = :tenantId AND u.lastLogin IS NOT NULL AND CAST(u.lastLogin AS date) BETWEEN :startDate AND :endDate")
    List<User> findActiveUsersByTenantIdAndDateRange(
            @Param("tenantId") UUID tenantId,
            @Param("startDate") LocalDate startDate,
            @Param("endDate") LocalDate endDate
    );

    @Query("SELECT COUNT(u) FROM User u WHERE u.tenantId = :tenantId AND u.isActive = true")
    Long countActiveUsersByTenantId(@Param("tenantId") UUID tenantId);

    // Find users by email (exact match, case-insensitive) for a specific tenant and role
    @Query("SELECT u FROM User u WHERE u.tenantId = :tenantId AND u.role = :role " +
           "AND LOWER(u.email) = LOWER(:email)")
    List<User> findByEmailAndTenantIdAndRole(@Param("email") String email,
                                             @Param("tenantId") UUID tenantId,
                                             @Param("role") Role role);

    @Query("SELECT FUNCTION('date', u.createdAt) AS day, COUNT(u) AS cnt " +
           "FROM User u WHERE u.tenantId = :tenantId " +
           "AND CAST(u.createdAt AS date) BETWEEN :startDate AND :endDate " +
           "GROUP BY FUNCTION('date', u.createdAt) " +
           "ORDER BY day")
    java.util.List<com.iims.iims.user.repository.UserRepository.DateCountProjection> countNewUsersByDateRange(
            @Param("tenantId") UUID tenantId,
            @Param("startDate") LocalDate startDate,
            @Param("endDate") LocalDate endDate
    );

    interface DateCountProjection {
        LocalDate getDay();
        Long getCnt();
    }
```

4.5. ApplicationService

The ApplicationService class implements the core business logic called by the ApplicationController's POST methods, demonstrating separation of concerns between controller and service layers.



```java
/**
     * Submits a new application based on a given form.
     * Validates responses against the form's required fields.
     *
     * @param submitRequest The SubmitApplicationRequest DTO containing application details and responses.
     * @return The created ApplicationResponseDto.
     * @throws EntityNotFoundException if the form is not found.
     * @throws IllegalArgumentException if required fields are missing or field IDs are invalid.
     */
    @Transactional
    public ApplicationResponseDto submitApplication(SubmitApplicationRequest submitRequest) {
        int uploadedFileCount = submitRequest.getDocuments() != null ? submitRequest.getDocuments().size() : 0;
        Application application = createApplication(submitRequest, uploadedFileCount);
        return convertToDto(application);
    }

    /**
     * Submits a new application with MultipartFile uploads.
     * Stores actual files in uploads/applicants directory and saves file URLs in database.
     *
     * @param submitRequest The SubmitApplicationRequest DTO containing application details.
     * @param files The MultipartFile objects to be stored.
     * @return The created ApplicationResponseDto.
     */
    @Transactional
    public ApplicationResponseDto submitApplicationWithFiles(
            SubmitApplicationRequest submitRequest,
            java.util.List<org.springframework.web.multipart.MultipartFile> files,
            java.util.List<String> documentTypes) {

        int uploadedFileCount = files != null ? (int) files.stream().filter(file -> file != null && !file.isEmpty()).count() : 0;

        // First, create the application while accounting for file uploads
        Application application = createApplication(submitRequest, uploadedFileCount);

        // If there are files to process, handle them
        if (files != null && !files.isEmpty()) {
            for (int i = 0; i < files.size(); i++) {
                org.springframework.web.multipart.MultipartFile file = files.get(i);
                if (file != null && !file.isEmpty()) {
                    try {
                        // Use the new uploadMultipartFile method (like progress tracking system)
                        String documentType = documentTypes != null && i < documentTypes.size() ? documentTypes.get(i) : "SUPPORTING_DOCUMENT";
                        documentService.uploadMultipartFile(
                            application.getId(),
                            file,
                            documentType,
                            "Uploaded via application form"
                        );

                    } catch (Exception e) {
                        throw new RuntimeException("Failed to store file: " + file.getOriginalFilename(), e);
                    }
                }
            }
        }

        // Refresh application to include uploaded documents
        Application savedApplication = applicationRepository.findById(application.getId())
                .orElseThrow(() -> new EntityNotFoundException("Application not found with ID: " + application.getId()));

        return convertToDto(savedApplication);
    }

    private Application createApplication(SubmitApplicationRequest submitRequest, int uploadedFileCount) {
        ApplicationForm form = applicationFormRepository.findById(submitRequest.getFormId())
                .orElseThrow(() -> new EntityNotFoundException("Application form not found with ID: " + submitRequest.getFormId()));
        if (!form.getActive()) {
            throw new IllegalArgumentException("Cannot submit to an inactive form.");
        }

        Application application = new Application();
        application.setForm(form);
        application.setTenant(form.getTenant());
        application.setEmail(submitRequest.getEmail());
        application.setFirst_name(submitRequest.getFirstName());
        application.setLast_name(submitRequest.getLastName());
        application.setApplicantType(submitRequest.getApplicantType());
        application.setStatus(ApplicationStatus.PENDING);

        Map<UUID, ApplicationFormField> formFieldsMap = form.getFields().stream()
                .collect(Collectors.toMap(ApplicationFormField::getId, Function.identity()));

        List<ApplicationFieldResponseRequest> submittedResponses = submitRequest.getFieldResponses() != null
                ? submitRequest.getFieldResponses()
                : List.of();

        List<ApplicationResponse> responses = submittedResponses.stream()
                .map(responseDto -> {
                    ApplicationFormField field = formFieldsMap.get(responseDto.getFieldId());
                    if (field == null) {
                        throw new IllegalArgumentException("Invalid field ID provided: " + responseDto.getFieldId());
                    }
                    ApplicationResponse response = new ApplicationResponse();
                    response.setApplication(application);
                    response.setField(field);
                    response.setResponse(responseDto.getResponse());
                    return response;
                })
                .collect(Collectors.toList());

        validateRequiredFields(form, responses, uploadedFileCount);

        application.setResponses(responses);
        Application savedApplication = applicationRepository.save(application);

        // Create notification for tenant admins about new application submission
        createApplicationSubmittedNotification(savedApplication, form);

        return savedApplication;
    }
```

ApplicationService - Part 2

The updateApplicationStatus method demonstrates complex business logic handled in a single transaction.

```java
@Transactional
public ApplicationResponseDto updateApplicationStatus(ApplicationReviewDto reviewDto, UUID tenantId) {
    Tenant tenant = tenantRepository.findById(tenantId)
            .orElseThrow(() -> new EntityNotFoundException("Tenant not found with ID: " + tenantId));

    Application application = applicationRepository.findByIdAndTenant(reviewDto.getApplicationId(), tenant)
            .orElseThrow(() -> new EntityNotFoundException("Application not found with ID: " + reviewDto.getApplicationId() + " for tenant: " + tenantId));

    application.setStatus(reviewDto.getNewStatus());
    application.setReviewedAt(java.time.LocalDateTime.now());

    // Set rejection details if status is REJECTED
    if (reviewDto.getNewStatus() == ApplicationStatus.REJECTED) {
        application.setRejectionCategory(reviewDto.getRejectionCategory());
        application.setRejectionReason(reviewDto.getRejectionReason());
    }

    Application updatedApplication = applicationRepository.save(application);

    // Send rejection email if status is REJECTED
    if (reviewDto.getNewStatus() == ApplicationStatus.REJECTED) {
        try {
            String applicantName = (updatedApplication.getFirst_name() != null ? updatedApplication.getFirst_name() : "") +
                    (updatedApplication.getLast_name() != null ? " " + updatedApplication.getLast_name() : "").trim();
            if (applicantName.isEmpty()) {
                applicantName = "Applicant";
            }
            String formName = updatedApplication.getForm() != null && updatedApplication.getForm().getName() != null
                ? updatedApplication.getForm().getName()
                : "Application";
            String tenantName = updatedApplication.getTenant() != null && updatedApplication.getTenant().getName() != null
                ? updatedApplication.getTenant().getName()
                : "Organization";

            emailService.sendApplicationRejectionEmail(
                updatedApplication.getEmail(),
                applicantName,
                formName,
                updatedApplication.getRejectionCategory(),
                updatedApplication.getRejectionReason(),
                tenantName
            );
        } catch (Exception e) {
            // Log error but don't fail rejection if email fails
            System.err.println("Warning: Application rejected but email could not be sent: " + e.getMessage());
        }
    }

    // On approval, create user and corresponding profile, then email credentials
    if (reviewDto.getNewStatus() == ApplicationStatus.APPROVED) {
        // Skip if a user already exists with this email
        if (userRepository.findByEmail(application.getEmail()).isEmpty()) {
            // Build and create user with random password via service to ensure encoding and emailing
            com.iims.iims.user.dto.TenantUserCreationRequest req = new com.iims.iims.user.dto.TenantUserCreationRequest();
            req.setEmail(application.getEmail());
            req.setFullName(application.getFirst_name() + " " + application.getLast_name());
            // Map applicant type to role
            com.iims.iims.user.entity.Role role = application.getApplicantType() == ApplicantType.STARTUP
                    ? com.iims.iims.user.entity.Role.STARTUP
                    : application.getApplicantType() == ApplicantType.MENTOR
                      ? com.iims.iims.user.entity.Role.MENTOR
                      : com.iims.iims.user.entity.Role.ALUMNI;
            req.setRole(role);
            // Create user under this tenant using a synthetic tenant admin context
            com.iims.iims.user.entity.User tenantAdminProxy = new com.iims.iims.user.entity.User();
            tenantAdminProxy.setTenantId(tenant.getId());
            com.iims.iims.user.entity.User created = userService.createTenantUser(req, tenantAdminProxy);

            // Create domain profile using same user id
            if (role == com.iims.iims.user.entity.Role.STARTUP) {
                // Get cohort and industry from the application form (not from responses)
                String cohortName = null;
                String industryName = null;
                if (application.getForm() != null) {
                    if (application.getForm().getCohort() != null) {
                        cohortName = application.getForm().getCohort().getName();
                    }
                    if (application.getForm().getIndustry() != null) {
                        industryName = application.getForm().getIndustry().getName();
                    }
                }

                com.iims.iims.profile.entity.StartupProfile sp = com.iims.iims.profile.entity.StartupProfile.builder()
                        .user(created)
                        .startupName(application.getFirst_name() + " " + application.getLast_name())
                        .industry(industryName)
                        .cohortName(cohortName)
                        .build();
                startupProfileRepository.save(sp);
            } else if (role == com.iims.iims.user.entity.Role.MENTOR) {
                com.iims.iims.profile.entity.MentorProfile mp = com.iims.iims.profile.entity.MentorProfile.builder()
                        .user(created)
                        .firstName(application.getFirst_name())
                        .lastName(application.getLast_name())
                        .build();
                mentorProfileRepository.save(mp);
            }
        }
    }

    return convertToDto(updatedApplication);
}
```

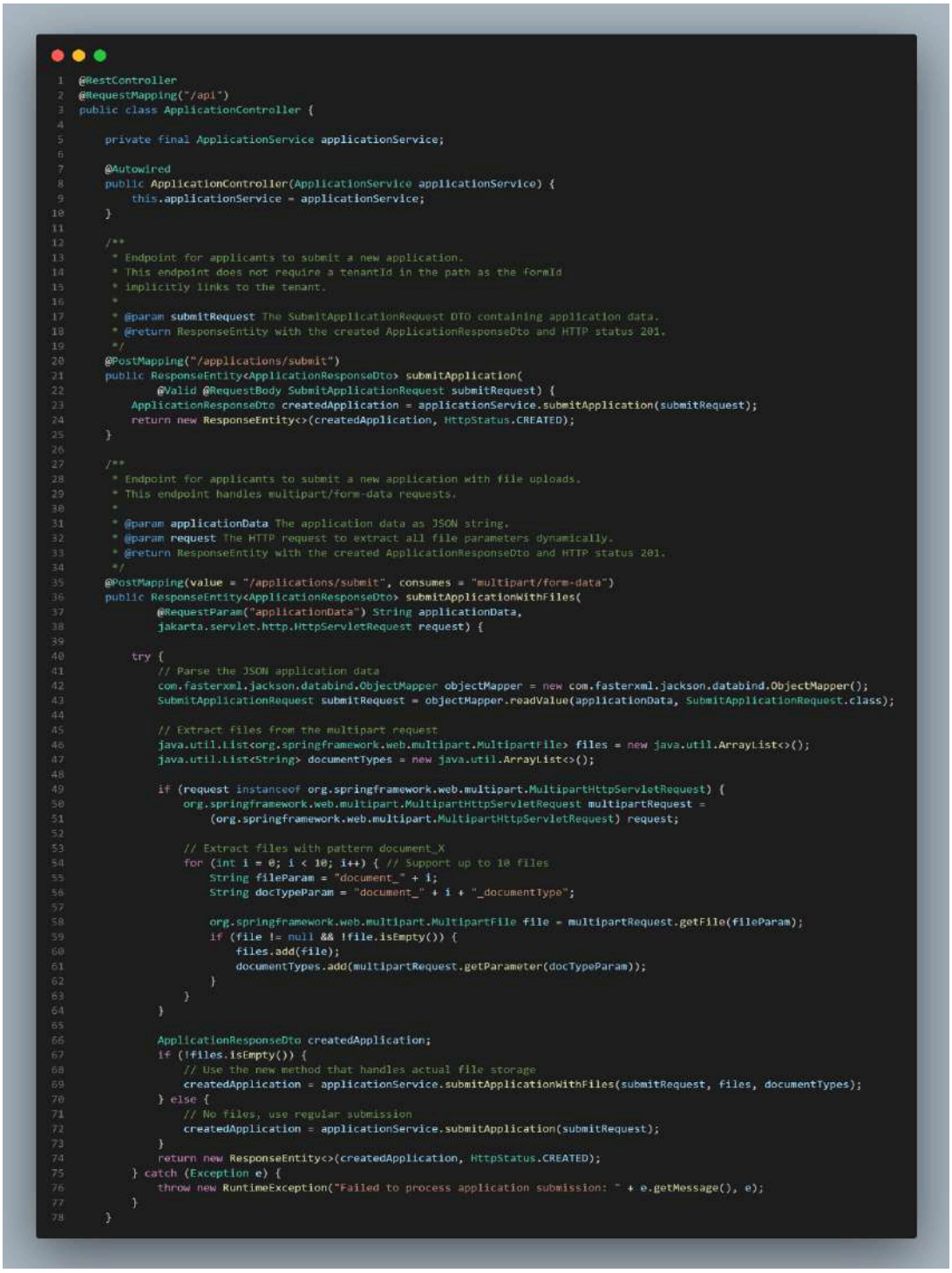
4.6. AuthService

The AuthService class implements authentication and authorization logic using object-oriented principles. This service demonstrates dependency injection through constructor parameters, showcasing loose coupling. The class uses the Builder pattern (via Lombok) for object creation, and acts as a facade coordinating between multiple components to provide a clean interface for authentication operations.

```java
@Service
@Transactional
@RequiredArgsConstructor
public class AuthService {

    private final UserRepository userRepo;
    private final PasswordEncoder encoder;
    private final JwtService jwtService;
    private final AuthenticationManager authManager;
    private final RefreshTokenService refreshTokenService;

    public AuthResponse authenticate(AuthRequest req) {
        try {
            authManager.authenticate(
                new UsernamePasswordAuthenticationToken(req.getEmail(), req.getPassword())
            );
            User user = userRepo.findByEmail(req.getEmail())
                .orElseThrow(() -> new RuntimeException("User not found"));

            String refreshToken = refreshTokenService.createOrUpdateRefreshToken(user).getToken();

            return new AuthResponse(
                jwtService.generateToken(user),
                refreshToken,
                user.getEmail(),
                user.getRole().name(),
                user.getFullName(),
                user.getId(),
                user.getTenantId() // Return tenantId for all users who have one
            );
        } catch (Exception e) {
            e.printStackTrace();
            throw new RuntimeException("Authentication failed: " + e.getMessage());
        }
    }

    public AuthResponse refreshAccessToken(String requestRefreshToken) {
        var refreshToken = refreshTokenService.findByToken(requestRefreshToken);

        if (refreshToken == null || refreshTokenService.isTokenExpired(refreshToken)) {
            throw new RuntimeException("Invalid or expired refresh token");
        }

        var user = refreshToken.getUser();
        var newAccessToken = jwtService.generateToken(user);

        // *** FIX: Update the existing token instead of creating a new one ***
        String newRefreshToken = refreshTokenService.createOrUpdateRefreshToken(user).getToken();

        return new AuthResponse(
            newAccessToken,
            newRefreshToken,
            user.getEmail(),
            user.getRole().name(),
            user.getFullName(),
            user.getId(),
            user.getTenantId() // Return tenantId for all users who have one
        );
    }

    public AuthResponse registerSuperAdmin(RegisterRequest req) {
        User admin = User.builder()
            .email(req.getEmail())
            .password(encoder.encode(req.getPassword()))
            .fullName(req.getFullName())
            .role(Role.SUPER_ADMIN)
            .build();
        userRepo.saveAndFlush(admin);

        // *** FIX: This is a new user, so a direct create is fine, but for consistency,
        // you could still use the createOrUpdateRefreshToken method.
        String refreshToken = refreshTokenService.createOrUpdateRefreshToken(admin).getToken();

        return new AuthResponse(
            jwtService.generateToken(admin),
            refreshToken,
            admin.getEmail(),
            admin.getRole().name(),
            admin.getFullName(),
            admin.getId(),
            null
        );
    }
}
```

4.7. ApplicationController

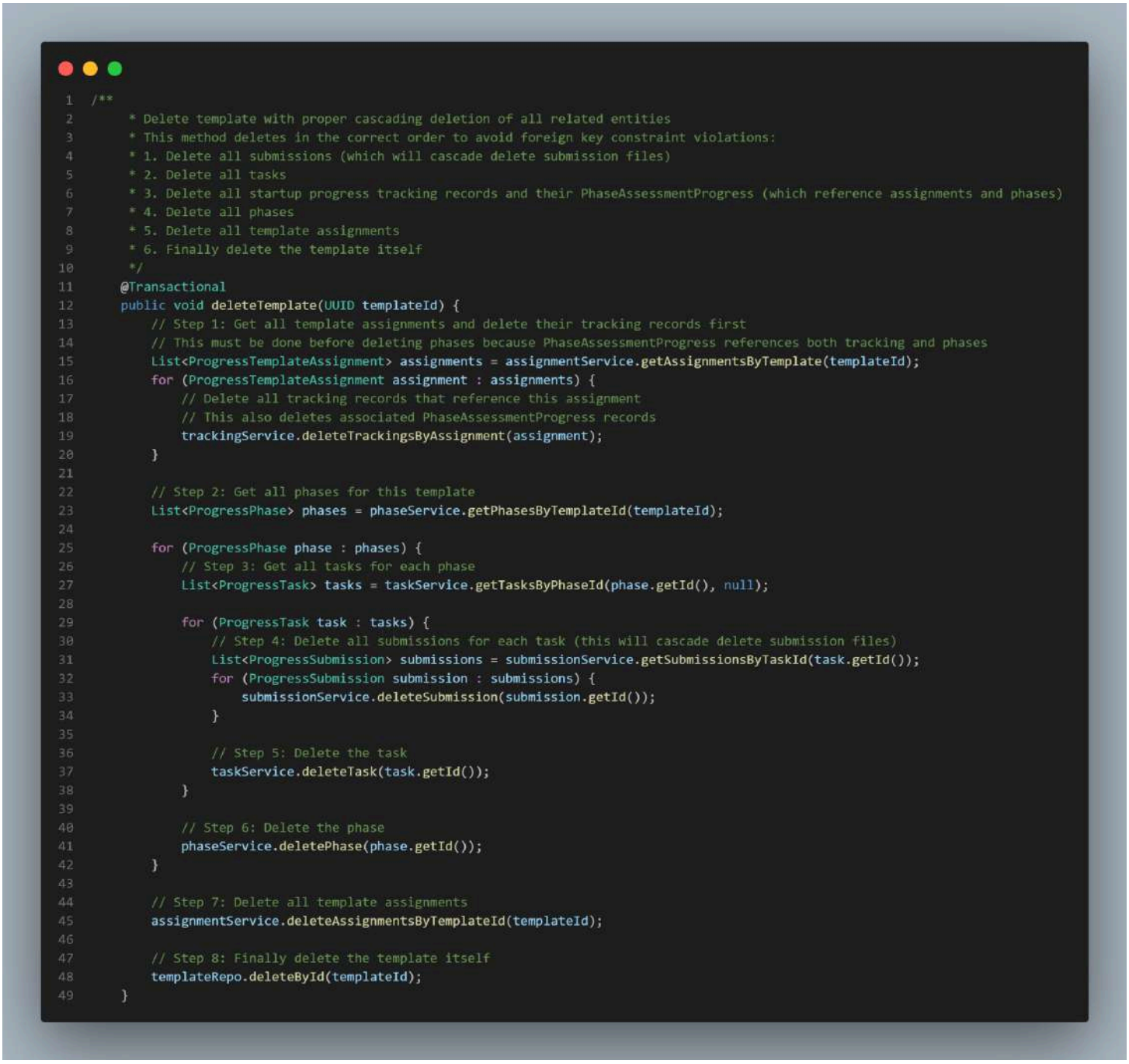
The ApplicationController class demonstrates REST API implementation with multiple HTTP methods (POST, GET, PUT), showcasing CRUD operations in object-oriented design. The class uses dependency injection through constructor injection, and the POST methods demonstrate request validation with @Valid annotation and delegation to the service layer.



```java
@RestController
@RequestMapping("/api")
public class ApplicationController {

    private final ApplicationService applicationService;

    @Autowired
    public ApplicationController(ApplicationService applicationService) {
        this.applicationService = applicationService;
    }

    /**
     * Endpoint for applicants to submit a new application.
     * This endpoint does not require a tenantId in the path as the formId
     * implicitly links to the tenant.
     *
     * @param submitRequest The SubmitApplicationRequest DTO containing application data.
     * @return ResponseEntity with the created ApplicationResponseDto and HTTP status 201.
     */
    @PostMapping("/applications/submit")
    public ResponseEntity<ApplicationResponseDto> submitApplication(
            @Valid @RequestBody SubmitApplicationRequest submitRequest) {
        ApplicationResponseDto createdApplication = applicationService.submitApplication(submitRequest);
        return new ResponseEntity<>(createdApplication, HttpStatus.CREATED);
    }

    /**
     * Endpoint for applicants to submit a new application with file uploads.
     * This endpoint handles multipart/form-data requests.
     *
     * @param applicationData The application data as JSON string.
     * @param request The HTTP request to extract all file parameters dynamically.
     * @return ResponseEntity with the created ApplicationResponseDto and HTTP status 201.
     */
    @PostMapping(value = "/applications/submit", consumes = "multipart/form-data")
    public ResponseEntity<ApplicationResponseDto> submitApplicationWithFiles(
            @RequestParam("applicationData") String applicationData,
            jakarta.servlet.http.HttpServletRequest request) {

        try {
            // Parse the JSON application data
            com.fasterxml.jackson.databind.ObjectMapper objectMapper = new com.fasterxml.jackson.databind.ObjectMapper();
            SubmitApplicationRequest submitRequest = objectMapper.readValue(applicationData, SubmitApplicationRequest.class);

            // Extract files from the multipart request
            java.util.List<org.springframework.web.multipart.MultipartFile> files = new java.util.ArrayList<>();
            java.util.List<String> documentTypes = new java.util.ArrayList<>();

            if (request instanceof org.springframework.web.multipart.MultipartHttpServletRequest) {
                org.springframework.web.multipart.MultipartHttpServletRequest multipartRequest =
                    (org.springframework.web.multipart.MultipartHttpServletRequest) request;

                // Extract files with pattern document_X
                for (int i = 0; i < 10; i++) { // Support up to 10 files
                    String fileParam = "document_" + i;
                    String docTypeParam = "document_" + i + "_documentType";

                    org.springframework.web.multipart.MultipartFile file = multipartRequest.getFile(fileParam);
                    if (file != null && !file.isEmpty()) {
                        files.add(file);
                        documentTypes.add(multipartRequest.getParameter(docTypeParam));
                    }
                }
            }

            ApplicationResponseDto createdApplication;
            if (!files.isEmpty()) {
                // Use the new method that handles actual file storage
                createdApplication = applicationService.submitApplicationWithFiles(submitRequest, files, documentTypes);
            } else {
                // No files, use regular submission
                createdApplication = applicationService.submitApplication(submitRequest);
            }
            return new ResponseEntity<>(createdApplication, HttpStatus.CREATED);
        } catch (Exception e) {
            throw new RuntimeException("Failed to process application submission: " + e.getMessage(), e);
        }
    }
```

4.8. Progress Tracking Service

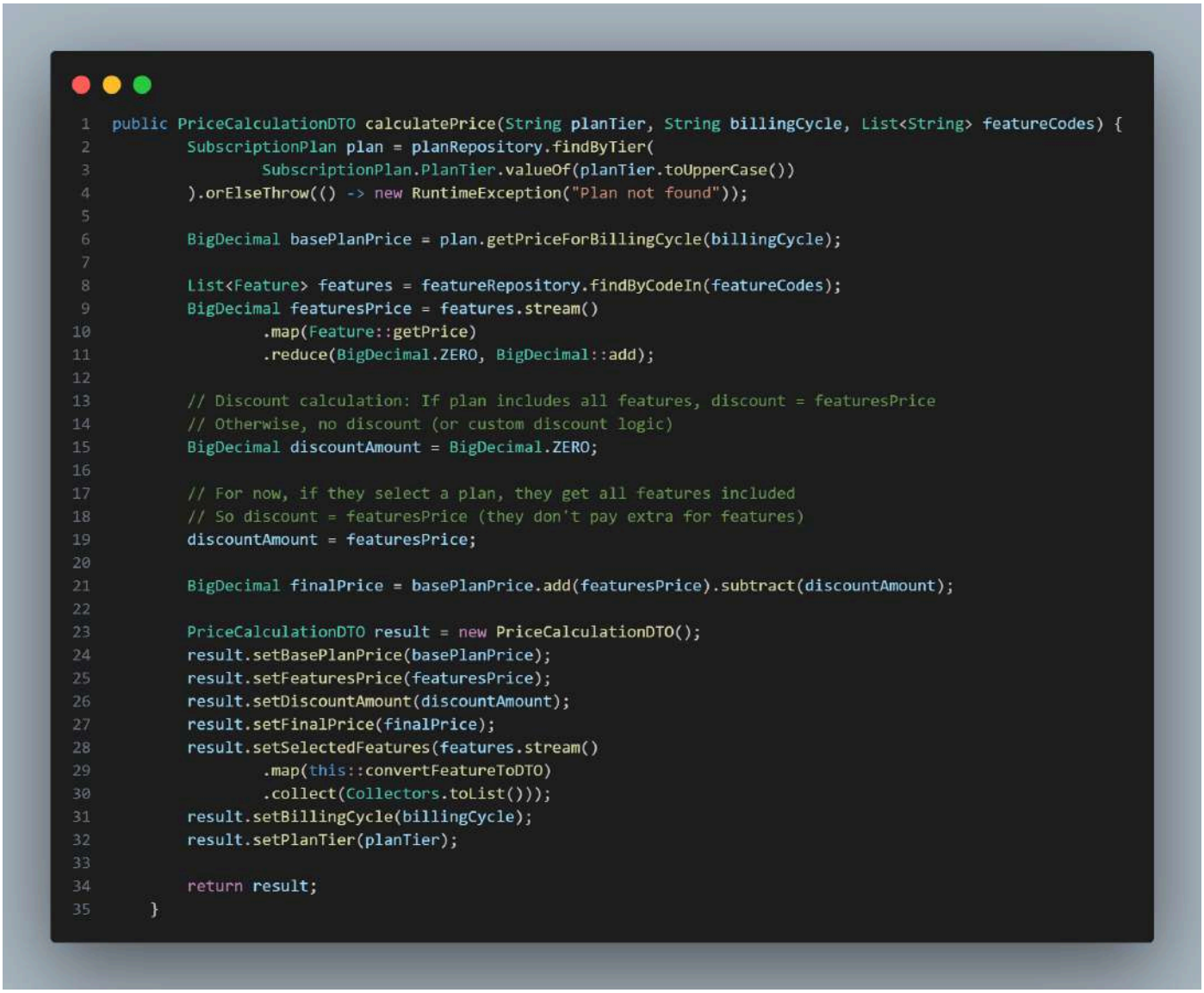
The ProgressTemplateService class demonstrates complex business logic for managing progress templates. This method demonstrates object-oriented design principles by coordinating between multiple service dependencies and managing complex relationships between entities. The comprehensive JavaDoc comments explain the deletion sequence, demonstrating best practices for documenting complex business logic.



```java
/**
     * Delete template with proper cascading deletion of all related entities
     * This method deletes in the correct order to avoid foreign key constraint violations:
     * 1. Delete all submissions (which will cascade delete submission files)
     * 2. Delete all tasks
     * 3. Delete all startup progress tracking records and their PhaseAssessmentProgress (which reference assignments and phases)
     * 4. Delete all phases
     * 5. Delete all template assignments
     * 6. Finally delete the template itself
     */
    @Transactional
    public void deleteTemplate(UUID templateId) {
        // Step 1: Get all template assignments and delete their tracking records first
        // This must be done before deleting phases because PhaseAssessmentProgress references both tracking and phases
        List<ProgressTemplateAssignment> assignments = assignmentService.getAssignmentsByTemplate(templateId);
        for (ProgressTemplateAssignment assignment : assignments) {
            // Delete all tracking records that reference this assignment
            // This also deletes associated PhaseAssessmentProgress records
            trackingService.deleteTrackingsByAssignment(assignment);
        }

        // Step 2: Get all phases for this template
        List<ProgressPhase> phases = phaseService.getPhasesByTemplateId(templateId);

        for (ProgressPhase phase : phases) {
            // Step 3: Get all tasks for each phase
            List<ProgressTask> tasks = taskService.getTasksByPhaseId(phase.getId(), null);

            for (ProgressTask task : tasks) {
                // Step 4: Delete all submissions for each task (this will cascade delete submission files)
                List<ProgressSubmission> submissions = submissionService.getSubmissionsByTaskId(task.getId());
                for (ProgressSubmission submission : submissions) {
                    submissionService.deleteSubmission(submission.getId());
                }

                // Step 5: Delete the task
                taskService.deleteTask(task.getId());
            }

            // Step 6: Delete the phase
            phaseService.deletePhase(phase.getId());
        }

        // Step 7: Delete all template assignments
        assignmentService.deleteAssignmentsByTemplateId(templateId);

        // Step 8: Finally delete the template itself
        templateRepo.deleteById(templateId);
    }
```

4.9. Subscription Service

The SubscriptionService class implements subscription management business logic using object-oriented principles. The calculatePrice method demonstrates complex business calculations including base plan pricing, feature pricing, and discount calculations. This method showcases encapsulation by handling all pricing logic within a single method, and demonstrates the use of Java Streams API for functional programming patterns. The method returns a DTO object, illustrating the Data Transfer Object pattern for clean data representation.



```java
public PriceCalculationDTO calculatePrice(String planTier, String billingCycle, List<String> featureCodes) {
        SubscriptionPlan plan = planRepository.findByTier(
                SubscriptionPlan.PlanTier.valueOf(planTier.toUpperCase())
        ).orElseThrow(() -> new RuntimeException("Plan not found"));

        BigDecimal basePlanPrice = plan.getPriceForBillingCycle(billingCycle);

        List<Feature> features = featureRepository.findByCodeIn(featureCodes);
        BigDecimal featuresPrice = features.stream()
                .map(Feature::getPrice)
                .reduce(BigDecimal.ZERO, BigDecimal::add);

        // Discount calculation: If plan includes all features, discount = featuresPrice
        // Otherwise, no discount (or custom discount logic)
        BigDecimal discountAmount = BigDecimal.ZERO;

        // For now, if they select a plan, they get all features included
        // So discount = featuresPrice (they don't pay extra for features)
        discountAmount = featuresPrice;

        BigDecimal finalPrice = basePlanPrice.add(featuresPrice).subtract(discountAmount);

        PriceCalculationDTO result = new PriceCalculationDTO();
        result.setBasePlanPrice(basePlanPrice);
        result.setFeaturesPrice(featuresPrice);
        result.setDiscountAmount(discountAmount);
        result.setFinalPrice(finalPrice);
        result.setSelectedFeatures(features.stream()
                .map(this::convertFeatureToDTO)
                .collect(Collectors.toList()));
        result.setBillingCycle(billingCycle);
        result.setPlanTier(planTier);

        return result;
    }
```

Chapter Five: Testing and Evaluation

The IIMS system follows a comprehensive testing strategy covering multiple testing levels to ensure system reliability and quality. The testing approach includes unit testing, integration testing, system testing, and user acceptance testing. Unit tests validate individual components and methods, while integration tests verify interactions between modules. System testing ensures end-to-end functionality, and user acceptance testing validates that the system meets business requirements. The testing strategy prioritizes critical functionality such as authentication, tenant management, and data security, ensuring these core features are thoroughly validated before deployment.

5.1. Authentication & Authorization

Example Test Case

- **TC-AUTH-001:** User Login with Valid Credentials
  - Scenario: Successful login with email and password
  - Expected: JWT token issued, user authenticated

- **TC-AUTH-002**: JWT Token Validation
  - Scenario: Access protected endpoint with valid token
  - Expected: Request succeeds, data returned

5.2. Tenant Management & Subscriptions

Example Test Case:

- **TC-TENANT-001:** Tenant Registration Application
  - Scenario: Submit tenant application with valid data
  - Expected: Application created, status PENDING

- **TC-SUBSCRIPTION-001:** Subscription Plan Management
  - Scenario: Create subscription plan with pricing tiers
  - Expected: Plan created successfully

5.3. Application Management

Example Test Case:

- **TC-APP-001**: Submit Application
  - Scenario: Startup submits application with required fields
  - Expected: Application created, status PENDING

- **TC-APP-002:** Review and Approve Application
  - Scenario: Tenant admin approves application
  - Expected: Application approved, user and profile created

5.4. Progress Tracking

Example Test Case:

- **TC-PROGRESS-001**: Create Progress Template
  - Scenario: Tenant admin creates progress tracking template
  - Expected: Template created with phases and tasks

- **TC-PROGRESS-002:** Task Submission and Review
  - Scenario: Startup submits task, mentor reviews
  - Expected: Task submitted, mentor can review and approve

5.5. Assessment Management

Example Test Case:

- TC-ASSESSMENT-001: Create Assessment
  - Scenario: Create assessment with questions and scoring
  - Expected: Assessment created successfully

- TC-ASSESSMENT-002: Submit Assessment Response
  - Scenario: User submits assessment answers
  - Expected: Response saved, score calculated

5.6. User Management

Example Test Case:

- TC-USER-001: Create User Account
  - Scenario: Tenant admin creates new user
  - Expected: User created with assigned role

- TC-USER-002: Role-Based Access Control
  - Scenario: User accesses endpoint based on role
  - Expected: Access granted/denied based on permissions

5.7. Non-Functional Requirements

Example Test Case:

- TC-NFR-001: Performance Testing
  - Scenario: System handles 100 concurrent users
  - Expected: Response time < 2 seconds

- TC-NFR-002: Security Testing
  - Scenario: SQL injection and XSS attack attempts
  - Expected: Attacks blocked, system secure

5.8. System Evaluation

The IIMS system was evaluated against functional requirements, performance criteria, and user acceptance criteria. Functional evaluation confirmed that all critical and high-priority features are implemented and working as specified. Performance evaluation demonstrated that the system meets response time requirements and can handle expected user loads. Security evaluation verified that authentication, authorization, and data protection mechanisms are properly implemented. Usability evaluation confirmed that the system interface is intuitive and user-friendly.

References

- **Sommerville, I. (2015).** *Software Engineering.* 10th Edition. Pearson Education.
- **Object Management Group (OMG).** *Unified Modeling Language (UML) Specification v2.5.1.* [Online] Available at: https://www.omg.org/spec/UML/
- **Larman, C. (2004).** *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design.* Prentice Hall.
- **Lucidchart.** *UML Diagram Tutorial: Sequence, Class, and Activity Diagrams.* [Online] Available at: https://www.lucidchart.com/pages/uml-diagram-tutorial
- **Nielsen, J. (1994).** *Usability Engineering.* Academic Press.
- **Fowler, M. (2004).** *UML Distilled: A Brief Guide to the Standard Object Modeling Language.* Addison-Wesley Professional.
- **GeeksforGeeks.** *Software Engineering: Rapid Application Development (RAD) Model.* [Online] Available at: https://www.geeksforgeeks.org/software-engineering-rapid-application-development-model-rad/
- **Spring Boot.** (2024). *Spring Boot Reference Documentation (v3.5.3)*. [Online] Available at: https://docs.spring.io/spring-boot/docs/current/reference/html/
- **React Teams.** (2024). *React Documentation: Thinking in React*. [Online] Available at: https://react.dev/learn
- **Oracle.** (2023). *Java Platform, Standard Edition 21 Documentation*. [Online] Available at: https://docs.oracle.com/en/java/javase/21/
- **PostgreSQL Global Development Group.** *PostgreSQL 16 Documentation*. [Online] Available at: https://www.postgresql.org/docs/
- **SpringDoc.** (2024). *SpringDoc OpenAPI UI Documentation (v2.7.0)*. [Online] Available at: https://springdoc.org/
- **Tailwind Labs.** *Tailwind CSS Documentation*. [Online] Available at: https://tailwindcss.com/docs

Appendices

Appendix A: Samples of Working Forms

Manual documents currently used by incubation centers that the new system will automate.

- **Manual Intake/Application Form:** A paper-based template used by startups to submit their business ideas, founder details, and industry type.
- **Monthly Mentor Progress Report:** A checklist used by mentors to manually grade startup milestones before the digital dashboard.
- **Cohort Enrollment Sheet:** An Excel-based tracker used to organize startups into specific batches or industry sectors.

Appendix B: Requirement Gathering Instruments

These instruments represent the interviews and surveys conducted by the group (Arsema, Tsion, Rediet, and Tsedenya) to define system requirements.

1. **Interview Guide for Center Admins:**
    a. What is the current manual process for opening a "Call for Applications"?
    b. How do you determine the price for custom features added to a subscription?
    c. What specific data must be included in the "Global Export" for your stakeholders?
2. **Questionnaire for Mentors:**
    a. How do you currently schedule meetings with startups?
    b. Would a task-tracking board (To-do, Doing, Done) help you monitor startup progress?
    c. What metrics do you use to score a startup's pre-incubation level?
3. **Survey for Startup Founders:**
    a. What is the biggest challenge in your current application process?
    b. Would you find a chat feature with your mentor more useful than email?
    c. What information would you like to see on your "Incubation Progress" chart?

Appendix C: User Interface (UI) Mockups

Visual wireframes showing the layout and navigation of the system.

- **Admin Control Center:** A mockup showing the sidebar navigation, the "Tenant Plan" toggle switch, and the analytics overview for all cohorts.
- **Subscription & Pricing Interface:** A screen where admins can select "Modular Features" (e.g., Google Calendar, Chat) and see the price update in real-time.
- **Startup Dashboard:** A visual mockup of the startup's view, featuring a radar chart comparing **Pre-incubation** vs. **Post-incubation** scores.
- **Mentor Task & Chat View:** A mockup showing the integrated workspace where mentors assign tasks and communicate via the built-in chat module.
- **C.5 Public Landing Page:** The external-facing page showing active "Application Calls" with a countdown timer.

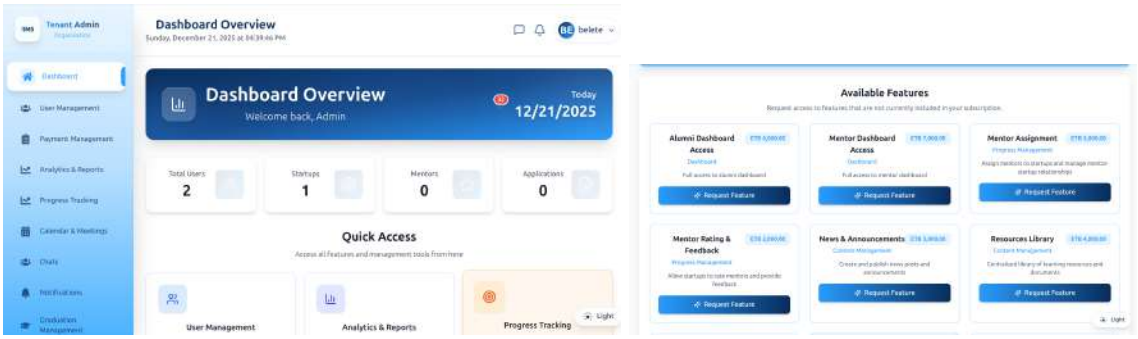


Figure 2.18 Admin Dashboard and Feature Management

Appendix D: Glossary of Terms

Definition of terms used within this document.

- **Tenant:** A specific Incubation Center or Hub using the platform.
- **Super Admin:** The system owner who manages all Tenants and Subscriptions.
- **Modular Feature:** An optional software component that can be added to a subscription plan for an extra fee.
- **Cohort:** A group of startups following the same incubation timeline.

Appendix E: Technology Stack & Infrastructure

The system architecture follows a decoupled client-server model designed for scalability and real-time interaction.

- **Backend:** Java 21 with Spring Boot 3.5.3 (Security, Data JPA, Web, WebSocket).
- **Frontend:** React 19.1.0, Tailwind CSS for styling, and Recharts for analytics dashboards.
- **Database:** PostgreSQL 16 hosted on Neon Serverless Postgres.
- **Real-time:** SockJS and STOMP for live chat and notifications.

Appendix F: API Documentation (Swagger/OpenAPI)

The backend exposes a RESTful API, documented using SpringDoc OpenAPI. This allows for interactive testing and clear endpoint definitions.

- **Swagger UI URL:** https://iims-backend.onrender.com/swagger-ui.html
- **OpenAPI Spec:** /v3/api-docs
- **Key Modules documented:** Authentication, Tenant Management, Subscription Plans, Progress Tracking, and Meeting Scheduling.

Appendix F: Third-Party Service Integration

- **Google Calendar API:** Used for automated scheduling of mentor-startup meetings and generating Google Meet links.
- **Gmail SMTP:** Handles transactional emails, including registration confirmations and system alerts.